

# Irene Documentation

***Release 1.5.0***

**Mehdi Ghasemi**

**Oct 27, 2026**

# GETTING STARTED


**1 Introduction** **3**
- 1.1 Documentation Scope . . . 3
- 1.2 Requirements and dependencies . . . 3
- 1.3 Download . . . 5
- 1.4 Installation . . . 5
- 1.5 License . . . 5

**2 Architecture and Method Guide** **7**
- 2.1 From Polynomial Expressions to Group-Rings . . . 7
- 2.2 Module Layers . . . 7
- 2.3 Choosing a Relaxation Family . . . 8
- 2.4 Typical Workflow . . . 8

**3 Legacy API Migration Guide** **9**
- 3.1 Problem Construction . . . 9
- 3.2 Objective and Constraints . . . 9
- 3.3 Moment Constraints . . . 10
- 3.4 Solver Selection and Solving . . . 10
- 3.5 Probability and Moment Settings . . . 10
- 3.6 SOS Decomposition . . . 11
- 3.7 Solution Extraction . . . 11
- 3.8 Complete Example: Legacy vs Modern . . . 11

**4 Theory Overview** **13**
- 4.1 The optimization problem . . . 13
- 4.2 Step 1 – the algebraic layer . . . 13
- 4.3 Step 2 – moment/SOS hierarchy . . . 14
- 4.4 Step 3 – structure exploitation . . . 14
- 4.5 Step 4 – GP and SONC alternatives . . . 14
- 4.6 Step 5 – combining certificates . . . 15
- 4.7 Step 6 – leaving the polynomial world . . . 15
- 4.8 Theory-to-code map . . . 16
- 4.9 Further reading . . . 16

**5 Group-Ring Foundations** **17**
- 5.1 Algebraic Setting . . . 17
- 5.2 Commutative Semigroup and Semigroup Algebra . . . 17
- 5.3 Support and Geometry . . . 17
- 5.4 Differential Operators . . . 19
- 5.5 Why This Matters for POP . . . 20

**6 Problem Representation** **21**
6.1 Problem Template . . . 21
6.2 Core Responsibilities . . . 21
6.3 Degree Conventions . . . 21
6.4 Key Methods . . . 21
6.5 From Equations to Methods . . . 22
6.6 Delta Sets and Newton Data . . . 22
6.7 Minimal Construction Pattern . . . 22

**7 Semidefinite Programming Relaxations** **23**
7.1 Moment and Localizing Matrices . . . 23
7.2 API Overview . . . 23
7.3 Solver Routing . . . 24
7.4 Practical Example: Bounded Polynomial . . . 24
7.5 Hierarchy Convergence . . . 25
7.6 References . . . 25

**8 Geometric Programming Relaxations** **27**
8.1 Theory to Implementation . . . 27
8.2 Interpretation of the Matrix Transform . . . 27
8.3 Constraint Families in `solve` . . . 28
8.4 Lower-Bound Meaning . . . 28
8.5 Practical Notes . . . 28
8.6 Runable Example . . . 28

**9 SONC Relaxations** **29**
9.1 Formulation Focus . . . 29
9.2 Variable Roles . . . 29
9.3 Branching on $\lambda_0^{(\beta)}$ . . . 30
9.4 Implementation Stages in `SONCRelaxations` . . . 30
9.5 Theory-to-Code Anchors . . . 30
9.6 Geometric Interpretation . . . 30
9.7 Repository Anchors . . . 31
9.8 Runnable Example . . . 31

**10 SOS+SONC Relaxations** **33**
10.1 Theory . . . 33
10.2 Implementation in Irene . . . 34
10.3 Implementation Pipeline . . . 37
10.4 Relation to the Mean Polynomial Hierarchy . . . 37
10.5 Example . . . 37
10.6 Test Suite . . . 38
10.7 Further Reading . . . 38

**11 Optimization** **39**
11.1 Notation Bridge to Geometric and SONC Chapters . . . 40
11.2 Beyond SDP: Alternative Lower-Bound Mechanisms . . . 41
11.3 Conic Duality and Nonnegativity Certificates . . . 43

**12 Border Basis Theory and Implementation** **45**
12.1 Theory . . . 45
12.2 API Reference . . . 46
12.3 Integration with SDP Hierarchies . . . 47
12.4 Practical Notes . . . 47
12.5 References . . . 48

**13 Sparse Moment Method (SparseMomentSDP) 49**
13.1 Theory . . . 49
13.2 Pipeline . . . 50
13.3 Quick Example . . . 50
13.4 Public API . . . 50
13.5 Numerical Notes . . . 51
13.6 Integration Notes . . . 51

**14 Correlative Sparsity Detection 53**
14.1 Theory . . . 53
14.2 API Reference . . . 54
14.3 Integration with Relaxation Pipeline . . . 54
14.4 Practical Notes . . . 55
14.5 References . . . 55

**15 Newton Polytope Pruning 57**
15.1 Theory . . . 57
15.2 API Reference . . . 58
15.3 Integration with Relaxation Pipeline . . . 59
15.4 Practical Notes . . . 59
15.5 References . . . 60

**16 Unified Relaxation API 61**
16.1 RelaxationEngine . . . 61
16.2 RelaxationConfig . . . 62
16.3 Two-Stage Hybrid Monoid-Graph Reduction Theorem . . . 63
16.4 compare_all() . . . 65

**17 Approximating Optimum Value 67**
17.1 Using `pyProximation.OrthSystem` . . . 67

**18 Non-Polynomial Optimization (NonPOPSDP) 77**
18.1 Pipeline . . . 77
18.2 Quick Example . . . 77
18.3 API . . . 78
18.4 Numerical Fixes in the Port (vs original Irene) . . . 78
18.5 Integration Notes . . . 78

**19 Differential SDP and Mean Relaxations 79**
19.1 Differential SDP Connection . . . 79
19.2 Mean Polynomial Forms . . . 79
19.3 Convergence Theory: CGIK Framework and Archimedean Conditions . . . 80
19.4 API Overview . . . 80
19.5 SymEngine Bridging . . . 81
19.6 Practical Notes . . . 81
19.7 References . . . 81

**20 CVXPY Solver Layer 83**
20.1 Architecture . . . 83
20.2 API Reference . . . 84
20.3 Performance Notes . . . 85

**21 Benchmarks and Performance Evaluation 87**
21.1 Benchmark Problem Gallery . . . 87
21.2 Phase 3 Optimization Benchmarks . . . 89

21.3 Constrained Optimization Examples . . . 90
21.4 Performance Profiling Tools . . . 91
21.5 Solver Routing Behavior . . . 91
21.6 References . . . 91

**22 Examples and Validation** **93**
22.1 Recommended Example Sequence . . . 93
22.2 API Quick Reference . . . 93
22.3 SDP Example . . . 93
22.4 Geometric Programming Example . . . 94
22.5 SONC Examples . . . 94
22.6 Example 3.3 Traceability . . . 94
22.7 Benchmark Gallery System . . . 94
22.8 Backend Comparison Benchmark . . . 94
22.9 Additional Examples . . . 95
22.10 Regression Validation . . . 95

**23 pyProximation (lean)** **97**
23.1 Requirements and dependencies . . . 97
23.2 Download . . . 97
23.3 Installation . . . 97
23.4 License . . . 97

**24 Measures** **99**
24.1 Density function . . . 99
24.2 Integrals . . . 100
24.3 $p$-norms . . . 100
24.4 Drawing samples . . . 101

**25 Hilbert Spaces** **103**
25.1 Orthonormal system of functions . . . 103
25.2 OrthSystem . . . 104
25.3 Approximation . . . 106
25.4 Rational Approximation . . . 108

**26 Interpolation** **109**
26.1 Lagrange interpolation . . . 109
26.2 Lagrange interpolation as an $L^2$-orthogonal projection . . . 110

**27 Code Documentation** **113**

**28 Code Documentation** **117**
28.1 SOS+SONC Relaxation Framework . . . 140
28.2 Unified Relaxation API . . . 149
28.3 NOTATION (paper sections referenced) . . . 152
28.4 Public API . . . 171
28.5 Usage example . . . 171

**29 Doctest Integration** **175**

**30 Revision History** **177**

**31 Appendix** **179**
31.1 Global Notation Index . . . 179
31.2 pyProximation . . . 180
31.3 pyOptSparse . . . 180

31.4 LaTeX support . . . 182

**32 References** **183**
32.1 The truncated moment problem on algebras . . . 183
32.2 Moment/SOS hierarchy and polynomial optimization . . . 183
32.3 Geometric-programming bounds . . . 183
32.4 SONC relaxations . . . 184
32.5 SOS+SONC hybrid certificates . . . 184
32.6 Mean forms and differential SDP . . . 184
32.7 Structure exploitation and numerics . . . 185
32.8 Archival versions . . . 185

**33 Indices and tables** **187**

**Bibliography** **189**

**Python Module Index** **191**

**Index** **193**

Irene is a Python toolkit for polynomial optimization via semidefinite programming, geometric programming, and SONC/SOS hierarchies. It implements Lasserre's moment-SDP hierarchy, circuit-based SONC relaxations, mean polynomial forms, correlative sparsity detection, Newton polytope pruning, and differential SDP extensions.

Contents:

CHAPTER

ONE

# INTRODUCTION

This is a brief documentation for using *Irene*. Irene was originally written to find reliable approximations for optimum value of an arbitrary optimization problem. It implements a modification of Lasserre's SDP Relaxations based on generalized truncated moment problem to handle general optimization problems algebraically.

## 1.1 Documentation Scope

The documentation now covers three complementary pathways for constrained polynomial optimization:

1. SDP hierarchy methods (moment/SOS based).
2. Geometric programming relaxations.
3. SONC relaxations.

The recommended reading path is:

1. *Theory Overview* for the mathematical chain of constructions (problem setup, algebraic layer, hierarchy, GP/SONC certificates, differential extensions).
2. *Architecture and Method Guide* for conceptual map.
3. *Group-Ring Foundations* and *Problem Representation* for representation layer.
4. *Semidefinite Programming Relaxations*, *Geometric Programming Relaxations*, and *SONC Relaxations* for method-specific formulations.

Symbol conventions are collected in the *Global Notation Index* at the end of *Appendix*; consult it before reading the theory or method chapters.

## 1.2 Requirements and dependencies

This is a python package, so clearly python is an obvious requirement. Irene relies on the following packages:

- **for vector calculations:**
  - NumPy.
  - SciPy.
- **for symbolic computations:**
  - SymPy.
  - SymEngine (primary engine; SymPy fallback).
- **for semidefinite optimization (choose one path):**

- **CVXPY** (recommended default) with CLARABEL backend,
- cvxopt (legacy native path),
- dsdp,
- sdpa,
- csdp.

### 1.2.1 Dependency Matrix by Method Family

| Method family | Core Python packages | Optional packages | External solver requirement |
|---|---|---|---|
| SDP relaxations | numpy, scipy, sympy, symengine | cvxpy, clarabel | one of cvxpy (default), cvxopt, dsdp, sdpa, csdp |
| Geometric relaxations | numpy, scipy, sympy | gpkit | gpkit-supported GP backend |
| SONC relaxations | numpy, scipy, sympy | gpkit | gpkit-supported GP backend |

### 1.2.2 Solver Prerequisites

Before running examples, verify available solvers from Python:

```
from Irene.base import base
print(base().AvailableSDPSolvers())
```

### 1.2.3 Quick Validation Workflow

After installation, the following commands provide a practical smoke test:

```
python benchmarks/Rosenbrock.py
python benchmarks/GPExample.py
python benchmarks/SONCExample.py
python -m pytest tests/ -q
```

### 1.2.4 Solver Troubleshooting

Common runtime signatures and first actions:

1. `AvailableSDPSolvers()` returns an empty list.

   This indicates that no configured SDP backend is currently reachable. Install at least one supported solver and verify it is available on `PATH` (or configured in solver path settings on platforms that require explicit paths).

2. `RuntimeError: GP solve failed` or `RuntimeError: SONC GP solve failed`.

   These messages usually indicate missing GP backend support, an unavailable solver, or an infeasible/numerically unstable relaxation for the selected formulation. Start with the shipped examples, lower verbosity, and simplified instances.

3. `ModuleNotFoundError: No module named 'gpkit'`.

   Install `gpkit` before running geometric or SONC examples.

4. SDP solve runs but returns non-optimal status.

   Try another supported SDP solver, inspect constraints for scaling issues, and compare with a lower relaxation order before increasing model complexity.

## 1.3 Download

Irene can be obtained from GitHub.

## 1.4 Installation

**Using ``venv`` (recommended)**:

```
cd Irene
python3 -m venv .venv
source .venv/bin/activate  # On Windows: .venv\Scripts\activate
pip install -e ".[dev]"
```

**Using ``uv`` (faster alternative)**:

```
cd Irene
uv venv
source .venv/bin/activate
uv pip install -e ".[dev]"
```

Both methods create an editable installation with development dependencies. The virtual environment isolates solver backends and avoids system-wide conflicts.

### 1.4.1 Documentation

The documentation of *Irene* is prepared via Sphinx.

To compile html version of the documentation run:

```
cd doc
make html
```

To make a pdf file, subject to existence of `latexpdf` run:

```
cd doc
make latexpdf
```

## 1.5 License

*Irene* is distributed under MIT license:

### 1.5.1 MIT License

Copyright (c) 2016-2026 Mehdi Ghasemi

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense,

and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

CHAPTER

# TWO

# ARCHITECTURE AND METHOD GUIDE

The current codebase supports multiple constrained polynomial optimization pathways. Historically, the documentation emphasized SDP hierarchies. The present structure extends this to include geometric and SONC methods on the same algebraic backbone.

## 2.1 From Polynomial Expressions to Group-Rings

Instead of treating input only as classical polynomial expressions, Irene builds optimization models through commutative semigroups and their semigroup algebras. This representation supports:

1. Symbolic manipulation of monomial support and exponents.
2. Geometric operations on Newton polytopes.
3. Differential operators through user-provided derivation maps.

The shift in perspective is important: instead of viewing optimization only as operations in $\mathbb{R}[x_1, \ldots, x_n]$, Irene treats expressions as elements of a semigroup algebra that can be converted to geometric data (supports, exponent tuples, convex combinations) used directly in GP and SONC routines.

## 2.2 Module Layers

The codebase is organized into six abstraction layers, each with clear boundaries:

1. **Symbolic Engine**: `symbolic_engine.py` — SymEngine primary / SymPy fallback router for all polynomial operations (expand, Gröbner basis, Poly API, matrix construction).
2. **Algebra layer**: `grouprings.py` — commutative semigroups, semigroup algebras, and derivation operators.
3. **Problem layer**: `program.py` — optimization problem definition with objective and constraint management.
4. **Relaxation layer**: `relaxations.py` (SDP hierarchy), `geometric.py` (GP relaxations), `sonc.py` (SONC circuit polynomials), `sosonc.py` (SOS+SONC combined bounds). Unified access via `relaxation_api.py` (`RelaxationEngine` + `compare_all()`).
5. **Structural optimization**: `border_basis.py` (border basis for quotient algebras), `sparsity.py` (correlative sparsity detection), `newton_polytope.py` (Newton polytope pruning of monomial bases).
6. **Solver layer**: `sdp.py` (native SDP solver interface), `cvxpy_solver.py` (CVXPY DCP layer bridging to CLARABEL/SCS/CVXOPT backends), `dsdp.py` (DSDP mean relaxation with SymEngine bridge).
7. **Tooling**: `telemetry.py` (timing/decoration utilities), `matrices.py` (moment/localizing matrix construction), `invariant.py` (group-invariant polynomial optimization).

## 2.3 Choosing a Relaxation Family

Use SDP relaxations when you need a moment/SOS hierarchy and semidefinite certificates. Use geometric relaxations when the transformed formulation is naturally handled by GP. Use SONC relaxations for sparse circuit-structured formulations in constrained settings.

Table 1: Method Selection Snapshot

| **Method**<br>• Primary object<br>• Typical strength<br>• Main module |
|---|
| **SDP hierarchy**<br>• Moment and localizing matrices<br>• Certified lower bounds via SOS/moment conditions<br>• `relaxations.py` and `sdp.py` |
| **Geometric relaxation**<br>• Posynomial/signomial GP model<br>• Scalable lower bounds for suitable transformed instances<br>• `geometric.py` |
| **SONC relaxation**<br>• Circuit polynomial constraints<br>• Sparse constrained formulations with barycentric structure<br>• `sonc.py` |

## 2.4 Typical Workflow

1. Define algebra generators and build expressions in a semigroup algebra.
2. Form an optimization problem with objective and constraints.
3. Select relaxation family (SDP, GP, or SONC).
4. Solve and interpret a certified lower bound.

CHAPTER

THREE

# LEGACY API MIGRATION GUIDE

This chapter maps every construct from the original Irene API (`SDPRelaxations`, `Mom()`, `Probability=False`, etc.) to the modern Irene pipeline (`OptimizationProblem` + `RelaxationEngine`).

The legacy API remains available for backward compatibility — existing code will continue to run. However, new development should use the modern pipeline for its unified configuration, reduction pipeline integration, and consistent result types.

## 3.1 Problem Construction

## 3.2 Objective and Constraints

Listing 1: Legacy

```
from sympy import symbols
x, y, z = symbols('x y z')
Rlx = SDPRelaxations([x, y, z])
Rlx.SetObjective(-2*x + y - z)
Rlx.AddConstraint(x + y + z <= 4)
Rlx.AddConstraint(x >= 0)
```

Listing 2: Modern

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem

sg = CommutativeSemigroup(['x', 'y', 'z'])
sga = SemigroupAlgebra(sg)
x, y, z = sga['x'], sga['y'], sga['z']

prog = OptimizationProblem(sga)
prog.set_objective(-2*x + y - z)
prog.add_constraint(x + y + z <= 4)
prog.add_constraint(x >= 0)
```

## 3.3 Moment Constraints

Table 1: Legacy → Modern Mapping: Moment Constraints

| Legacy API | Modern API |
|---|---|
| `Rlx.MomentConstraint(Mom(x*y) >= 0.5)` | `prog.add_moment_constraint(...)` |
| `Rlx.MomentConstraint(Mom(x**2) == 1/3)` | `prog.add_moment_constraint(..., equality=True)` |

## 3.4 Solver Selection and Solving

Table 2: Legacy → Modern Mapping: Solving

| Legacy API | Modern API |
|---|---|
| `Rlx.SetSDPSolver('dsdp')` | `RelaxationEngine(prog, solver='dsdp')` |
| `Rlx.InitSDP()` | Automatic on `engine.solve()` |
| `Rlx.Minimize()` | `engine.solve('sos')` |
| `print(Rlx.Solution)` | `print(result)` / `result.value` / `result.status` |
| `Rlx.Solution[x*y]` | Result object attributes — see `RelaxResult` |

## 3.5 Probability and Moment Settings

Table 3: Legacy → Modern Mapping: Settings

| Legacy API | Modern API |
|---|---|
| `Rlx.Probability = False` | Configured through `OptimizationProblem` problem-level settings |
| `Rlx.PSDMoment = True` | Always True in modern API (PSD constraint is always enforced) |
| `Rlx.ErrorTolerance` | Configured through solver-specific tolerance parameters (see *CVXPY Solver Layer*) |

## 3.6 SOS Decomposition

Listing 3: Legacy

```
Rlx.Minimize()
V = Rlx.Decompose()
# V = {0: [a01, a02, ...], 1: [a11, ...], ...}
sos = expand(Rlx.ReduceExp(sum([p**2 for p in V[0]])))
```

Listing 4: Modern

```
result = engine.solve('sos')
# result.certificate contains the SOS decomposition
# result.certificate = {'f_sos': ..., 'f_sonc': ...}
```

## 3.7 Solution Extraction

Table 4: Legacy → Modern Mapping: Solution Extraction

| Legacy API | Modern API |
|---|---|
| `Rlx.Solution.ExtractSolution('LH', card)` | `result` attributes; see `RelaxResult.solver_info` |
| `Rlx.Solution.ExtractSolution('scipy', card)` | Configured through solver backend options |
| `Rlx.Solution.Support` | `result.solver_info` dictionary |

## 3.8 Complete Example: Legacy vs Modern

Listing 5: Legacy (original Irene)

```
from sympy import symbols
from Irene import SDPRelaxations, Mom

x, y, z = symbols('x y z')
Rlx = SDPRelaxations([x, y, z])
Rlx.SetObjective(-2*x + y - z)
Rlx.AddConstraint(24 - 20*x + 9*y - 13*z + 4*x**2
                  - 4*x*y + 4*x*z + 2*y**2 - 2*y*z + 2*z**2 >= 0)
Rlx.AddConstraint(x + y + z <= 4)
Rlx.AddConstraint(3*y + z <= 6)
Rlx.MomentsOrd(3)
Rlx.SetSDPSolver('dsdp')
Rlx.InitSDP()
Rlx.Minimize()
print(Rlx.Solution)
```

Listing 6: Modern (Irene)

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.relaxation_api import RelaxationEngine
```

(continues on next page)

(continued from previous page)

```
from Irene.relaxations import RelaxationConfig

sg = CommutativeSemigroup(['x', 'y', 'z'])
sga = SemigroupAlgebra(sg)
x, y, z = sga['x'], sga['y'], sga['z']

prog = OptimizationProblem(sga)
prog.set_objective(-2*x + y - z)
prog.add_constraint(24 - 20*x + 9*y - 13*z + 4*x**2
                    - 4*x*y + 4*x*z + 2*y**2 - 2*y*z + 2*z**2 >= 0)
prog.add_constraint(x + y + z <= 4)
prog.add_constraint(3*y + z <= 6)
prog.add_constraint(x >= 0)
prog.add_constraint(x <= 2)
prog.add_constraint(y >= 0)
prog.add_constraint(z >= 0)
prog.add_constraint(z <= 3)

config = RelaxationConfig(
    reduction_method="newton_polytope",
    monomial_pruning=True,
)
engine = RelaxationEngine(prog, order=3, solver='dsdp', config=config)
result = engine.solve('sos')
print(f"Lower bound: {result.value:.8f}")
print(f"Status: {result.status}")
```

CHAPTER

FOUR

# THEORY OVERVIEW

This chapter is the entry point to the mathematics behind Irene. It states the objects each relaxation family builds, in the order the package constructs them, and points to the full treatment on the corresponding chapter page. Nothing here is implementation detail: for API-level material start at *Introduction*, and for symbol conventions see the *Global Notation Index* in *Appendix*.

The underlying manuscript is cited as *the companion paper* below; its Section 2 contains the same chain of constructions in archival form.

## 4.1 The optimization problem

Let $X$ be a nonempty topological space and $A$ a unital sub-algebra of continuous functions on $X$ separating points of $X$. Irene addresses

$$\begin{aligned} \min \quad & f(x) \\ \text{subject to} \quad & g_i(x) \geq 0, \qquad i = 1, \dots, m, \end{aligned} \tag{4.1}$$

with feasibility set $K = \{x \in X : g_i(x) \geq 0,\ i = 1, \dots, m\}$ and optimum value $\rho$. Writing $\mathcal{M}_1^+(K)$ for the probability Borel measures supported on $K$, the value coincides with the moment formulation

$$\rho = \inf_{\mu \in \mathcal{M}_1^+(K)} \int f \, d\mu = \inf_{\substack{L(Q) \geq 0 \\ L(1) = 1}} L(f), \tag{4.2}$$

where $Q_{\mathbf{g}}$ is the quadratic module generated by the constraints, $\sigma_0 + \sigma_1 g_1 + \cdots + \sigma_m g_m$ with each $\sigma_i$ a sum of squares in $A$. Equality of the three quantities in (4.2) is what turns an optimization problem into a *truncated moment problem*, and it requires the Archimedean hypothesis $M \pm h \in Q$ for all $h \in A$ and some $M > 0$; see *Optimization* for the full derivation, including the Marshall/Haviland and Jacobi representation steps.

The decisive point for the rest of the package: Irene does **not** assume $A = \mathbb{R}[x_1, \dots, x_n]$. Any unital commutative algebra works, and $K$ only has to be bounded. That is the generalization the whole design rests on.

## 4.2 Step 1 – the algebraic layer

Problems are represented over the semigroup algebra $\mathbb{R}[S]$ of a finitely generated commutative semigroup $S$, so that

$$f = \sum_{\alpha \in \operatorname{supp}(f)} c_\alpha \, \alpha, \qquad c_\alpha \in \mathbb{R}, \tag{4.3}$$

with $\alpha$ a semigroup element rather than a monomial in fixed generators. Quotient structures are handled by reduction in the semigroup representation. This buys two things at once: exponent vectors are available

directly for convex geometry, and the same representation feeds the SDP, GP and SONC families. Registered derivations extend the layer with Leibniz-rule maps $D : \mathbb{R}[S] \to \mathbb{R}[S]$.

See *Group-Ring Foundations* for the group-ring construction and *Problem Representation* for `OptimizationProblem`, the object that carries $f$, the $g_i$ and the even-degree convention used to build relaxations.

## 4.3 Step 2 – moment/SOS hierarchy

At relaxation order $t$, introduce moment variables $y_\alpha$ on $\Lambda_t = \{\alpha : |\alpha| \le t\}$ and the moment and localizing matrices

$$M_t(y)_{\alpha,\beta} = y_{\alpha+\beta}, \qquad M_t(g_i y)_{\alpha,\beta} = \sum_\gamma h_{i,\gamma}\, y_{\alpha+\beta+\gamma}, \tag{4.4}$$

for $g_i = \sum_\gamma h_{i,\gamma} x^\gamma$ and $\alpha, \beta \in B_t$. The order-$t$ SDP is

$$\min\ y_f = \sum_\alpha f_\alpha y_\alpha \quad \text{s.t.} \quad M_t(y) \succeq 0,\ \ M_t(g_i y) \succeq 0\ \ (i = 1, \dots, m). \tag{4.5}$$

Its optimal values form a monotonically non-decreasing sequence

$$\gamma_t \le \gamma_{t+1} \le \cdots \le \rho, \tag{4.6}$$

with $\gamma_t \to \rho$ as $t \to \infty$, and finite termination at some $t^*$ under the Archimedean condition (Putinar). Each $\gamma_t$ is a certified lower bound together with an SOS certificate $f - \gamma_t \in Q_{\mathbf{g}}$.

See *Semidefinite Programming Relaxations* for the SDP formulation and solver routing, and *Optimization* for the functional-analytic derivation.

## 4.4 Step 3 – structure exploitation

The hierarchy above is correct but expensive. Three reduction families shrink the semidefinite program without weakening the certificate:

- **Correlative sparsity** (*Correlative Sparsity Detection*) – build the sparsity graph on the variables and use its connected components to replace one large moment matrix by a block-diagonal family.
- **Newton polytope pruning** (*Newton Polytope Pruning*) – use $\mathrm{New}(f) = \mathrm{conv}\{\alpha : c_\alpha \neq 0\}$ and Minkowski sums over the system to filter candidate monomials. Note that hull membership alone does not certify removability, so the pipeline currently retains the full degree-bounded support by design.
- **Border bases** (*Border Basis Theory and Implementation*) – compute a basis of $V_d/(I \cap V_d)$ for $V_d = \mathrm{span}\{x^\alpha : |\alpha| \le d\}$ without fixing a monomial ordering, avoiding the numerical conditioning problems of Gröbner bases.

Plus *Sparse Moment Method (SparseMomentSDP)*, the toric extension of the moment method that computes real points directly on the semigroup ring $R[A]$.

## 4.5 Step 4 – GP and SONC alternatives

Not every problem is best attacked through moment matrices. Two certificate families reduce to *geometric programming*, which is polynomial-time per iteration and insensitive to degree growth.

**Geometric programming** (*Geometric Programming Relaxations*). With $g_0 = -f$ and a transform matrix $H$, build $h_j = \sum_k H_{k,j} g_k$ and minimize

$$p(\mu, w, z) = \sum_{j=1}^{m} h_+(h_j)\mu_j + \sum_{\alpha \in \Delta^{<d}} (d - \deg\alpha) \left(\frac{w_\alpha}{d}\right)^{\frac{d}{d-\deg\alpha}} \prod_i \left(\frac{\alpha_i}{z_{\alpha,i}}\right)^{\frac{\alpha_i}{d-\deg\alpha}} \tag{4.7}$$

with $d$ the effective even relaxation order. The matrix $H$ is structural, not merely numerical: it decides which combinations of objective and constraints the GP model sees, and hence the sign and support pattern of the resulting inequalities.

**SONC** (*SONC Relaxations*). With $G(\mu) = f - \sum_i \mu_i g_i$, the constrained circuit structure is encoded by four families over circuit weights $a_{\beta,j}$, auxiliary bounds $b_\beta$ and barycentric weights $\lambda^{(\beta)}$ satisfying $\sum_j \lambda_j^{(\beta)} = 1$:

$$\sum_j a_{\beta,j} \le G(\mu)_{\alpha(j)}, \qquad \prod_j \left(\frac{a_{\beta,j}}{\lambda_j^{(\beta)}}\right)^{\lambda_j^{(\beta)}} \ge b_\beta, \qquad G(\mu)_\beta^+ \le b_\beta, \qquad G(\mu)_\beta^- \le b_\beta. \tag{4.8}$$

The GP objective is minimized over $(\mu, a, b)$, branching on whether $\lambda_0^{(\beta)} > 0$.

## 4.6 Step 5 – combining certificates

Let $\Sigma_{n,2d}$ and $C_{n,2d}$ denote the SOS and SONC cones of degree $2d$ in $n$ variables. Neither contains the other in general, so their Minkowski sum is a strictly larger certificate family:

$$(\Sigma + C)_{n,2d} = \{s + c \mid s \in \Sigma_{n,2d},\ c \in C_{n,2d}\}, \qquad f^*_{\Sigma+C} = \sup\{\lambda \mid f - \lambda \in (\Sigma + C)_{n,2d}\}. \tag{4.9}$$

Because each cone embeds in the sum,

$$\max\{f^*_\Sigma,\, f^*_C\} \ \le\ f^*_{\Sigma+C} \ \le\ f^*, \tag{4.10}$$

and the bound is never worse than either pure approach. The two-step preprocessing of *SOS+SONC Relaxations* achieves it with one SDP plus one signomial program: SOS-first finds $g^* \in \Sigma_{n,2d}$ minimizing a convex distance to $f$, then a SONC relaxation is solved on the residual $h = f - g^*$, yielding $f^*_{\Sigma+C} \ge \mu^*$ with the explicit decomposition $f - \mu^* = g^* + (h - \mu^*) \in \Sigma + C$. SONC-first swaps the roles.

## 4.7 Step 6 – leaving the polynomial world

Two extensions push past polynomials in finitely many variables:

- **Differential SDP** (*Differential SDP and Mean Relaxations*, built on the derivation machinery of *Group-Ring Foundations*). A family of commuting derivations $D_1, \dots, D_m$ defines an operator semigroup $\Theta = \langle \delta_1, \dots, \delta_m \rangle$ and a jet space $\Theta Y = \{\theta y_j\}$ satisfying the multi-index Leibniz rule

$$D_i \left( \prod_{k=1}^{r} \theta_k y_{j_k} \right) = \sum_{k=1}^{r} (\delta_i \theta_k y_{j_k}) \prod_{l \ne k} \theta_l y_{j_l}. \tag{4.11}$$

  Applying the $D_i$ to algebraic differential relations $\mathcal{F}$ constructs the truncated differential ideal $\mathcal{I}_{\le 2d} = [\mathcal{F}]_{\le 2d}$, and the quotient $\mathbb{R}[S]/\mathcal{I}_{\le 2d}$ supplies the monomial basis of the moment matrix. Weighted power mean forms $M_{q,p}(X, w) = \sum_i w_i X_i^p (\sum_j w_j X_j^q)^{(p-q)/q}$ are used as certificates in the mean relaxation, generalizing both SOS and SONC.

- **Non-polynomial optimization** (*Non-Polynomial Optimization (NonPOPSDP)*, *Approximating Optimum Value*). When separating functions from transcendental terms is impossible – $x$, $\sin x$ and $e^x$ are not algebraically independent, and the dependence admits no finite relation set – Irene replaces the transcendental terms by polynomial surrogates (Taylor or Chebyshev, with the radius-of-convergence pitfall in mind), solves the resulting polynomial optimization problem, and adds a redundant ball constraint per Josz–Henrion (2014) to guarantee strong duality. The result is a reliable estimate rather than a certificate.

## 4.8 Theory-to-code map

| Object | Module | Chapter | Companion paper |
|---|---|---|---|
| Semigroup algebra $\mathbb{R}[S]$, derivations | `Irene/grouprings.py` | *Group-Ring Foundations* | Sec. 2.1 |
| `OptimizationProblem`, degree conventions | `Irene/program.py` | *Problem Representation* | Sec. 2.1 |
| Moment/localizing matrices, hierarchy $\gamma_t$ | `Irene/relaxations.py`, `Irene/sdp.py` | *Semidefinite Programming Relaxations*, *Optimization* | Sec. 2.2 |
| Correlative sparsity | `Irene/sparsity.py` | *Correlative Sparsity Detection* | Sec. 2.2 |
| Newton polytope pruning | `Irene/newton_polytope.py` | *Newton Polytope Pruning* | Sec. 2.2 |
| Border bases | `Irene/border_basis.py` | *Border Basis Theory and Implementation* | Sec. 2.2 |
| Sparse moment / toric real radical | `Irene/sparse_moment.py` | *Sparse Moment Method (SparseMomentSDP)* | Sec. 2.2 |
| GP relaxation | `Irene/geometric.py` | *Geometric Programming Relaxations* | Sec. 2.3 |
| SONC relaxation | `Irene/sonc.py` | *SONC Relaxations* | Sec. 2.3 |
| SOS+SONC hybrid | `Irene/sosonc.py` | *SOS+SONC Relaxations* | Sec. 2.4 |
| Differential SDP, mean forms | `Irene/dsdp.py` | *Differential SDP and Mean Relaxations* | Sec. 2.5 |
| Non-polynomial surrogates | `Irene/nonpopsdp.py` | *Non-Polynomial Optimization (NonPOPSDP)*, *Approximating Optimum Value* | Sec. 2.5 |
| Unified entry point | `Irene/relaxation_api.py` | *Unified Relaxation API* | Sec. 3.2 |

## 4.9 Further reading

The consolidated bibliography is *References*: the truncated moment problem on unital commutative algebras (Curto–Ghasemi–Infusino–Kuhlmann, 2023), the moment/SOS hierarchy (Lasserre 2001; Parrilo 2000; Laurent 2009; Lasserre 2015; Marshall 2008), geometric-programming bounds (Ghasemi–Marshall 2012; Iliman–de Wolff 2016), the SONC and SOS+SONC literature (Dressler–Iliman–de Wolff 2019; Schick 2025; Papp 2023), and the mean-form and differential-algebra sources. *Differential SDP and Mean Relaxations* documents how the CGIK framework specializes to the differential setting. For the package-level narrative – layer structure, solver routing, benchmark evidence and limitations – see the companion paper's Sections 3 to 7.

CHAPTER

# FIVE

# GROUP-RING FOUNDATIONS

This chapter introduces the algebraic layer behind the optimization modules.

## 5.1 Algebraic Setting

Let $S$ be a finitely generated commutative semigroup and let $\mathbb{R}[S]$ denote its semigroup algebra. In Irene, elements of $\mathbb{R}[S]$ are represented as finite sums

$$f = \sum_{\alpha \in \mathrm{supp}(f)} c_\alpha \alpha,$$

where $\alpha$ is a semigroup element and $c_\alpha \in \mathbb{R}$. This perspective generalizes classical polynomial notation and keeps support, degree, and structural operations explicit for optimization routines.

When relations are present, the algebra is effectively handled modulo those relations through reduction in the semigroup representation. This is useful for modeling quotient structures that arise naturally in symbolic formulations.

## 5.2 Commutative Semigroup and Semigroup Algebra

The module `grouprings.py` provides:

1. `CommutativeSemigroup`: generator-based semigroup representation.
2. `SemigroupAlgebra`: algebra construction over that semigroup.
3. `AtomicSGElement` and `SemigroupAlgebraElement`: monomial and polynomial-like elements.

This representation is used by higher layers to extract supports, exponents, and structural information required by geometric and SONC relaxations.

## 5.3 Support and Geometry

The support of $f$ is central in both geometric and SONC constructions. By converting semigroup monomials to exponent tuples, Irene can compute Newton polytope information and barycentric relations directly from algebraic input.

This is the key bridge from symbolic algebra to convex-geometric objects used in lower-bound certificates.

### 5.3.1 Symbolic Engine: SymEngine Primary with SymPy Fallback

Irene uses a dual-engine design for symbolic computation, implemented in `symbolic_engine.py` as the `SymbolicEngine` class (imported as `engine`).

**Design Rationale.** SymEngine provides a C++ backend that is significantly faster for polynomial expansion, numeric evaluation, and matrix construction. However, it lacks several advanced APIs required by SDP relaxation pipelines — notably Gröbner basis computation, the full `Poly` API (`as_dict()`, domain arithmetic), `PolyMatrix`/`DomainMatrix`, and `lambdify`. The dual-engine router resolves this by attempting SymEngine first and falling back to SymPy transparently when an operation is unsupported.

**Fallback Mechanism.** Every routed operation follows this pattern:

1. Attempt the SymEngine C++ path (e.g., `se.expand()`, `se.DenseMatrix`).
2. On `AttributeError`, `NotImplementedError`, or `LibExpressionException`, cast all SymEngine inputs to SymPy via `to_sympy()` and call the native SymPy equivalent.
3. Return the result (SymPy objects are accepted downstream; no forced cast-back).

The `to_sympy()` helper short-circuits when the input is already a SymPy object, avoiding redundant tree conversions. The `fallback_log` attribute records which calls fell back, and `engine.fallback_stats()` returns per-operation counts for profiling.

**Performance Profile.** Instrumented traces show that SymEngine handles the bulk of expand/matrix/symbol operations, while Gröbner basis, `Poly`, and `lambdify` always route to SymPy (these APIs simply do not exist in SymEngine). The net effect is faster polynomial manipulation with no loss of advanced algebraic functionality.

**Usage Pattern.** Existing modules import the unified engine rather than raw SymPy/SymEngine:

```
from Irene.symbolic_engine import engine
x = engine.Symbol('x')
expr = engine.expand((x + 1)**2)       # SymEngine C++ expand
g = engine.groebner([f1, f2], x)       # auto-fallback to SymPy
p = engine.Poly(expr, x)               # SymPy Poly (SymEngine lacks this)
```

This pattern ensures that all symbolic code in Irene benefits from the dual-engine routing without importing either backend directly.

**Selecting the Symbolic Backend.** Users can choose between SymEngine and pure SymPy at runtime, either programmatically or via the environment:

```
# Programmatic selection (applies to the default `engine` singleton):
from Irene.symbolic_engine import engine, set_symbolic_backend, get_symbolic_backend
set_symbolic_backend('symengine')    # or 'sympy' / 'auto'
assert get_symbolic_backend() == 'symengine'

# Environment variable (read once at import time):
#   IRENE_SYMBOLIC_BACKEND=symengine python3 my_script.py
#   IRENE_SYMBOLIC_BACKEND=sympy     python3 my_script.py
#   IRENE_SYMBOLIC_BACKEND=auto      python3 my_script.py   (default)
```

Accepted values: `symengine` (default when installed), `sympy`, and `auto` (prefer SymEngine when available, else SymPy). When `symengine` is not installed the engine automatically runs in SymPy mode and `engine.available_backends()` reports `['sympy']`; installing the package with the optional extra `pip install .[symengine]` enables the C++ backend. Operations that only exist in SymPy (Gröbner basis, `Poly`, `lambdify`, `DomainMatrix`) are backend-independent and always run on SymPy.

## 5.4 Differential Operators

Beyond static algebraic representation, `SemigroupAlgebra` supports derivations. Given a base map, derivatives are propagated through expressions using product-rule behavior.

For factors $u$ and $v$, the implementation follows:

$$D(uv) = D(u)v + uD(v).$$

This enables a workflow where optimization is not only over algebraic objects, but also over structures enriched with differential operators.

In code, the derivative path is organized as:

1. `SemigroupAlgebra.add_derivative` registers a derivation map.
2. `SemigroupAlgebra.derivative` selects a registered derivation by index.
3. `SemigroupAlgebra.diff` applies recursive product-rule expansion.

This method-level design makes differentiation explicit and extensible for problem formulations where algebraic structure and operator behavior are coupled.

**Multiple Derivation Support.** The `derivatives` attribute is a list, so the algebra can carry several independent derivation operators simultaneously. Each call to `add_derivative(base_map)` appends a new map at index `len(derivatives)`. The `derivative(expr, idx)` method then selects operator `idx` by position:

$$d_0, d_1, \ldots, d_{k-1} : \mathbb{R}[S] \to \mathbb{R}[S],$$

where each $d_i$ is registered independently via its own base map on the generators. This supports systems with up to ~10 derivation generators (sufficient for holonomic function representations and multi-variable differential constraints).

**Notation.** The derivation operators $d_x$, $d_y$, etc., are linear maps on the semigroup algebra satisfying Leibniz rules. They are **not** Leibniz fractions ($dy/dx$ is notation for a ratio of differentials; $d_x$ is an operator). In code, `derivative(expr, 0)` applies the first registered derivation (e.g., $d_x$), while `derivative(expr, 1)` applies the second (e.g., $d_y$).

From a theoretical viewpoint, derivations are linear maps $D : \mathbb{R}[S] \to \mathbb{R}[S]$ that satisfy Leibniz rules. Irene's `add_derivative` and `diff` pipeline implements this behavior directly on semigroup-algebra elements.

### 5.4.1 Formal Lie Prolongations and Multi-Derivation Framework

The derivation framework generalizes naturally to a system of commuting derivation operators $D_1, \ldots, D_m : \mathbb{R}[S] \to \mathbb{R}[S]$, each satisfying the Leibniz rule

$$D_i(uv) = D_i(u)v + uD_i(v), \qquad u, v \in \mathbb{R}[S].$$

These operators generate an **operator semigroup** $\Theta = \langle \delta_1, \ldots, \delta_m \rangle$ where each $\delta_i$ is a formal derivation symbol. The **jet space** of the algebra is the set

$$\Theta Y = \{\theta y_j \mid \theta \in \Theta,\ 1 \le j \le n\},$$

where $y_1, \ldots, y_n$ are the original semigroup generators. An element $\theta y_j$ represents the result of applying the differential operator $\theta$ to $y_j$.

The multi-derivation rule acts on products of jet-space elements via the multi-index Leibniz formula:

$$D_i\left(\prod_{k=1}^{r} \theta_k y_{j_k}\right) = \sum_{k=1}^{r} (\delta_i \theta_k y_{j_k}) \prod_{l \neq k} \theta_l y_{j_l}.$$

This formulation is the algebraic backbone of **differential SDP** (DSDP): applying the $D_i$ to a set of algebraic differential relations $\mathcal{F}$ programmatically constructs the truncated differential ideal

$$\mathcal{I}_{\leq 2d} = [\mathcal{F}]_{\leq 2d}$$

without symbolic expression tree traversal. The resulting quotient algebra $\mathbb{R}[S]/\mathcal{I}_{\leq 2d}$ is then used as the monomial basis for the moment matrix in the differential SDP hierarchy — see *Differential SDP and Mean Relaxations*.

**Implementation Status.** The current `grouprings.py` implementation supports single-derivation operators via recursive product-rule traversal (see the `diff` method). The multi-derivation operator semigroup $\Theta$ and the full multi-index Leibniz product rule described above represent the natural extension required for the DSDP research track. The existing infrastructure (`derivatives` list, `add_derivative`, `derivative` with index dispatch) is designed to accommodate this generalization.

## 5.5 Why This Matters for POP

The shift from polynomial-only notation to group-ring structures improves the connection between symbolic representation and geometric computation:

1. Exponent vectors are directly accessible for convex hull and delta-set routines.
2. Algebraic reductions remain explicit and programmatic.
3. The same representation supports SDP, GP, and SONC method families.

CHAPTER

SIX

# PROBLEM REPRESENTATION

The class `OptimizationProblem` in `program.py` is the bridge between algebraic expressions and concrete relaxation models.

## 6.1 Problem Template

The optimization model is represented in algebraic form as:

$$\min f \quad \text{subject to} \quad g_i \geq 0,\ i = 1, \ldots, m,$$

where $f$ and $g_i$ are semigroup-algebra elements. This keeps the problem in a symbolic domain until a relaxation method converts it to SDP or GP data structures.

## 6.2 Core Responsibilities

1. Store objective and constraints as semigroup-algebra expressions.
2. Track degrees used by relaxation constructors.
3. Provide support analysis utilities used by geometric and SONC modules.

## 6.3 Degree Conventions

`OptimizationProblem` tracks objective and constraint degrees and uses an even program degree for relaxation construction. This convention matches the structure needed by polynomial nonnegativity certificates and by the GP/SONC term-partitioning routines.

In particular, the effective order used by downstream methods is the smallest even degree greater than or equal to the maximum polynomial degree in the problem.

## 6.4 Key Methods

**`set_objective`**
: Registers the objective and updates degree metadata.

**`add_constraints`**
: Adds constrained expressions and updates per-constraint degree metadata.

**`delta`**
: Extracts terms relevant for nonnegativity and circuit-based conditions.

**`mono2ord_tuple` and `tuple2mono`**
: Convert symbolic monomials to numeric exponent tuples and back.

**`newton`, `convex_combination`, and related helpers**
: Support Newton-polytope and barycentric calculations required by SONC/GP paths.

## 6.5 From Equations to Methods

This chapter connects internal utilities in `OptimizationProblem` to the equation-level objects used in *Geometric Programming Relaxations* and *SONC Relaxations*.

| Mathematical role | OptimizationProblem method | Where it appears later |
|---|---|---|
| Build active support and delta terms | `delta` | Section 4 GP construction and Section 3 constrained SONC families |
| Move between symbolic monomials and exponent tuples | `mono2ord_tuple` and `tuple2mono` | Newton-polytope geometry and constraint assembly in GP and SONC |
| Compute support geometry | `newton` | Support-point extraction for SONC and transformed GP structures |
| Compute barycentric coefficients | `convex_combination` and `linear_combination` | SONC weights $\lambda^{(\beta)}$ and related support relations |

In practice, these methods are the bridge from symbolic semigroup-algebra input to the numeric data structures used by the geometric and SONC solvers.

## 6.6 Delta Sets and Newton Data

The `delta` and `newton` family of methods provide the two core theoretical objects for non-SDP relaxations:

1. Delta-style term partitions that identify relevant support terms by sign and exponent structure.
2. Newton-polytope geometry used to compute support vertices and barycentric combinations.

Together, these objects drive variable construction and inequality families in both geometric and SONC chapters.

## 6.7 Minimal Construction Pattern

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem

S = CommutativeSemigroup(['x', 'y'])
SA = SemigroupAlgebra(S)
x = SA['x']
y = SA['y']

P = OptimizationProblem(SA)
P.set_objective(1 + x**2 + y**2)
P.add_constraints([1 - x**2, 1 - y**2])
```

CHAPTER

SEVEN

# SEMIDEFINITE PROGRAMMING RELAXATIONS

The SDP module implements Lasserre's hierarchy of semidefinite programming relaxations for polynomial optimization problems. Given a problem

$$\min\{f(x) : g_1(x) \geq 0, \ldots, g_m(x) \geq 0, x \in K\},$$

the hierarchy constructs a sequence of SDPs whose optimal values converge monotonically to the true optimum under mild topological conditions.

## 7.1 Moment and Localizing Matrices

At relaxation order $t$, the method introduces moment variables $y_\alpha$ for each exponent $\alpha \in \Lambda_t = \{\alpha : |\alpha| \leq t\}$ and requires that the **moment matrix** $M_t(y)$ and all **localizing matrices** $M_t(g_i y)$ be positive semidefinite.

The moment matrix has entries indexed by monomials in the basis $B_t$:

$$M_t(y)_{\alpha,\beta} = y_{\alpha+\beta}, \quad \alpha, \beta \in B_t.$$

For each constraint $g_i(x) = \sum_\gamma h_{i,\gamma} x^\gamma$, the localizing matrix is defined by:

$$M_t(g_i y)_{\alpha,\beta} = \sum_\gamma h_{i,\gamma} y_{\alpha+\beta+\gamma}.$$

The SDP at order $t$ reads:

$$\min y_f = \sum_\alpha f_\alpha y_\alpha \quad \text{s.t.} \quad M_t(y) \succeq 0, \;\; M_t(g_i y) \succeq 0.$$

## 7.2 API Overview

The `SDPRelaxations` class provides the primary interface:

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebraElement
from Irene.program import OptimizationProblem
from Irene.relaxations import SDPRelaxations

# Define semigroup and variables
sg = CommutativeSemigroup(['x', 'y'])
x, y = sg.generators[0], sg.generators[1]

# Build problem with semigroup algebra elements
objective = SemigroupAlgebraElement(sg, {sg.one: 1, sg.monomial({0: 2}): -1})  # x^2
```

(continues on next page)

(continued from previous page)

```
constraint = SemigroupAlgebraElement(sg, {sg.one: 1, sg.monomial({0: 2, 1: 2}): 1})  # 1␣
↪+ x^2*y^2

prog = OptimizationProblem(sg, objective)
prog.add_constraint(constraint >= 0)

# Solve with SDP hierarchy
sdp = SDPRelaxations(prog)
result = sdp.solve(order=4)
print(f"Lower bound: {result['value']:.6f}")
print(f"Solver status: {result['status']}")
```

## 7.3 Solver Routing

Irene routes SDP solves through multiple backends automatically:

**Primary path (CVXPY + CLARABEL)**: The default solver uses CVXPY's DCP-compliant formulation with the CLARABEL conic interior-point method. This provides robust handling of ill-conditioned moment matrices and reliable infeasibility detection.

**Fallback path (native CVXOPT)**: If CVXPY or CLARABEL are unavailable, the solver falls back to the native CVXOPT implementation. Note that CVXOPT's infeasibility detection can differ from CLARABEL — problems declared infeasible by CLARABEL may return unbounded solutions in CVXOPT due to different tolerance handling.

**External solvers (DSDP, SDPA, CSDP)**: For very large instances, external CLI-based solvers can be invoked. These require separate installation and are configured via the solver parameter.

```
# Explicit solver selection
result = sdp.solve(order=4, solver='clarabel')    # CLARABEL via CVXPY (default)
result = sdp.solve(order=4, solver='cvxopt')      # Native CVXOPT path
result = sdp.solve(order=4, solver='dsdp')        # External DSDP CLI
```

### 7.3.1 Return Structure

The `solve()` method returns a dictionary with the following keys:

- `value` (float): Primal objective value (lower bound on minimum)
- `status` (str): Solver status string ('optimal', 'infeasible', etc.)
- `order` (int): Relaxation order used
- `basis_size` (int): Number of moment variables
- `time_init` (float): SDP construction time in seconds
- `time_solve` (float): Solver runtime in seconds

## 7.4 Practical Example: Bounded Polynomial

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebraElement
from Irene.program import OptimizationProblem
from Irene.relaxations import SDPRelaxations
```

(continues on next page)

(continued from previous page)

```
sg = CommutativeSemigroup(['x'])
x = sg.generators[0]

# Minimize (x - 2)^2 subject to x^2 <= 4
objective = SemigroupAlgebraElement(sg, {sg.monomial({0: 1}): -4, sg.monomial({0: 2}): 1}
↪)  # x^2 - 4x
# Add constant term separately if needed
constraint = SemigroupAlgebraElement(sg, {sg.one: 4, sg.monomial({0: 2}): -1})  # 4 - x^2

prog = OptimizationProblem(sg, objective)
prog.add_constraint(constraint >= 0)

sdp = SDPRelaxations(prog)
for t in range(1, 5):
    result = sdp.solve(order=t)
    print(f"Order {t}: bound = {result['value']:.6f}, "
          f"time = {result['time_solve']:.3f}s, "
          f"basis = {result['basis_size']}")
```

## 7.5 Hierarchy Convergence

Under the Archimedean condition (the set $K$ is contained in a compact spectrahedron), Putinar's Positivstellensatz guarantees that the hierarchy terminates: for some finite order $t^*$, the SDP at order $t^*$ returns the exact global minimum. In practice, convergence is often achieved at much lower orders than the theoretical bound suggests.

For non-Archimedean sets, the hierarchy still provides valid lower bounds that converge asymptotically, but termination is not guaranteed at any finite order.

## 7.6 References


- Lasserre, J.-B. (2001). "Global optimization with polynomials and the problem of sums of squares." *SIAM Journal on Optimization*, 11(3), 793–812.
- Parrilo, P. A. (2000). "Structured semidefinite programs and semialgebraic geometry methods in robustness and optimization." *Caltech PhD Thesis*.
- Laurent, M. (2009). "Sums of squares, moment matrices and optimization over polynomials." *Developments in Mathematics*, 14, 157–270.

CHAPTER

EIGHT

# GEOMETRIC PROGRAMMING RELAXATIONS

The module `geometric.py` implements a geometric-programming lower-bound strategy for constrained polynomial optimization over basic semialgebraic sets.

## 8.1 Theory to Implementation

The implementation follows the Section 4 construction used in the repository notes, including a transformed program defined by a matrix $H$ and a GP objective/constraint system over auxiliary variables.

Let $g_0 = -f$ and let $g_1, \ldots, g_m$ be the constraint polynomials. The geometric pipeline builds transformed expressions $h_j$ using $H$ and then solves for a lower bound through GP variables $\mu$, $w_\alpha$, and $z_\alpha$.

At a high level, the transformed family is:

$$h_j = \sum_k H_{k,j} g_k.$$

The relaxation objective then combines transformed positive parts and residual terms indexed by active support elements. In implementation-oriented notation, the objective has the form

$$p(\mu, w, z) = \sum_{j=1}^{m} h_+(h_j)\mu_j + \sum_{\alpha \in \Delta^{<d}} (d - \deg \alpha) \left(\frac{w_\alpha}{d}\right)^{\frac{d}{d-\deg \alpha}} \prod_i \left(\frac{\alpha_i}{z_{\alpha,i}}\right)^{\frac{\alpha_i}{d-\deg \alpha}}.$$

Here $d$ denotes the effective even relaxation order used by the problem.

The implementation follows the lower-bound construction associated with Section 4, including equation (3) as implemented in `geometric.py` for transformed objective and constraint families.

The main implementation stages are:

1. `auto_transform_matrix`: builds a transformation matrix satisfying the required sign structure.
2. `transform_program`: constructs transformed expressions from the original objective/constraints.
3. `_build_delta_sets` and `_initialize_variables`: prepare monomial sets and GP variables.
4. `_build_objective` and constraint assembly in `solve`: formulate and solve the GP model.

## 8.2 Interpretation of the Matrix Transform

The matrix $H$ is not only a numerical preconditioner. It defines which linear combinations of objective/constraint polynomials are exposed to the GP model and therefore determines the sign and support structure of the final inequalities. The `auto_transform_matrix` routine computes a feasible transformation using diagonal/sign conditions encoded in the implementation.

## 8.3 Constraint Families in `solve`

The method assembles several GP/signomial constraint blocks:

1. Normalization and variable bounds (including $\mu_0 = 1$).
2. Generator-wise inequalities over transformed coefficients.
3. Degree-matching constraints for terms in $\Delta^{=d}$.
4. Positive/negative coefficient balancing for all active terms.

These blocks correspond to the computational implementation of the Section 4 equation family and are solved through `gpkit.Model`.

## 8.4 Lower-Bound Meaning

The solved value is used as a lower bound certificate for the original POP. As with any hierarchy-style relaxation strategy, bound quality depends on model order, transform quality, and problem structure.

## 8.5 Practical Notes

1. The method returns a lower bound as a floating-point value.
2. Solver availability and numerical conditioning can affect runtime behavior.
3. If automatic transformation is unstable for a given instance, a custom matrix can be supplied by passing the `H` keyword argument or setting `gp.H` before calling `solve`.

## 8.6 Runable Example

The following example minimizes $-y - 2x^2$ over a basic semialgebraic set using GP relaxations:

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.geometric import GPRelaxations

sg = CommutativeSemigroup(['x', 'y'])
sga = SemigroupAlgebra(sg)
x, y = sga['x'], sga['y']

prog = OptimizationProblem(sga)
prog.set_objective(-y - 2 * x**2)
prog.add_constraints([1.0 - x**4 - y**4])

gp = GPRelaxations(prog, verbosity=0)
lower_bound = gp.solve()
print(f"GP lower bound: {lower_bound:.6f}")
```

The `verbosity` keyword controls solver output (`0` for silent). The returned value is a certified lower bound on the global minimum of the objective over the feasible set defined by the constraints.

CHAPTER

NINE

# SONC RELAXATIONS

The module `sonc.py` implements constrained SONC relaxations as geometric programs.

## 9.1 Formulation Focus

The implementation follows the Section 3 constrained SONC structure used in this project and supports the Example 3.3 workflow in the repository examples.

The central transformed object is:

$$G(\mu) = f - \sum_i \mu_i g_i.$$

For each relevant exponent term, the model introduces auxiliary variables and constraints that bound positive and negative parts of coefficients and connect them through barycentric weights.

For each $\beta$ in the active delta set and each support point $\alpha(j)$, the implementation uses the standard constrained SONC pattern with variables $a_{\beta,j}$, $b_\beta$, and multipliers $\mu$. To align with the Section 3 constrained formulation (equation (3.3)), the key families implemented in code are:

## 9.2 Variable Roles

1. $\mu$: Lagrange-style multipliers combining objective and constraints through $G(\mu)$.
2. $a_{\beta,j}$: circuit weights attached to support points for each active $\beta$.
3. $b_\beta$: auxiliary upper bounds controlling positive and negative parts of $G(\mu)_\beta$.

**(F1) support-point inequalities**

$$\sum_j a_{\beta,j} \le G(\mu)_{\alpha(j)},$$

**(F2) circuit-product inequality**

$$\prod_j \left( \frac{a_{\beta,j}}{\lambda_j^{(\beta)}} \right)^{\lambda_j^{(\beta)}} \ge b_\beta,$$

**(F3) positive-part bound**

$$G(\mu)_\beta^+ \le b_\beta,$$

**(F4) negative-part bound**

$$G(\mu)_\beta^- \le b_\beta.$$

The GP objective minimized in this chapter is denoted by $p(\mu, a, b)$, consistent with the Section 3 notation used by the constrained SONC formulation.

In implementation-oriented form, the objective can be read as:

$$p(\mu, a, b) = \sum_{i=1}^{m} \mu_i g_i^+(\alpha_0) + \sum_{\beta:\lambda_0^{(\beta)}>0} \lambda_0^{(\beta)} b_\beta^{1/\lambda_0^{(\beta)}} \prod_{j\neq 0} \left( \frac{\lambda_j^{(\beta)}}{a_{\beta,j}} \right)^{\lambda_j^{(\beta)}/\lambda_0^{(\beta)}}.$$

This is the objective assembled in `_build_objective` after barycentric data is available.

The barycentric weights $\lambda^{(\beta)}$ are computed from support points via convex-combination routines before model assembly.

## 9.3 Branching on $\lambda_0^{(\beta)}$

The implementation distinguishes two geometric cases:

1. $\lambda_0^{(\beta)} > 0$: objective includes a weighted exponential/product term for $\beta$.
2. $\lambda_0^{(\beta)} = 0$: circuit-product inequality (F2) controls $b_\beta$ directly without the $\lambda_0$ objective branch.

This split matches the constrained SONC structure implemented in the solver assembly pipeline.

## 9.4 Implementation Stages in `SONCRelaxations`

1. `_build_delta_sets`: collects candidate terms.
2. `_build_support_points`: computes support points/Newton vertices.
3. `_build_beta_info`: computes barycentric coordinates.
4. `_initialize_variables`: allocates variables for multipliers and circuit terms.
5. `_build_constraints` and `_build_objective`: constructs the constrained GP.
6. `solve`: solves and returns a lower bound.

## 9.5 Theory-to-Code Anchors

1. `_build_delta_sets` corresponds to active $\beta$ extraction.
2. `_build_support_points` and `_build_beta_info` provide support vertices and barycentric weights $\lambda^{(\beta)}$.
3. `_g_split` computes positive and negative parts of coefficients in $G(\mu)$.
4. `_build_constraints` encodes the constrained SONC families.
5. `_build_objective` builds the GP objective over $\mu$, $a$, and $b$ terms.

## 9.6 Geometric Interpretation

At a high level, SONC uses support geometry to construct nonnegativity certificates from circuit-like structures rather than semidefinite moment matrices. Irene's pipeline computes that geometry first, then injects it into the constrained GP model.

## 9.7 Repository Anchors

1. `tests/test_sonc_section3.py`: unit tests for barycentric weights, support points, delta sets, and end-to-end solve behavior.
2. The benchmark suite in `benchmarks/` includes SONC traces via the gallery system.

## 9.8 Runnable Example

The following example reproduces Example 3.3 from the constrained SONC paper, minimizing $1 + 2x^2y^4 + \frac{1}{2}x^3y^2$ subject to $\frac{1}{3} - x^6y^2 \geqslant 0$:

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.sonc import SONCRelaxations

sg = CommutativeSemigroup(['x', 'y'])
sga = SemigroupAlgebra(sg)
x, y = sga['x'], sga['y']

prog = OptimizationProblem(sga)
prog.set_objective(1 + 2 * x**2 * y**4 + 0.5 * x**3 * y**2)
prog.add_constraints([(1.0 / 3.0) - x**6 * y**2])

sonc = SONCRelaxations(prog, verbosity=0)
lower_bound = sonc.solve(verbosity=0)
print(f"SONC lower bound: {lower_bound:.6f}")
```

The `verbosity` keyword controls GP solver output. The returned value is a certified lower bound on the global minimum over the feasible set. To verify barycentric weight correctness, inspect `sonc._build_beta_info(...)` which returns convex-combination weights $\sum_j \lambda_j^{(\beta)} = 1$ for each active term $\beta$.

CHAPTER

TEN

# SOS+SONC RELAXATIONS

The module `sosonc.py` implements the SOS+SONC two-step optimization framework from Moritz Schick's PhD thesis (*Sums of squares plus sums of nonnegative circuit polynomials*, Universität Konstanz). It provides combined SOS+SONC lower bounds for unconstrained polynomial optimization, translating Schick's MATLAB toolbox to Python and integrating it into the Irene framework.

## 10.1 Theory

### 10.1.1 The SOS+SONC Cone $\Sigma + C$

Let $\Sigma_{n,2d}$ denote the cone of sums of squares (SOS) of polynomials in $n$ variables of degree at most $2d$, and let $C_{n,2d}$ denote the cone of sums of nonnegative circuit polynomials (SONC) of the same degree bound. Both cones are proper subsets of the PSD cone, and neither contains the other in general (see Corollary 6.14 of the companion manuscript on mean polynomials).

The Minkowski sum

$$(\Sigma + C)_{n,2d} = \{s + c \mid s \in \Sigma_{n,2d},\ c \in C_{n,2d}\}$$

is a natural certificate family that strictly contains both SOS and SONC. For an unconstrained polynomial optimization problem

$$f^* = \inf_{x \in \mathbb{R}^n} f(x),$$

the SOS+SONC relaxation computes

$$f^*_{\Sigma+C} = \sup\{\lambda \in \mathbb{R} \mid f - \lambda \in (\Sigma + C)_{n,2d}\}.$$

Because $\Sigma \subseteq \Sigma + C$ and $C \subseteq \Sigma + C$, we always have

$$\max\{f^*_\Sigma,\ f^*_C\} \ \le\ f^*_{\Sigma+C} \ \le\ f^*.$$

### 10.1.2 Two-Step Preprocessing (Algorithms 4 & 5)

Schick's thesis introduces two complementary strategies that avoid solving the full SDP-plus-exponential-cone feasibility problem.

**Algorithm 4 — SOS-first (SOS preprocessing \rightarrow SONC relaxation)**

1. Find $g^* \in \Sigma_{n,2d}$ minimising a convex distance $\varphi(f, g^*)$ (e.g., the $\ell_2$-norm of the coefficient vector of $f - g^*$). The intuition is to "cover" as much of $f$ as possible with an SOS certificate, leaving a residual that is well-suited for SONC.
2. Solve the SONC relaxation for the residual $h = f - g^*$, obtaining $\mu^* = h^*_C = \sup\{\mu \mid h - \mu \in C\}$.

3. The combined lower bound is

$$f^*_{\Sigma+C} \geq \mu^*,$$

with the decomposition $f - \mu^* = g^* + (h - \mu^*) \in \Sigma + C$.

**Algorithm 5 — SONC-first (SONC preprocessing \rightarrow SOS relaxation)**

The roles of SOS and SONC are swapped: first find $g^* \in C$ minimising $\psi(f, g^*)$, then solve the SOS relaxation on $h = f - g^*$.

### 10.1.3 Computational Complexity

- **Pure SOS:** semidefinite programming — $O(n^{6r})$ in the Lasserre relaxation order $r$.
- **Pure SONC:** signomial geometric programming — polynomial-time per sequential GP iteration. Very fast for ST-polynomials; insensitive to degree increases.
- **SOS+SONC (two-step):** the cost of one SDP plus one signomial program. The preprocessing overhead is minimal.

When SOS is infeasible (the polynomial is not SOS), the SOS-first two-step falls back to pure SONC, guaranteeing at least the SONC bound. Symmetrically for the SONC-first variant.

## 10.2 Implementation in Irene

The central class is *SOSONCRelaxations*.

`class Irene.sosonc.SOSONCRelaxations(`*prog:* OptimizationProblem, ***kwargs*`)`

Combined SOS+SONC relaxation framework.

Implements the algorithms from Schick's SOS+SONC toolbox:

- `globalMinSOS` – pure SOS relaxation (SDP via Gram matrix)
- `globalMinSONC` – pure SONC relaxation (signomial GP)
- `globalMinSOSPSONC` – two-step SOS+SONC (Algorithms 4 & 5)

**Parameters**

- **`prog`** (OptimizationProblem) – An Irene `OptimizationProblem` with an objective set via `prog.Minimize(f)`. Constraints are optional.
- **`error_bound`** (*float*) – Numerical zero tolerance (default `1e-10`).
- **`verbosity`** (*int*) – Log level (0 = silent, 1 = verbose; default 1).
- **`solver`** (*str*) – SDP solver name forwarded to `SDPRelaxations` (`'cvxopt'`, `'csdp'`, `'sdpa'`, `'dsdp'`).
- **`use_local_solve`** (*bool*) – Use signomial GP local solve for the SONC portion (default `True`).
- **`relaxation_order`** (*int*) – Relaxation order for SDP (default `1`, i.e., the first Lasserre moment relaxation).

`globalMinSONC()` → *SOSONCRelaxSol*

Compute a lower bound using the SONC relaxation.

Solves $\sup\{\lambda : f - \lambda \in C\}$ via a signomial geometric program using Irene's `SONCRelaxations`.

**Return type**
*SOSONCRelaxSol*

`globalMinSOS()` → *SOSONCRelaxSol*

Compute a lower bound using the SOS relaxation.

Solves $\sup\{\lambda : f - \lambda \in \Sigma\}$ via a Gram-matrix SDP using Irene's `SDPRelaxations`.

**Return type**
*SOSONCRelaxSol*

`globalMinSOSPSONC`(*first: str = 'sos'*) → *SOSONCRelaxSol*

Two-step SOS+SONC lower bound (Algorithms 4 & 5).

**Parameters**
**first** (*str*) – `'sos'` (Algorithm 4: SOS preprocess, then SONC) or `'sonc'` (Algorithm 5: SONC preprocess, then SOS).

**Returns**
Lower bound $f^*_{Sigma+C}$ together with decomposed certificate.

**Return type**
*SOSONCRelaxSol*

### 10.2.1 Constructor

```
from Irene.sosonc import SOSONCRelaxations
from Irene.program import OptimizationProblem

engine = SOSONCRelaxations(
    prog,                          # OptimizationProblem instance
    error_bound=1e-10,
    verbosity=1,
    solver='cvxopt',               # SDP solver
    use_local_solve=True,          # signomial GP local solve
    relaxation_order=1,            # Lasserre relaxation order
)
```

### 10.2.2 Detailed Method Reference

*globalMinSOS()*
Computes $\lambda^* = \sup\{\lambda \mid f - \lambda \in \Sigma\}$ by wrapping *SDPRelaxations*. Uses a Gram-matrix SDP via the selected solver (CVXOPT, CSDP, SDPA, or DSDP).

Returns *SOSONCRelaxSol* with `.val`, `.status`, `.error_code`, and `.runtime`.

*globalMinSONC()*
Computes $\lambda^* = \sup\{\lambda \mid f - \lambda \in C\}$ by wrapping *SONCRelaxations*. Uses signomial geometric programming via GPkit with the CVXOPT backend.

Returns *SOSONCRelaxSol*.

*globalMinSOSPSONC()*
Implements the two-step SOS+SONC lower bound.

- `first='sos'` \rightarrow Algorithm 4 (SOS preprocessing \rightarrow SONC residual)
- `first='sonc'` \rightarrow Algorithm 5 (SONC preprocessing \rightarrow SOS residual)

The residual $h = f - \lambda^*$ is constructed by shifting the constant term of the *SemigroupAlgebraElement*. If residual construction fails, the method falls back to $\max\{\lambda_{\mathrm{SOS}}, \lambda_{\mathrm{SONC}}\}$.

### 10.2.3 Result Container

**class** Irene.sosonc.**SOSONCRelaxSol**

Carries optimisation results for SOS+SONC relaxations.

**val**

Optimal $\lambda^*$, a lower bound on $f^*$.

**Type**
float

**method**

'sos', 'sonc', 'sos-first', or 'sonc-first'.

**Type**
str

**f_sos**

SOS summand of `f - val` (if applicable).

**Type**
optional

**f_sonc**

SONC summand of `f - val` (if applicable).

**Type**
optional

**status**

'optimal', 'infeasible', or 'error'.

**Type**
str

**error_code**

0 = success, 1 = infeasible, 2 = solver error.

**Type**
int

**runtime**

Wall-clock time in seconds.

**Type**
float

**message**

Human-readable status message.

**Type**
str

### 10.2.4 Convenience Function

`Irene.sosonc.sosonc_bounds`(*prog:* OptimizationProblem, ***kwargs*) → dict[str, float]

Compute SOS, SONC, and SOS+SONC lower bounds.

Returns a dict with keys `'sos'`, `'sonc'`, `'sos_first'`, `'sonc_first'`.

Returns a dictionary with four keys: `'sos'`, `'sonc'`, `'sos_first'`, `'sonc_first'`, each mapping to the corresponding lower bound.

## 10.3 Implementation Pipeline

The internal pipeline of `globalMinSOSPSONC` when called with `first='sos'` is:

1. **Solve** SOS relaxation on the original problem \rightarrow $\lambda_{\text{SOS}}$.
2. **Build residual** $h = f - \lambda_{\text{SOS}}$ by cloning the objective's coefficient-content list and subtracting $\lambda_{\text{SOS}}$ from the identity monomial's coefficient.
3. **Construct** a new *OptimizationProblem* with $h$ as the objective and the same constraints (if any).
4. **Solve** SONC on the residual \rightarrow $\mu^*$.
5. **Combine** $\lambda_{\text{SOS}} + \mu^*$ as the final lower bound.
6. **Fallback:** if any step raises an exception, return $\max\{\lambda_{\text{SOS}}, \lambda_{\text{SONC}}\}$.

The SONC-first variant is symmetric.

## 10.4 Relation to the Mean Polynomial Hierarchy

The SOS+SONC cone $\Sigma + C$ is a proper subset of the mean polynomial preprime $T_{\text{mean}}$ introduced in Sections 6–7 of the companion manuscript (Ghasemi & Kuhlmann, 2026). Consequently, the SOS+SONC lower bounds from this module are dominated by the mean-polynomial hierarchy bounds described in `algorithm_mean_polynomial_hierarchy.md`.

The natural extension of this module to the full mean-polynomial hierarchy replaces $C_{n,2d}$ (SONC) with $T^{(1)}_{\text{mean},r}$ (single mean forms) in the signomial program, and uses $M_{2,1}$ forms to capture the SOS component directly without requiring an SDP. This extension is planned for a future `mean_polynomial.py` module.

## 10.5 Example

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.sosonc import SOSONCRelaxations, sosonc_bounds

# x^4 - x^2 on R (global minimum = -0.25)
sg = CommutativeSemigroup(['x'])
sga = SemigroupAlgebra(sg)
x = sga['x']
prog = OptimizationProblem(sga)
prog.set_objective(x ** 4 - x ** 2)

engine = SOSONCRelaxations(prog, verbosity=0, relaxation_order=2)
result = engine.globalMinSOSPSONC(first='sos')
print(result)
```

(continues on next page)

(continued from previous page)

```
# Compare all four bounds
bounds = sosonc_bounds(prog, verbosity=0)
for method, val in bounds.items():
    print(f"{method}: {val:.6f}")
```

## 10.6 Test Suite

The test file `tests/test_sosonc.py` exercises the module with:

- **Container validation:** default values and string representation of *SOSONCRelaxSol*.
- **SOS integration:** $x^2 + y^2$ (minimum 0), $x^4 - x^2$ (minimum -0.25) with relaxation order 2.
- **SONC integration:** Motzkin polynomial (a known SONC form).
- **Two-step pipeline:** SOS-first and SONC-first on quadratic problems.
- **Convenience wrapper:** `sosonc_bounds` returning all four keys.
- **Error handling:** invalid `first` argument rejection.

Run with:

```
source .venv/bin/activate
python -m pytest tests/test_sosonc.py -v
```

## 10.7 Further Reading

1. M. Schick, *Sums of squares plus sums of nonnegative circuit polynomials*, PhD dissertation, Universität Konstanz, 2025. GitHub repository.
2. M. Dressler, S. Iliman, and T. de Wolff, *An Approach to Constrained Polynomial Optimization via Nonnegative Circuit Polynomials and Geometric Programming*, Journal of Symbolic Computation 91 (2019), 149–172.
3. M. Ghasemi and S. Kuhlmann, *The Cone Generated by Positive Semidefinite Mean Polynomials*, 2026 (companion manuscript, Section 7; preprint arXiv:2608.22739).

For the wider SONC and SOS+SONC literature – including the characterization of nonnegative circuit polynomials, the constrained Positivstellensatz and the duality theory behind optimal SONC bounds – see *References*.

CHAPTER

# ELEVEN

# OPTIMIZATION

Let $X$ be a nonempty topological space and $A$ be a unital sub-algebra of continuous functions over $X$ which separates points of $X$. We consider the following optimization problem:

$$\begin{aligned} &\min f(x) \\ &\text{subject to} \\ &\quad g_i(x) \geq 0 \quad i = 1, \ldots, m. \end{aligned}$$

Denote the feasibility set of the above program by $K$ (i.e., $K = \{x \in X : g_i(x) \geq 0,\ i = 1, \ldots, m\}$). Let $\rho$ be the optimum value of the above program and $\mathcal{M}_1^+(K)$ be the space of all probability Borel measures supported on $K$. One can show that:

$$\rho = \inf_{\mu \in \mathcal{M}_1^+(K)} \int f \, d\mu.$$

This associates a $K$-positive linear functional $L_\mu$ to every measure $\mu \in \mathcal{M}_1^+(K)$. Let us denote the set of all elements of $A$ nonnegative on $K$ by $Psd_A(K)$. If $\exists p \in Psd_A(K)$ such that $p^{-1}([0, n])$ is compact for each $n \in \mathbb{N}$, then one can show that every $K$-positive linear functional admits an integral representation via a Borel measure on $K$ (Marshall's generalization of Haviland's theorem). Let $Q_{\mathbf{g}}$ be the quadratic module generated by $g_1, \ldots, g_m$, i.e, the set of all elements in $A$ of the form

$$\sigma_0 + \sigma_1 g_1 + \cdots + \sigma_m g_m, \tag{11.1}$$

where $\sigma_0, \ldots, \sigma_m \in \sum A^2$ are sums of squares of elements of $A$. A quadratic module $Q$ is said to be Archimedean if for every $h \in A$ there exists $M > 0$ such that $M \pm h \in Q$. By Jacobi's representation theorem, if $Q$ is Archimedean and $h > 0$ on $K$, where $K = \{x \in X : g(x) \geq 0 \ \forall g \in Q\}$, then $h \in Q$. Since $Q$ is Archimedean, $K$ is compact and this implies that if a linear functional on $A$ is nonnegative on $Q$, then it is $K$-positive and hence admits an integral representation. Therefore:

$$\rho = \inf_{L(Q) \geq 0 \ L(1)=1} L(f).$$

Let $Q = Q_{\mathbf{g}}$ and $L(Q) \subseteq [0, \infty)$. Then clearly $L(\sum A^2) \subseteq [0, \infty)$ which means $L$ is positive semidefinite. Moreover, for each $i = 1, \ldots, m$, $L(g_i \sum A^2) \subseteq [0, \infty)$ which means the maps

$$\begin{aligned} L_{g_i} : A &\longrightarrow \quad \mathbb{R} \\ h &\mapsto \quad L(g_i h) \end{aligned}$$

are positive semidefinite. So the optimum value of the following program is still equal to $\rho$:

$$\begin{aligned} &\min L(f) \\ &\text{subject to} \\ &\quad L \succeq 0 \\ &\quad L_{g_i} \succeq 0 \quad i = 1, \ldots, m. \end{aligned} \tag{11.2}$$

This is still not a semidefinite program, since each constraint is infinite dimensional. One plausible idea is to consider functionals on finite dimensional subspaces of $A$ containing $f, g_1, \ldots, g_m$. This was done by Lasserre for polynomials [JBL].

Let $B \subseteq A$ be a linear subspace. If $L : A \longrightarrow \mathbb{R}$ is $K$-positive, so is its restriction on $B$. But generally, $K$-positive maps on $B$ do not extend to $K$-positive one on $A$ and hence existence of integral representations are not guaranteed. Under a rather mild condition, this issue can be resolved:

**Theorem.** [CGIK] Let $K \subseteq X$ be compact, $B \subseteq A$ a linear subspace such that there exists $p \in B$ strictly positive on $K$. Then every linear functional $L : B \longrightarrow \mathbb{R}$ satisfying $L(Psd_B(K)) \subseteq [0, \infty)$ admits an integral representation via a Borel measure supported on $K$.

Now taking $B$ to be a finite dimensional linear space containing $f, g_1, \ldots, g_m$ and satisfying the assumptions of the above theorem, turns (11.2) into a semidefinite program. Note that this does not imply that the optimum value of the resulting SDP is equal to $\rho$ since

- :math:`Q_{bf g}cap B

**neq Psd_{B}(K)` and,**

  - there may not exist a decomposition of $f - \rho$ as in (11.1) inside $B$ (i.e., the summands may not belong to $B$).

Thus, the optimum value gives only a lower bound for $\rho$. However, walking through a $K$-frame, as explained in [CGIK], constructs a net of lower bounds for $\rho$ that eventually approaches $\rho$.

In practice, one needs a sufficiently large finite-dimensional linear space that contains $f, g_1, \ldots, g_m$ and admits a (11.1) decomposition of $f - \rho$ within that space. Therefore, convergence happens in finitely many steps, subject to finding a suitable $K$-frame for the problem. The same truncation viewpoint for compact semialgebraic sets is developed in the A-truncated K-moment problem of Nie [JNie].

The significance of this method is that it converts an optimization problem into finitely many semidefinite programs whose optimum values approach the optimum value of the original program, while each semidefinite program can be solved in polynomial time. This does not imply a polynomial-time algorithm for the full NP-complete optimization problem, because the number of SDP levels needed to reach the optimum is unknown, and no general bound is available in the Archimedean setting.

**Note**

1. One behavior that distinguishes this method from others is that using SDP relaxations always provides a lower bound for theminimum value of the objective function over the feasibility set. While other methods usually involve evaluation of the objectiveand hence the result is always an upper bound for the minimum.

2. The SDP relaxation method relies on symbolic computations which could be quite costly and slow. Therefore, dealing with rather large problems -although *Irene* takes advantage from multiple cores- can be rather slow.

## 11.1 Notation Bridge to Geometric and SONC Chapters

The same constrained problem data $(f, g_1, \ldots, g_m, K)$ is reused across all method families documented in this manual.

1. In the SDP hierarchy chapter, the central object is the moment functional $L$.
2. In the geometric chapter, transformed families $h_j$ are built from $f$ and $g_i$.
3. In the SONC chapter, the central transformed expression is $G(\mu) = f - \sum_i \mu_i g_i$.

This shared notation is intentional and helps compare lower-bound certificates across SDP, GP, and SONC formulations on the same POP instance.

## 11.2 Beyond SDP: Alternative Lower-Bound Mechanisms

The remainder of this documentation extends the same POP template to two additional lower-bound mechanisms:

1. **Geometric programming relaxations, which use transformed polynomial families** and sign/support constraints to produce GP certificates.
2. **SONC relaxations, which use circuit-structured support geometry and** barycentric weights.

These alternatives do not replace the SDP hierarchy. Instead, they provide complementary certificates that can be more suitable for specific sparse, high-degree, or structure-rich instances.

### 11.2.1 Modern API Quick Reference

Irene provides a unified pipeline for defining and solving polynomial optimization problems. The core workflow is:

1. **Build a semigroup algebra** from generators (`CommutativeSemigroup` + `SemigroupAlgebra`)
2. **Formulate the problem** (`OptimizationProblem` with objective and constraints)
3. **Configure and solve** via `RelaxationEngine` with the chosen relaxation method

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.relaxation_api import RelaxationEngine
from Irene.relaxations import RelaxationConfig

# 1. Algebra
sg = CommutativeSemigroup(['x', 'y', 'z'])
sga = SemigroupAlgebra(sg)
x, y, z = sga['x'], sga['y'], sga['z']

# 2. Problem
prog = OptimizationProblem(sga)
prog.set_objective(-2*x + y - z)
prog.add_constraint(24 - 20*x + 9*y - 13*z + 4*x**2
                    - 4*x*y + 4*x*z + 2*y**2 - 2*y*z + 2*z**2 >= 0)
prog.add_constraint(x + y + z <= 4)
prog.add_constraint(3*y + z <= 6)
prog.add_constraint(x >= 0)
prog.add_constraint(x <= 2)
prog.add_constraint(y >= 0)
prog.add_constraint(z >= 0)
prog.add_constraint(z <= 3)

# 3. Solve (SOS relaxation, order 3)
config = RelaxationConfig(
    reduction_method="newton_polytope",
    monomial_pruning=True,
    sparsity_detection=True,
```

(continues on next page)

(continued from previous page)

```
)
engine = RelaxationEngine(prog, order=3, solver='dsdp', config=config)
result = engine.solve('sos')
print(f"Lower bound: {result.value:.8f}")
```

See *Unified Relaxation API* for the full engine API, *Problem Representation* for problem construction, and *Examples and Validation* for a gallery of worked examples.

**Note**

**Legacy API users**: the original Irene API (`SDPRelaxations([x,y,z])`, `Mom()`, `Probability=False`, `SetObjective`, `AddConstraint`) remains available for backward compatibility. See *Legacy API Migration Guide* for a complete mapping from the legacy API to the modern pipeline.

### 11.2.2 Working with Quotient Algebras

The CGIK framework described above applies to arbitrary unital commutative algebras. In practice, the function space $A$ is often represented as a quotient of a polynomial algebra:

$$A = \mathbb{R}[x_1, \ldots, x_n, y_1, \ldots, y_k]/\mathcal{I},$$

where the auxiliary symbols $y_1, \ldots, y_k$ encode algebraic relations satisfied by the original functions. For example:

- **Radicals**: $\sqrt[3]{(xy)^2}$ is encoded via $f$ with $\mathcal{I} = \langle xy - f^3 \rangle$.
- **Trigonometric functions**: $\sin(x), \cos(x)$ are encoded via $s, c$ with $\mathcal{I} = \langle s^2 + c^2 - 1 \rangle$.
- **Exponential functions**: $e^x$ is encoded via $y$ with $\mathcal{I} = \langle \text{ADE relations} \rangle$ — see *Differential SDP and Mean Relaxations* for the differential-algebraic treatment.

In Irene, algebraic relations are passed as the `relations` parameter when constructing a `SemigroupAlgebra` or `OptimizationProblem`. The quotient basis is computed automatically (via Gröbner or border basis, as configured in `RelaxationConfig`) and all subsequent moment matrix constructions use the reduced monomial basis.

### 11.2.3 Moment Constraints

Beyond the standard moment matrix and localizing matrix constraints (which enforce $L \succeq 0$ and $L_{g_i} \succeq 0$), users may impose linear constraints on moments — e.g., $\int xy \, d\mu \geq 1/2$. In the modern API, these are added directly to the `OptimizationProblem` using `add_moment_constraint`.

Equality constraints are handled similarly. For rational function optimization ($\min p(x)/q(x)$), the probability normalization $\int q(x) d\mu = 1$ replaces the default $\int 1 d\mu = 1$ condition. This is configured through the problem-level settings on `OptimizationProblem`.

### 11.2.4 Solution Extraction and SOS Decomposition

When the SDP is feasible, the dual solution provides:

1. **A certified lower bound** $\gamma$ on the global minimum.
2. **An SOS decomposition** proving $f - \gamma \in Q_{\mathbf{g}}$ (when the moment matrix has the expected rank, i.e., the flat extension condition holds).
3. **Support extraction**: the minimizers can be recovered from the moment matrix via the Lasserre–Henrion eigenvalue method [HL] or via scipy-based moment matching.

The modern API returns a `RelaxResult` object with attributes `value`, `status`, `certificate`, and `solver_info`. For detailed SOS decomposition and support extraction, see *Semidefinite Programming Relaxations*.

### 11.2.5 Theoretical Convergence Guarantees

Under the Archimedean condition on the quadratic module $Q_{\mathbf{g}}$, the SDP hierarchy produces a monotonically non-decreasing sequence of lower bounds $\{\gamma_t\}_{t \geq t_0}$ satisfying

$$\gamma_t \leq \gamma_{t+1} \leq \cdots \leq \rho,$$

and $\gamma_t \to \rho$ as $t \to \infty$. Convergence is finite when a suitable $K$-frame exists (see [CGIK] for the general theory on unital commutative algebras). In the differential setting, convergence additionally requires the Archimedean boxing condition — see *Differential SDP and Mean Relaxations*.

## 11.3 Conic Duality and Nonnegativity Certificates

The primal SDP minimizes $L(f)$ subject to PSD constraints; its dual maximizes $\gamma$ such that $f - \gamma = \sigma_0 + \sum_i \sigma_i g_i$ with $\sigma_j$ sums of squares of bounded degree. A dual unboundedness certificate (reported as `'infeasible'` status) is equivalent to a **certified SOS proof** that $f$ cannot be represented as an element of $Q_{\mathbf{g}}$ at the given relaxation order — see *CVXPY Solver Layer* for solver-specific behavior.

CHAPTER

TWELVE

# BORDER BASIS THEORY AND IMPLEMENTATION

The `border_basis.py` module provides tools for computing border bases of quotient algebras $\mathbb{R}[x_1, \ldots, x_n]/I$ at a fixed degree. Border bases offer numerical advantages over Gröbner bases for polynomial optimization, particularly for moment matrix constructions in SDP hierarchies.

## 12.1 Theory

### 12.1.1 Border Bases vs Gröbner Bases

A **Gröbner basis** of an ideal $I$ depends on a monomial ordering and produces a unique standard monomial set (the normal form basis). However, the choice of ordering can severely affect numerical conditioning: lexicographic orderings tend to produce large coefficients, while graded reverse lexicographic orderings may include high-degree monomials that inflate matrix dimensions.

A **border basis** at degree $d$ works with the vector space $V_d = \text{span}\{x^\alpha : |\alpha| \leq d\}$ and computes a basis for the quotient $V_d/(I \cap V_d)$ without fixing a monomial ordering. The key insight is that multiplication by variables maps $V_d$ into a larger space, and the **border** $\partial V_d = \{x_i x^\alpha : |\alpha| = d\}$ encodes how the quotient algebra extends to degree $d + 1$.

### 12.1.2 Admissible Term Orders and QR Pivot Selection

The numerical QR column-pivoting scheme in `BorderBasis._compute_basis` selects a monomial basis for the quotient ring $\mathbb{R}[x_1, \ldots, x_n]/I$ by eliminating monomials whose coefficient columns are linearly dependent on the ideal generators. To guarantee that this numerical selection recovers the standard monomial basis of the quotient, the column norm selection rule is perturbed by a graded term-order weight.

Let $\prec$ be an admissible term order on the exponent vectors $\alpha \in \mathbb{N}^n$ (e.g., degree-lexicographic $\prec_{\text{deglex}}$ or degree-reverse-lexicographic $\prec_{\text{degrevlex}}$). Let $\text{rank}_\prec(\alpha)$ be the position of $\alpha$ in the ascending enumeration of exponent vectors under $\prec$ (smaller monomials have lower rank).

Each column in the relation matrix $R$ is scaled by the weight

$$w_\alpha = 10^{2 \cdot \|\alpha\|_1} \cdot \big(1 + \varepsilon \cdot \text{rank}_\prec(\alpha)\big),$$

where $\varepsilon \ll 1$ (e.g., $10^{-12}$) and $\|\alpha\|_1 = \sum_i \alpha_i$ is the total degree. The primary factor $10^{2\|\alpha\|_1}$ ensures that **higher-degree monomials pivot first** (respecting the graded structure required by border basis theory). The secondary perturbation $1 + \varepsilon \cdot \text{rank}_\prec(\alpha)$ breaks ties among monomials of the same total degree: the lex-larger monomial (higher $\text{rank}_\prec$) receives a slightly larger weight and is selected as a pivot column, eliminating it from the quotient basis. This guarantees that the QR pivoting uniquely recovers the standard monomial basis of $\mathbb{R}[x_1, \ldots, x_n]/I$.

### 12.1.3 Conditioning Benefits

Border bases are known to be better conditioned than Gröbner bases for degrees $d$ $geq6$ in multivariate settings. This is because:

1. **No monomial ordering bias in the basis itself**: The basis adapts to the numerical structure of the generators; the ordering only affects pivot tie-breaking.
2. **Compact representation**: Only monomials up to degree $d$ are considered, avoiding the high-degree terms that Gröbner bases may introduce.
3. **Orthogonalization-friendly**: The border basis algorithm uses QR factorization, which preserves numerical stability under perturbation.

For SDP hierarchies in polynomial optimization, this translates to better-conditioned moment matrices and more reliable semidefinite programming solves at higher orders.

### 12.1.4 Multiplication Tables

The border basis representation includes **multiplication tables** $M_{x_i}$ that describe how multiplication by each variable acts on the quotient algebra basis. These tables are symmetric when the ideal is zero-dimensional and the quotient admits an inner product structure (as in the moment problem setting).

Specifically, if $\{b_1, \ldots, b_k\}$ is a border basis of $V_d/(I \cap V_d)$, then for each variable $x_i$:

$$x_i \cdot b_j = \sum_{l=1}^{k} (M_{x_i})_{lj} b_l + \text{border terms}.$$

The multiplication tables encode the algebra structure of the quotient and can be used to recover roots via joint eigenvalue methods when $I$ is zero-dimensional.

## 12.2 API Reference

### 12.2.1 BorderBasis Class

```
from Irene.border_basis import BorderBasis

# Construct border basis at degree d for ideal generated by polys g1, ..., gm
bb = BorderBasis(variables=['x', 'y'], generators=[g1, g2], degree=4)

# Access the computed basis
basis = bb.basis          # List of monomials in quotient
mult_tables = bb.tables   # Multiplication tables M_xi for each variable
```

The constructor takes:

- **variables** (list[str]): Variable names defining the polynomial ring
- **generators** (list): Polynomial generators of the ideal $I$ (as SymPy/SymEngine expressions or semigroup algebra elements)
- **degree** (int): The degree bound $d$ for the border basis computation

The border basis is computed via QR factorization of the generator coefficient matrix restricted to monomials of degree up to $d$. The resulting basis spans a complement of $I \cap V_d$ in $V_d$.

## 12.3 Integration with SDP Hierarchies

In the Irene relaxation pipeline, border bases are used to replace the full monomial basis with a numerically stable quotient basis at each order. This reduces moment matrix dimension while preserving the algebraic structure needed for positive semidefinite constraints.

The `RelaxationEngine` automatically uses border basis reduction when configured:

```
from Irene.relaxations import RelaxationConfig
from Irene.relaxation_api import RelaxationEngine

config = RelaxationConfig(
    reduction_method="border_basis",
    monomial_pruning=True,
)
engine = RelaxationEngine(prog, order=3, config=config)
result = engine.solve("sos")
```

### 12.3.1 Selecting the Quotient-Basis Reduction Engine

The **quotient-basis option** (added 2026-08-09) lets users choose which engine performs the quotient-ring reduction inside `SDPRelaxations`:

- `quotient_basis="groebner"` (default) — classical SymPy Groebner-basis reduction via `sp.reduced`, matching original Irene.
- `quotient_basis="border"` — Irene's `BorderBasis` quotient-algebra reduction via multiplication tables (`ReduceExp` and `ReducedMonomialBase` both use it).

```
config = RelaxationConfig(quotient_basis="border")   # or "groebner"
rlx = SDPRelaxations([x, y], relations=[x**2 + y**2 - 1], config=config)
```

The environment variable `IRENE_QUOTIENT_BASIS=groebner|border` sets the default when no config is passed (useful for benchmark matrices).

**Numerical fix (2026-08-09):** the QR column-pivot selection used in `BorderBasis._compute_basis` previously resolved same-degree ties by column order, which could keep the generator's own leading monomial in the basis (e.g. $y^2$ instead of $x^2$ for
$langle x^2 + y^2 - 1$
$rangle$, whose lex leading monomial is $x^2$), producing a wrong quotient basis. Column weights now carry a tiny ascending-lex tie-break so the lex-larger monomial pivots first, consistent with the monomial order used during reduction. All test ideals verify against the theoretical standard monomials.

## 12.4 Practical Notes

1. Border bases are most beneficial for **multivariate problems at degree $geqslant 6$**, where Gröbner basis conditioning degrades significantly.
2. The multiplication tables can be used to extract **moment vectors** from the dual SDP solution via eigenvalue methods.
3. For zero-dimensional ideals, the border basis size equals the number of complex roots (counting multiplicity), providing a dimension check.
4. The border basis is computed numerically (QR + floating-point reduction); `reduce()` returns float coefficients. Use `quotient_basis="groebner"` when exact rational arithmetic is required.

## 12.5 References


- Möller, H. M. & Trager, B. M. (1987). "A new approach to polynomial system solution." *ISSAC '87*.
- Kehrein, A., Kreuzer, M., & Robbiano, L. (2005). "An algebraist's view on border bases." In *Solving Polynomial Equations: Foundations, Algorithms, and Applications*, Algorithms and Computation in Mathematics 14, Springer, 169–202. DOI: 10.1007/3-540-27357-3_4.
- Kehrein, A. & Kreuzer, M. (2006). "Computing border bases." *Journal of Pure and Applied Algebra*, 205(2), 279–295. DOI: 10.1016/j.jpaa.2005.07.006.


A consolidated bibliography, including the moment-SOS and SONC literature the other chapters rest on, is in *References*.

CHAPTER

# THIRTEEN

# SPARSE MOMENT METHOD (SPARSEMOMENTSDP)

The `sparse_moment.py` module implements the **sparse moment method** of Baumbach & Bender [BB2026] for computing the *real* points of sparse polynomial systems directly on an affine toric variety, without first solving the (larger) complex system. It is Irene's toric extension of Lasserre–Laurent–Rostalski moment-SDP real-radical computation to the semigroup ring $R[A]$.

## 13.1 Theory

For a finite exponent set $A = \{\alpha_1, \dots, \alpha_r\} \subset \mathbb{N}^m$, the semigroup ring $R[A] = \mathbb{R}[x^{\alpha_1}, \dots, x^{\alpha_r}] \subset \mathbb{R}[x_1, \dots, x_m]$ is the coordinate ring of an affine toric variety. A linear functional $\Lambda \in R[A]^*$ is *positive* when its moment matrix

$$M_s(\Lambda) := \big(\Lambda(x^\alpha x^\beta)\big)_{\alpha,\beta \in A_s}$$

is positive semidefinite, where $A_s = \{\sum_{i \le s} \alpha_i\}$ is the $s$-truncation. For a system $I = \langle f_1, \dots, f_\mu \rangle \subset R[A]$ with finite real toric variety, the truncated spectrahedron

$$K_t = \{\Lambda \in R[A]_t^* : \Lambda(1) = 1,\ M_{\lfloor t/2 \rfloor}(\Lambda) \succeq 0,\ \Lambda(h) = 0\ \forall h \in H_t\}$$

(where $H_t$ is the set of prolongations $x^\beta f_i$, eq. (9) of the paper) is a polytope whose vertices are the evaluations at the real toric points (paper Theorem 3.15). A *generic* element $\Lambda \in \operatorname{relint}(K_t)$ has a kernel ideal that eventually equals the real radical $\sqrt[R]{I}$ (Lemma 3.14, Theorem 3.26), and the flat extension theorem (paper Theorem 3.17) plus its rank conditions (Theorem 3.27) give a terminating algorithm:

1. Solve $\min_{\Lambda \in K_t} \Lambda(1)$ — a feasibility SDP (eq. (11)); a generic optimizer is a boundary-free point giving the real radical.
2. Test the flatness rank conditions $\operatorname{rank} M_s = \operatorname{rank} M_{s-1}$ for admissible $s$; if none holds, increment the truncation $t$ and re-solve.
3. On convergence, read the real toric points off the sparse Stickelberger eigenvalue problem on a connected-to-one border basis of $R[A]/\sqrt[R]{I}$.

Two details distinguish the sparse from the dense moment method and are enforced by this module:

- **Toric relation bound :math:`rho_A`** (paper eq. (3)): the flat extension theorem requires $s \ge \rho_A$, where $\rho_A = \max \deg(g)$ over generators $G_A$ of the toric ideal $T_A = \ker \psi_A$. Without this, one can have $\operatorname{rank} M_1 = \operatorname{rank} M_0 = 1$ yet no rank-one flat extension (paper Example 3.18 in $R[x^2, x^3]$, where $1^3 \neq 2^2$).
- **Connected-to-one border basis** (paper Appendix A, Remark 3.28): the Stickelberger functional values are recovered by choosing a monomial column basis of $M_{s-1}$ greedily in increasing $\deg_A$ order, which is a border basis connected to one.

Reference

## 13.2 Pipeline

1. `ToricSetup.from_exponents` computes the toric ideal $T_A$, its generators $G_A$, and the bound $\rho_A$ (paper equation (3)).
2. `moment_indices(A, t)` enumerates $R[A]^t$ (smaller than the dense monomial set) and *SparseMomentSDP* builds $K_t$ (eq. (10)) as an SDP.
3. `solve_algorithm1` runs the Algorithm-1 termination loop (Theorem 3.27 rank conditions; $K_t = \emptyset$ certifies "no real toric solution" via Lemma 3.29).
4. `SparseBorderBasis` builds the connected-to-one border basis and its multiplication tables; `recover_all_sparse_roots` solves the sparse Stickelberger eigenvalue problem and returns the real toric points.

## 13.3 Quick Example

```
import sympy as sp
from Irene.sparse_moment import solve_sparse_real_roots

x = sp.Symbol("x")

# Real roots of x^3 - x on the dense line R[x].
out = solve_sparse_real_roots([x**3 - x], A=[(1,)])
print(sorted(v["z1"] for v in out.roots))     # [-1.0, 0.0, 1.0]

# Sparse form: roots of (x^2 - 1)(x^2 - 4) = x^4 - 5x^2 + 4 in
# R[A] = R[x^2, x^3] (A = {(2,0),(3,0)}).  Only the two generators needed.
out = solve_sparse_real_roots([x**4 - 5*x**2 + 4], A=[(2, 0), (3, 0)])
print(sorted(v["z1"] for v in out.roots))     # [-2.0, -1.0, 1.0, 2.0]
```

The first is the dense/univariate case; the second exercises the toric relations $(x^2)^3 = (x^3)^2$ $(= x^6)$ so the moment matrices are sized by exponents of $R[A]$ rather than the full ring.

## 13.4 Public API

- `ToricSetup.from_exponents(A)` — semigroup ring $R[A]$, toric generators $G_A$, and $\rho_A$ (eq. (3)).
- `toric_ideal(A)` / `deg_A(f, A)` — toric ideal $T_A$ (via Sturmfels' elimination) and the A-graduation.
- `moment_indices(A, t)` / `prolongations(f_system, A, t, index)` — basis of $R[A]^t$ and the prolongation set $H_t$ (eq. (9)).
- `SparseMomentSDP(f_system, A, t, objective, ...)` — the spectrahedron $K_t$ (eq. (10)) as a CVXPY-based SDP; `objective='constant'` realizes Algorithm 1, a random Gaussian vector realizes Algorithm 2 (single real root).
- `test_theorem_327(...)` — the rank conditions of Theorem 3.27.
- `solve_algorithm1(f_system, A, t0, tol, solver)` — the full termination loop; returns `Algorithm1Result` (real radical generators, quotient basis, rank profile).
- `SparseBorderBasis(...)` — connected-to-one border basis of $R[A]/\sqrt[R]{I}$ with multiplication tables and `check_relations` residual.
- `recover_all_sparse_roots(result, A, f_system, seed, tol)` — sparse Stickelberger recovery of all real toric points with per-root backward error (paper eq. (12)).

- `solve_sparse_real_roots(f_system, A, ...)` — top-level pipeline (Algorithm 1 + border basis + Stickelberger).

## 13.5 Numerical Notes

- First-order SDP solvers may return a boundary point of $K_t$, making both the rank test and the greedy border-basis selection tolerance-sensitive. The module's border-basis selector enforces connected-to-one (Def A.2) and an explicit SVD rank test on the rectangular column matrix (rather than a squared Gram test), and normal forms are pinned to the flat-extension range $2s^* - \deg_A(e)$ — the moments genuinely fixed by Theorem 3.17.
- When a real root lies on the toric variety boundary (a generator value 0, e.g. paper Example 3.16's point $(0, 1)$), the recovered values still match the asserted ideal; the real radical $I = \langle x^{\alpha_1+\alpha_2}, x^{\alpha_1+\alpha_2} - x^{\alpha_1} - x^{\alpha_2} + 1\rangle$ yields the two toric points $\{(0, 1), (1, 0)\}$.

## 13.6 Integration Notes

- The module is an independent solver on the *support structure* of the system; it complements (rather than routes through) `SDPRelaxations`. Where the dense Lasserre hierarchy keeps every monomial up to degree $t$, the sparse moment method keeps only the monomials in $R[A]^t$, which is often far smaller.
- *scripts/verify_article_examples.py* reproduces the paper's concrete claims (Examples 3.8, 3.16, 3.18, 4.8, 4.13) against this implementation — run it with the project venv to confirm the numerical statements in this page.

CHAPTER

# FOURTEEN

# CORRELATIVE SPARSITY DETECTION

The `sparsity.py` module implements automatic detection of correlative sparsity in polynomial optimization problems. Correlative sparsity exploits the structure of variable interactions to decompose large moment matrices into smaller block- diagonal components, dramatically reducing SDP solve times.

## 14.1 Theory

### 14.1.1 Correlative Sparsity via UnionFind

A polynomial optimization problem exhibits **correlative sparsity** when the variables appearing in each constraint and the objective can be partitioned into overlapping subsets that interact only through shared variables. Formally, define the **sparsity graph** $G = (V, E)$ where:

- Vertices $V = \{x_1, \ldots, x_n\}$ are the problem variables
- An edge $(x_i, x_j) \in E$ exists if variables $x_i$ and $x_j$ appear together in some monomial of a constraint or the objective

The **connected components** of this graph determine which variable subsets can be treated independently. If $G$ has $k$ connected components with vertex sets $V_1, \ldots, V_k$, then the moment matrix decomposes into $k$ smaller blocks rather than one monolithic $n$-variable block.

The implementation uses a **Union-Find** (disjoint-set union) data structure to compute connected components efficiently:

```
# Pseudocode for sparsity detection from polynomial list
uf = UnionFind(n_variables)
for poly in [objective, *constraints]:
    for monomial in poly.monomials():
        vars_in_monomial = [i for i in range(n) if monomial.degree[i] > 0]
        for i in range(1, len(vars_in_monomial)):
            uf.union(vars_in_monomial[0], vars_in_monomial[i])

components = uf.components()  # List of integer sets
```

### 14.1.2 Chordal Decomposition of Moment Matrices

When the sparsity graph is **chordal** (every cycle of length \geqslant 4 has a chord), the moment matrix $M_t(y)$ admits an exact block-diagonal decomposition via the **running intersection property**. This means:

1. The PSD constraint $M_t(y) \succeq 0$ is equivalent to a set of smaller PSD constraints on clique-based submatrices

2. Each submatrix involves only the variables in one clique, reducing dimension from $\binom{n+td}{td}$ to sums of $\binom{|C_i|+td}{td}$

For non-chordal graphs, a **chordal completion** adds edges to make the graph chordal while preserving the problem structure. The implementation detects this automatically and reports the completed clique structure.

### 14.1.3 Block-Diagonal Reduction Factors

The reduction factor depends on the sparsity pattern:

- **Fully dense** (one component of size $n$): No reduction, matrix size $\binom{n+td}{td}$
- **Two disjoint components** of size $n/2$: Matrix sizes sum to $2 \cdot \binom{n/2+td}{td}$, typically a 10×–100× reduction for large $n$
- **Many small components**: Near-linear scaling in $n$ rather than polynomial

## 14.2 API Reference

### 14.2.1 CorrelativeSparsity Class

```
from Irene.sparsity import CorrelativeSparsity, detect_sparsity_from_polys

# From a list of polynomials (objective + constraints)
sparsity = detect_sparsity_from_polys([objective, g1, g2, ...], n_variables)

# Access component structure
components = sparsity.components    # List of sets of variable indices
n_components = len(components)      # Number of disjoint blocks
clique_sizes = [len(c) for c in components]  # Variables per block
```

The `detect_sparsity_from_polys` function returns a `CorrelativeSparsity` object with:

- **components** (list[set[int]]): Connected components as sets of variable indices
- **n_components** (int): Number of disjoint blocks
- **max_clique_size** (int): Largest component size (determines worst-case block dimension)
- **reduction_factor** (float): Estimated dimension reduction ratio

## 14.3 Integration with Relaxation Pipeline

Sparsity detection is automatically applied when configured in the relaxation engine:

```
from Irene.relaxations import RelaxationConfig
from Irene.relaxation_api import RelaxationEngine

config = RelaxationConfig(
    sparsity_detection=True,
    verbose_reduction=True,  # Shows detected components
)
engine = RelaxationEngine(prog, order=2, config=config)
result = engine.solve("sos")
```

When `verbose_reduction=True`, the engine prints detected component structure:

```
Sparsity detection: 3 components found
  Component 0: variables {x1, x2, x5} (size 3)
  Component 1: variables {x3, x4} (size 2)
  Component 2: variables {x6, x7, x8} (size 3)
Estimated reduction factor: 12.4x
```

## 14.4 Practical Notes

1. Sparsity detection is **free** — it adds negligible overhead to the relaxation pipeline
2. The Union-Find structure runs in nearly-linear time $O(m \cdot \alpha(n))$ where $m$ is the total monomial count and $\alpha$ is the inverse Ackermann function
3. For problems with **no sparsity** (all variables interact), detection correctly returns a single component of size $n$, incurring no penalty
4. Sparsity works best when combined with Newton polytope pruning — the two reductions are complementary

## 14.5 References


- Waki, H., Kim, S.-J., & Vanderbei, R. J. (2007). "Sums of squares and sparsity in semidefinite programming." *SIAM Journal on Optimization*, 18(1), 41–60.
- Lasserre, J.-B. & Parrilo, P. A. (2004). "Sparse polynomial optimizations via sum-of-squares and global optimization." *Mathematical Programming*, 103(1), 267–292.
- Aufberger, J., Dimauri, A., & Safey El Din, M. (2018). "Exploiting sparsity in polynomial optimization via the Lasserre hierarchy." *SIAM Journal on Optimization*, 28(4), 3365–3391.

CHAPTER

FIFTEEN

# NEWTON POLYTOPE PRUNING

The `newton_polytope.py` module implements basis pruning via Newton polytope geometry: it computes the convex hull of the exponent vectors of a polynomial system, filters candidate monomials by membership in the scaled Newton body, and always retains every exponent passed as `required_support`. The relaxation pipeline currently passes the full degree-bounded support – hull membership alone does not certify that an exponent is removable from a Gram/moment basis – so until a certified sparsity set exists, the reduction is a designed no-op (see Theory).

## 15.1 Theory

### 15.1.1 Newton Polytopes of Polynomial Systems

The **Newton polytope** of a polynomial $f = \sum_\alpha c_\alpha x^\alpha$ is the convex hull of its exponent vectors:

$$\mathrm{New}(f) = \mathrm{conv}\{\alpha \in \mathbb{N}^n : c_\alpha \neq 0\}.$$

For a system of polynomials $F = \{f_0, f_1, \ldots, f_m\}$ (objective plus constraints), the relevant geometry is captured by the **Minkowski sum**:

$$\mathrm{New}(F) = \sum_{i=0}^{m} \mathrm{New}(f_i).$$

At relaxation order $t$, the moment matrix uses a monomial basis indexed by exponents in $\Lambda_t = \{\alpha : |\alpha| \leq t\}$. However, many of these exponents may lie outside the scaled Newton body $t \cdot \mathrm{New}(F)$, meaning they cannot contribute to valid certificates of nonnegativity for the given problem structure.

### 15.1.2 Scaled Newton Bodies and Basis Pruning

The key theorem (see Parrilo 2000, Lasserre 2006) states that the moment matrix can be restricted to exponents in:

$$\Lambda_t^{\mathrm{pruned}} = \Lambda_t \cap t \cdot \mathrm{New}(F) \cap \mathbb{N}^n.$$

This intersection removes monomials whose exponents lie outside the scaled Newton body while preserving all monomials needed for valid SDP relaxations. In practice, however, hull membership alone is *not* a certificate that an exponent may be removed from a Gram/moment basis (the order-3 Motzkin instance is a known counterexample: hull-only over-pruning left the relaxed SDP with an unbounded recession ray). Until such a certificate exists, the relaxation pipeline retains the full degree-bounded support via `required_support`, which makes the prune a designed no-op on dense instances; see "Integration with Relaxation Pipeline".

### 15.1.3 Dimension Reduction in Practice

For a bivariate degree-6 problem like Motzkin ($x^4y^2 + x^2y^4 - 3x^2y^2 + 1$), the full basis at order 3 has $\binom{8}{2} = 28$ elements. The hull-only Newton prune (no required support) reduces this to **10** monomials – a factor of 2.8x – because 18 exponents of total degree at most 6 lie outside $2\,\mathrm{New}(f)$, where $\mathrm{New}(f) = \mathrm{conv}\{(0,0),(2,2),(2,4),(4,2)\}$.

In higher dimensions, the hull-only reduction factor grows exponentially with $n$ for sparse polynomial systems, making Newton pruning one of the most impactful optimizations there; the pipeline activates it as soon as a certified sparsity set replaces today's full-support retention.

### 15.1.4 Minkowski Sum Computation

The implementation computes Minkowski sums via convex hull operations on the union of translated exponent sets:

$$P \oplus Q = \mathrm{conv}\{p + q : p \in P, q \in Q\}.$$

For efficiency, the sum is computed incrementally using the `scipy.spatial.ConvexHull` routine on the combined vertex set. The final scaled body is obtained by multiplying all vertices by the relaxation order $t$.

## 15.2 API Reference

### 15.2.1 NewtonPruner Class

```
from Irene.newton_polytope import NewtonPruner, prune_basis_from_polys

# Prune the monomial basis of a polynomial system at degree bound 6
pruner = prune_basis_from_polys([objective, g1, g2], num_vars=2, max_degree=6)
```

`prune_basis_from_polys` (and `prune_basis_from_problem`) return a configured `NewtonPruner` with the pruned basis already computed. Key attributes:

- **full_basis_size** (int): Number of monomials in the unpruned degree-bounded basis $\Lambda_t$
- **pruned_basis_size** (int): Number of monomials after Newton polytope filtering, plus every exponent in `required_support`
- **reduction_ratio** (float): Fraction of the basis retained (1.0 = nothing pruned; smaller means more pruning). The reduction factor is its reciprocal, e.g. a ratio of $10/28 \approx 0.36$ corresponds to a factor of 2.8x.

Extra detail (`matrix_entry_reduction`, `entries_saved`) is available via `pruner.summary()` or `pruner.moment_matrix_dimension_reduction()`, and the exponent tuples themselves via `pruner.compute_pruned_basis()`.

### 15.2.2 Example Usage

The classic bivariate degree-6 test case is Motzkin's polynomial. Built through Irene's documented semigroup-algebra construction path, it prunes from the full 28-monomial basis to the 10 monomials inside the scaled Newton body:

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.newton_polytope import prune_basis_from_polys

sg = CommutativeSemigroup(['x', 'y'])
```

(continues on next page)

(continued from previous page)

```
sga = SemigroupAlgebra(sg)
x, y = sga['x'], sga['y']

# Motzkin polynomial (total degree 6)
motzkin = x**4 * y**2 + x**2 * y**4 + 1 - 3 * x**2 * y**2

pruner = prune_basis_from_polys([motzkin], num_vars=2, max_degree=6)
print(pruner.full_basis_size)    # 28
print(pruner.pruned_basis_size)  # 10
print(pruner.reduction_ratio)    # ~0.357 (retained fraction; factor = 28/10 = 2.8x)
```

Plain SymPy input works identically: `prune_basis_from_polys([f], num_vars=2, max_degree=6)` with $f = x^4y^2 + x^2y^4 - 3x^2y^2 + 1$ returns the same 28 to 10 counts. The exponent tuples are obtained from `pruner.compute_pruned_basis()`; `prune_basis_from_problem(prog, max_degree=6)` accepts an *OptimizationProblem* directly and gives the same result.

## 15.3 Integration with Relaxation Pipeline

Newton polytope pruning is activated via the relaxation configuration:

```
from Irene.relaxations import RelaxationConfig
from Irene.relaxation_api import RelaxationEngine

config = RelaxationConfig(
    reduction_method="newton_polytope",
    monomial_pruning=True,
    verbose_reduction=True,
)
engine = RelaxationEngine(prog, order=3, config=config)
result = engine.solve("sos")
```

When `verbose_reduction=True`, the pipeline prints one measured line per monomial basis it builds – e.g., on Motzkin at order 3:

```
[SDPRlx] Newton pruning: 28 -> 28 (100.00% retained)
```

The standalone hull-only call in "Example Usage" above reduces the same basis from 28 to 10, but the relaxation pipeline passes the full degree-bounded support as `required_support` (certified-safe retention): until a certified SOS-support reduction exists, hull membership alone does not justify dropping an exponent from the Gram/moment basis, so on dense instances like Motzkin the pruned basis is identical to the unpruned one by design – and the relaxed SDP (bound `-526.08114071`) matches the baseline at every order.

## 15.4 Practical Notes

1. Newton pruning is **most effective** for problems where the objective and constraints have sparse support relative to their degree
2. The pruning step adds negligible overhead (~milliseconds) compared to SDP solve times (seconds to minutes)
3. For fully dense polynomials, the reduction factor approaches 1x (no pruning benefit), but correctness is preserved

4. Newton pruning and correlative sparsity are **complementary** — they can be applied together for multiplicative reduction effects

## 15.5 References


- Parrilo, P. A. (2000). "Structured semidefinite programs and semialgebraic geometry methods in robustness and optimization." *Caltech PhD Thesis*.
- Lasserre, J.-B. (2006). "Convergent SDP-relaxations in polynomial optimization with sparsity." *SIAM Journal on Optimization*, 17(3), 822–843. DOI: 10.1137/05064504x.
- Iliman, S. & de Wolff, T. (2016). "Lower Bounds for Polynomials with Simplex Newton Polytopes Based on Geometric Programming." *SIAM Journal on Optimization*, 26(2), 1128–1146. DOI: 10.1137/140962425.

CHAPTER

# SIXTEEN

# UNIFIED RELAXATION API

The `relaxation_api.py` module provides a unified interface for running SOS, SONC, and SOSONC relaxations through a single engine class. It wraps the individual relaxation modules behind a consistent API and offers convenience functions for comparing multiple relaxation methods side by side.

## 16.1 RelaxationEngine

The `RelaxationEngine` class is the primary entry point for running relaxations with configurable reduction pipelines:

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.relaxation_api import RelaxationEngine
from Irene.relaxations import RelaxationConfig

# Build problem
sg = CommutativeSemigroup(['x', 'y'])
sga = SemigroupAlgebra(sg)
x, y = sga['x'], sga['y']

prog = OptimizationProblem(sga)
prog.set_objective(x**4 * y**2 + x**2 * y**4 + 1 - 3 * x**2 * y**2)

# Configure and run
config = RelaxationConfig(
    reduction_method="newton_polytope",
    monomial_pruning=True,
    sparsity_detection=True,
)

engine = RelaxationEngine(prog, order=2, solver="clarabel", config=config)
result = engine.solve("sos")  # or "sonc" or "sosonc"
print(f"SOS bound: {result.value:.8f}")
```

### 16.1.1 Constructor Parameters

- **prog** (OptimizationProblem): The problem to relax
- **order** (int): Relaxation order $t$
- **solver** (str, optional): Solver backend — `"cvxopt"` (default), `"clarabel"`, `"dsdp"`. Clarabel and CVXOPT are the primary backends for SDP relaxations; Clarabel is recommended on instances where

structural feasibility matters (e.g., Motzkin / Choi–Lam at relaxation order $\geq 3$, where some backends misreport infeasibility).

- **verbosity** (int, optional): Output level 0–2 (default: 1)
- **config** (RelaxationConfig, optional): Reduction pipeline configuration

### 16.1.2 The `solve()` Method

```
result = engine.solve(method="sos")
```

Accepts `method` as one of:

- ```**"sos"**```: Sum-of-squares relaxation via moment matrix PSD constraints
- ```**"sonc"**```: SONC relaxation via geometric programming
- ```**"sosonc_sos_first"**```: Combined SOS+SONC optimization, solving SOS first
- ```**"sosonc_sonc_first"**```: Combined SOS+SONC optimization, solving SONC first

Returns a `RelaxResult` with attributes:

- **value** (float): Lower bound on the global minimum (`-inf` when the solver reports infeasibility)
- **status** (str): `"optimal"`, `"infeasible"`, or `"error"`
- **method** (str): The relaxation method that was run
- **runtime**, **init_time** (float): Wall-clock seconds for solve and setup phases
- **certificate**: SOS certificate when available (SDP path)
- **solver_info** (dict): Backend diagnostics; on the SDP path includes the relaxation order actually used under `"order"` plus the raw solver status string

The unified engine does not expose a moment-basis size in this contract: basis-size telemetry comes from the reduction pipeline (set `verbose_reduction=True` to print measured lines such as `[SDPRlx] Newton pruning: 28 -> 28`) or from the legacy `SDPRelaxations` interface documented in `sdp.rst`, whose result dict includes `basis_size`.

## 16.2 RelaxationConfig

The `RelaxationConfig` dataclass controls the reduction pipeline:

```
from Irene.relaxations import RelaxationConfig

config = RelaxationConfig(
    reduction_method="newton_polytope",  # "none" | "border_basis" | "newton_polytope" |
↪"sparsity"
    monomial_pruning=False,              # Enable/disable pruning
    sparsity_detection=False,            # Auto-detect correlative sparsity
    quotient_basis="groebner",           # "groebner" | "border" -- reduction engine
    verbose_reduction=False,             # Print reduction diagnostics
)
```

Fields:

- **reduction_method** (str): Basis reduction strategy. `"none"` uses the full monomial basis; `"border_basis"` applies border basis reduction; `"newton_polytope"` prunes via Newton polytope geometry; `"sparsity"` decomposes into independent SDP blocks.

- **monomial_pruning** (bool): Whether to apply monomial-level pruning within the chosen method
- **sparsity_detection** (bool): Whether to auto-detect and exploit correlative sparsity
- **quotient_basis** (str): Quotient-ring reduction engine used by `ReduceExp` and `ReducedMonomialBase`. `"groebner"` (default) uses the classical SymPy Groebner-basis reduction — the behavior of original Irene; `"border"` uses Irene's `BorderBasis` quotient-algebra reduction (numerically computed multiplication tables). The environment variable `IRENE_QUOTIENT_BASIS=groebner|border` sets the default when no explicit config is passed.
- **verbose_reduction** (bool): Print detailed reduction diagnostics during construction

By default, reductions are disabled for compatibility. Enable them explicitly with `reduction_method` or the corresponding boolean flags.

## 16.3 Two-Stage Hybrid Monoid-Graph Reduction Theorem

The real power of Irene's reduction pipeline lies in the **synergistic combination** of algebraic quotienting and structural graph decomposition. When all three reduction flags are enabled, the engine applies a two-stage reduction that composes monoid-theoretic elimination with chordal-graph decomposition.

**Theorem (Hybrid Monoid-Graph Reduction)**

Let $\mathcal{F} = \{f_0, f_1, \ldots, f_m\} \subset \mathbb{R}[S]$ be a polynomial/differential system with ideal $\mathcal{I} = \langle \mathcal{F} \rangle$, and let $t$ be the relaxation order.

**Stage 1 — Inner Algebraic Reduction (Monoid Quotienting).** The quotient basis at degree $2t$ is

$$B_{2t} = \operatorname{supp}\big(\mathbb{R}[S]/\mathcal{I}_{\leq 2t}\big),$$

computed via the selected `quotient_basis` engine (Gröbner or Border). The Newton polytope pruner further restricts this to

$$B_{2t}^{\text{pruned}} = B_{2t} \cap (2t \cdot \text{New}(\mathcal{F})).$$

**Stage 2 — Outer Structural Reduction (Chordal Graph Decomposition).** Construct the correlative sparsity graph $G = (V, E)$ on the vertex set $V = B_t^{\text{pruned}} \cap \Theta_{\leq t} Y$. After chordal completion, extract maximal cliques $\{C_1, \ldots, C_p\}$ satisfying the running intersection property. Each clique defines a local sub-basis

$$B_{t,k} = \{\alpha \in B_t^{\text{pruned}} : \operatorname{supp}(\alpha) \subseteq C_k\},$$

and the dense PSD constraint $M_{B_t}(y) \succeq 0$ is replaced by $p$ coupled, smaller PSD blocks:

$$M_{B_{t,k}}(y) \succeq 0, \qquad k = 1, \ldots, p.$$

The reduction factors are multiplicative: if Newton pruning yields a factor $r_N$ and chordal decomposition yields $r_C$, the total moment matrix dimension is reduced by $\approx r_N \cdot r_C$.

The pipeline is illustrated below:

```
Polynomial / Differential System  F
                  |
                  v
```

(continues on next page)

(continued from previous page)

```
[Inner Step] Monoid Quotienting (Border / Groebner)
    B_{2t} = supp( R[S] / I_{<=2t} )
                  |
                  v
[Newton Pruning]  B_{2t} cap (2t * New(F))
                  |
                  v
[Outer Step] Correlative Sparsity Graph G
    Vertices V = Pruned Basis
                  |
                  v
Chordal Completion & Clique Extraction  C_1, ..., C_p
                  |
                  v
Coupled Local Moment Blocks:  M_{B_{t,k}}(y) >= 0
```

### 16.3.1 Configuration

All three reductions are activated simultaneously through `RelaxationConfig`:

```
from Irene.relaxations import RelaxationConfig
from Irene.relaxation_api import RelaxationEngine

config = RelaxationConfig(
    reduction_method="border_basis",      # or "groebner"
    quotient_basis="border",              # quotient-ring engine
    monomial_pruning=True,                # enable Newton polytope pruning
    sparsity_detection=True,              # enable correlative sparsity
    verbose_reduction=True,
)
engine = RelaxationEngine(prog, order=2, config=config)
```

When `verbose_reduction=True`, the engine prints one line per reduction stage that actually ran. For example, on Motzkin at order 3 with the Newton-polytope pipeline (`reduction_method="newton_polytope"`, `monomial_pruning=True`, `sparsity_detection=True`) the measured output is:

```
[SDPRlx] Newton pruning: 28 -> 28 (100.00% retained)
[SDPRlx] Newton pruning: 10 -> 10 (100.00% retained)
```

The two lines are the moment-matrix basis (degree $2t = 6$, 28 monomials) and the order-$t$ basis (degree 3, 10 monomials). The border-basis stage is a no-op on this instance: Motzkin has no free relations, so the quotient ring is the full polynomial ring and no border basis is built. Correlative-sparsity decomposition likewise does not apply here: every non-constant term of Motzkin couples both variables, so the sparsity graph is a single component covering the whole basis and the moment matrix stays dense. Newton pruning itself is also a designed no-op on this dense instance – until a certified SOS-support reduction exists, the pipeline retains the full degree-bounded support as `required_support`, which makes the relaxed SDP identical to the unpruned baseline (bound `-526.08114071`, status `optimal` at orders 1–3). Measured runtimes in `benchmarks/results/p3_vs_baseline.json` give speedups of 0.69x–1.32x across the three test problems and all orders; no larger speedup is claimed on instances of this size.

## 16.4 compare_all()

The `compare_all` convenience function runs SOS, SONC, and SOSONC relaxations on the same problem and returns a comparison table:

```
from Irene.relaxation_api import compare_all

results = compare_all(prog, order=2, solver="clarabel")
# Returns dict with 'sos', 'sonc', 'sosonc' keys, each containing value/status/timing
```

This is useful for quick benchmarking and method comparison without writing separate engine instances.

CHAPTER

# SEVENTEEN

# APPROXIMATING OPTIMUM VALUE

In various cases, separating functions and symbols are either very difficult or impossible. For example $x, \sin x$ and $e^x$ are not algebraically independent, but their dependency can not be easily expressed in finitely many relations. One possible approach to these problems is replacing transcendental terms with a reasonably good approximation. This certainly will introduce more numerical error, but at least gives a reliable estimate for the optimum value.

## 17.1 Using `pyProximation.OrthSystem`

A simple and common method to approximate transcendental functions is using truncated Taylor expansions. In spite of its simplicity, there are various pitfalls which needs to be avoided. The most common is that the radius of convergence of the Taylor expansion may be smaller than the feasibility region of the optimization problem.

### 17.1.1 Example 1:

*Find the minimum of* $x + e^{x \sin x}$ *where* $-\pi \leq x \leq \pi$.

The objective function includes terms of $x$ and transcendental functions. So, it is difficult to find a suitable algebraic representation to transform this optimization problem. Let us try to use Taylor expansion of $e^{x \sin x}$ to find an approximation for the

```
  from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebraElement
  from Irene.program import OptimizationProblem
  from Irene.relaxations import SDPRelaxations
  from sympy import Symbol, Function, exp, sin, pi

  # introduce symbols and functions
  x = Symbol('x')
  e = Function('e')(x)
  # transcendental term of objective
  f = exp(x * sin(x))
  # Taylor expansion
  f_app = f.series(x, 0, 12).removeO()
  # Define semigroup and build problem with current API
  sg = CommutativeSemigroup(['x'])
  x_sg = sg.generators[0]
  objective = SemigroupAlgebraElement(sg, {sg.one: 1})  # placeholder; expand f_app␣
↪coefficients here
  prog = OptimizationProblem(sg, objective)
  # add support constraint: pi^2 - x^2 >= 0
```

(continues on next page)

(continued from previous page)

```
   constraint = SemigroupAlgebraElement(sg, {sg.one: float(pi**2), sg.monomial({0: 2}): -
↪1})
   prog.add_constraint(constraint >= 0)
   # Solve with SDP hierarchy
   sdp = SDPRelaxations(prog)
   result = sdp.solve(order=6)
   print(f"Lower bound: {result['value']:.6f}")
        # initialize the SDP
        Rlx.InitSDP()
        # solve the SDP
        Rlx.Minimize()
print(Rlx.Solution)
        # using scipy
        from scipy.optimize import minimize
        fun = lambda x: x[0] + exp(x[0] * sin(x[0]))
        cons = (
            {'type': 'ineq', 'fun': lambda x: pi**2 - x[0]**2},
        )
        sol1 = minimize(fun, (0, 0), method='COBYLA', constraints=cons)
        sol2 = minimize(fun, (0, 0), method='SLSQP', constraints=cons)
print("solution according to 'COBYLA':")
print(sol1)
print("solution according to 'SLSQP':")
print(sol2)



        def objfunc(x):
            from numpy import sin, exp, pi
            f = x[0] + exp(x[0] * sin(x[0]))
            g = [
                x[0]**2 - pi**2
            ]
            fail = 0
            return f, g, fail

        opt_prob = Optimization('A mixed function', objfunc)
        opt_prob.addVar('x1', 'c', lower=-pi, upper=pi, value=0.0)
        opt_prob.addObj('f')
        opt_prob.addCon('g1', 'i')
        # Augmented Lagrangian Particle Swarm Optimizer
        alpso = ALPSO()
        alpso(opt_prob)
print(opt_prob.solution(0))
        # Non Sorting Genetic Algorithm II
        nsg2 = NSGA2()
        nsg2(opt_prob)
print(opt_prob.solution(1))
```

The output will look like:

```
Solution of a Semidefinite Program:
```

(continues on next page)

(continued from previous page)

```
               Solver: CVXOPT
               Status: Optimal
   Initialization Time: 0.270121097565 seconds
             Run Time: 0.012974 seconds
Primal Objective Value: -416.628918881
  Dual Objective Value: -416.628917197
Feasible solution for moments of order 5

solution according to 'COBYLA':
     fun: 0.76611902154788758
   maxcv: 0.0
 message: 'Optimization terminated successfully.'
    nfev: 34
  status: 1
 success: True
       x: array([ -4.42161128e-01,  -9.76206736e-05])
solution according to 'SLSQP':
     fun: 0.766119450232887
     jac: array([ 0.00154828,  0.        ,  0.        ])
 message: 'Optimization terminated successfully.'
    nfev: 17
     nit: 4
    njev: 4
  status: 0
 success: True
       x: array([-0.44164406,  0.        ])

ALPSO Solution to A mixed function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.0683
    Total Function Evaluations:      1240
    Lambda:     [ 0.]
    Seed: 1482112089.31088901

    Objectives:
        Name        Value        Optimum
             f        -2.14159            0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -3.141593      -3.14e+00     3.14e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
             g1           i       -1.00e+21 <= 0.000000 <= 0.00e+00

--------------------------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
NSGA-II Solution to A mixed function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.4231
    Total Function Evaluations:

    Objectives:
        Name        Value        Optimum
             f        -2.14159             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -3.141593      -3.14e+00     3.14e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
             g1           i       -1.00e+21 <= 0.000000 <= 0.00e+00

--------------------------------------------------------------------------------
```

Now instead of Taylor expansion, we use Legendre polynomials to estimate $e^{x \sin x}$. To find Legendre estimators, we use pyProximation which implements general Hilbert space methods (see Appendix-*pyProximation*):

```
       from sympy import *
import Irene
       from pyProximation import OrthSystem
       # introduce symbols and functions
       x = Symbol('x')
       e = Function('e')(x)
       # transcendental term of objective
       f = exp(x * sin(x))
       # Legendre polynomials via pyProximation
       D = [(-pi, pi)]
       S = OrthSystem([x], D)
       # set B = {1, x, x^2, ..., x^12}
       B = S.PolyBasis(12)
       # link B to S
       S.Basis(B)
       # generate the orthonormal basis
       S.FormBasis()
       # extract the coefficients of approximation
       Coeffs = S.Series(f)
       # form the approximation
       f_app = sum([S.OrthBase[i] * Coeffs[i] for i in range(len(S.OrthBase))])
       # initiate the Relaxation object
       Rlx = SDPRelaxations([x])
```

(continues on next page)

(continued from previous page)

```
        # set the objective
        Rlx.SetObjective(x + f_app)
        # add support constraints
        Rlx.AddConstraint(pi**2 - x**2 >= 0)
        # set the solver
        Rlx.SetSDPSolver('dsdp')
        # initialize the SDP
        Rlx.InitSDP()
        # solve the SDP
        Rlx.Minimize()
print(Rlx.Solution)
```

The output will be:

```
Solution of a Semidefinite Program:
                Solver: DSDP
                Status: Optimal
   Initialization Time: 0.722383022308 seconds
              Run Time: 0.077674 seconds
Primal Objective Value: -2.26145824829
  Dual Objective Value: -2.26145802066
Feasible solution for moments of order 6
```

By a small modification of the above code, we can employ Chebyshev polynomials for approximation:

```
        from sympy import *
import Irene
        from pyProximation import Measure, OrthSystem
        # introduce symbols and functions
        x = Symbol('x')
        e = Function('e')(x)
        # transcendental term of objective
        f = exp(x * sin(x))
        # Chebyshev polynomials via pyProximation
        D = [(-pi, pi)]
        # the Chebyshev weight
        w = lambda x: 1. / sqrt(pi**2 - x**2)
        M = Measure(D, w)
        S = OrthSystem([x], D)
        # link the measure to S
        S.SetMeasure(M)
        # set B = {1, x, x^2, ..., x^12}
        B = S.PolyBasis(12)
        # link B to S
        S.Basis(B)
        # generate the orthonormal basis
        S.FormBasis()
        # extract the coefficients of approximation
        Coeffs = S.Series(f)
        # form the approximation
        f_app = sum([S.OrthBase[i] * Coeffs[i] for i in range(len(S.OrthBase))])
        # initiate the Relaxation object
```

(continues on next page)

(continued from previous page)

```
        Rlx = SDPRelaxations([x])
        # set the objective
        Rlx.SetObjective(x + f_app)
        # add support constraints
        Rlx.AddConstraint(pi**2 - x**2 >= 0)
        # set the solver
        Rlx.SetSDPSolver('dsdp')
        # initialize the SDP
        Rlx.InitSDP()
        # solve the SDP
        Rlx.Minimize()
print(Rlx.Solution)
```

which returns:

```
Solution of a Semidefinite Program:
                Solver: DSDP
                Status: Optimal
   Initialization Time: 0.805300951004 seconds
              Run Time: 0.066767 seconds
Primal Objective Value: -2.17420785198
  Dual Objective Value: -2.17420816422
Feasible solution for moments of order 6
```

This gives a better approximation for the optimum value. The optimum values found via Legendre and Chebyshev polynomials are certainly better than Taylor expansion and the results of `scipy.optimize`.

### 17.1.2 Example 2:

*Find the minimum of* $x \sinh y + e^{y \sin x}$ *where* $-\pi \leq x, y \leq \pi$.

Again, we use Legendre approximations for $\sinh y$ and $e^{y \sin x}$:

```
        from sympy import *
import Irene
        from pyProximation import OrthSystem
        # introduce symbols and functions
        x = Symbol('x')
        y = Symbol('y')
        sh = Function('sh')(y)
        ch = Function('ch')(y)
        # transcendental term of objective
        f = exp(y * sin(x))
        g = sinh(y)
        # Legendre polynomials via pyProximation
        D_f = [(-pi, pi), (-pi, pi)]
        D_g = [(-pi, pi)]
        Orth_f = OrthSystem([x, y], D_f)
        Orth_g = OrthSystem([y], D_g)
        # set bases
        B_f = Orth_f.PolyBasis(10)
        B_g = Orth_g.PolyBasis(10)
        # link B_f to Orth_f and B_g to Orth_g
```

(continues on next page)

(continued from previous page)

```
        Orth_f.Basis(B_f)
        Orth_g.Basis(B_g)
        # generate the orthonormal bases
        Orth_f.FormBasis()
        Orth_g.FormBasis()
        # extract the coefficients of approximations
        Coeffs_f = Orth_f.Series(f)
        Coeffs_g = Orth_g.Series(g)
        # form the approximations
        f_app = sum([Orth_f.OrthBase[i] * Coeffs_f[i]
                     for i in range(len(Orth_f.OrthBase))])
        g_app = sum([Orth_g.OrthBase[i] * Coeffs_g[i]
                     for i in range(len(Orth_g.OrthBase))])
        # initiate the Relaxation object
        Rlx = SDPRelaxations([x, y])
        # set the objective
        Rlx.SetObjective(x * g_app + f_app)
        # add support constraints
        Rlx.AddConstraint(pi**2 - x**2 >= 0)
        Rlx.AddConstraint(pi**2 - y**2 >= 0)
        # set the sdp solver
        Rlx.SetSDPSolver('cvxopt')
        # initialize the SDP
        Rlx.InitSDP()
        # solve the SDP
        Rlx.Minimize()
print(Rlx.Solution)
        # using scipy
        from scipy.optimize import minimize
        fun = lambda x: x[0] * sinh(x[1]) + exp(x[1] * sin(x[0]))
        cons = (
            {'type': 'ineq', 'fun': lambda x: pi**2 - x[0]**2},
            {'type': 'ineq', 'fun': lambda x: pi**2 - x[1]**2}
        )
        sol1 = minimize(fun, (0, 0), method='COBYLA', constraints=cons)
        sol2 = minimize(fun, (0, 0), method='SLSQP', constraints=cons)
print("solution according to 'COBYLA':")
print(sol1)
print("solution according to 'SLSQP':")
print(sol2)



        def objfunc(x):
            from numpy import sin, sinh, exp, pi
            f = x[0] * sinh(x[1]) + exp(x[1] * sin(x[0]))
            g = [
                x[0]**2 - pi**2,
                x[1]**2 - pi**2
            ]
            fail = 0
            return f, g, fail
```

(continues on next page)

(continued from previous page)

```
        opt_prob = Optimization(
            'A trigonometric-hyperbolic-exponential function', objfunc)
        opt_prob.addVar('x1', 'c', lower=-pi, upper=pi, value=0.0)
        opt_prob.addVar('x2', 'c', lower=-pi, upper=pi, value=0.0)
        opt_prob.addObj('f')
        opt_prob.addCon('g1', 'i')
        opt_prob.addCon('g2', 'i')
        # Augmented Lagrangian Particle Swarm Optimizer
        alpso = ALPSO()
        alpso(opt_prob)
print(opt_prob.solution(0))
        # Non Sorting Genetic Algorithm II
        nsg2 = NSGA2()
        nsg2(opt_prob)
print(opt_prob.solution(1))
```

The result will be:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 4.09241986275 seconds
              Run Time: 0.123869 seconds
Primal Objective Value: -35.3574475835
  Dual Objective Value: -35.3574473266
Feasible solution for moments of order 5

solution according to 'COBYLA':
     fun: 1.0
   maxcv: 0.0
 message: 'Optimization terminated successfully.'
    nfev: 13
  status: 1
 success: True
       x: array([ 0.,  0.])
solution according to 'SLSQP':
     fun: 1
     jac: array([ 0.,  0.,  0.])
 message: 'Optimization terminated successfully.'
    nfev: 4
     nit: 1
    njev: 1
  status: 0
 success: True
       x: array([ 0.,  0.])

ALPSO Solution to A trigonometric-hyperbolic-exponential function
================================================================================

        Objective Function: objfunc
```

(continues on next page)

(continued from previous page)

```
    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.0946
    Total Function Evaluations:      1240
    Lambda: [ 0.  0.]
    Seed: 1482112613.82665610

    Objectives:
        Name        Value        Optimum
             f        -35.2814            0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -3.141593      -3.14e+00     3.14e+00
             x2       c       3.141593      -3.14e+00     3.14e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
             g1           i       -1.00e+21 <= 0.000000 <= 0.00e+00
             g2           i       -1.00e+21 <= 0.000000 <= 0.00e+00

--------------------------------------------------------------------------------


NSGA-II Solution to A trigonometric-hyperbolic-exponential function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.5331
    Total Function Evaluations:

    Objectives:
        Name        Value        Optimum
             f        -35.2814            0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c       3.141593      -3.14e+00     3.14e+00
             x2       c      -3.141593      -3.14e+00     3.14e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
             g1           i       -1.00e+21 <= 0.000000 <= 0.00e+00
             g2           i       -1.00e+21 <= 0.000000 <= 0.00e+00

--------------------------------------------------------------------------------
```

which shows a significant improvement compare to results of `scipi.minimize`.

CHAPTER

EIGHTEEN

# NON-POLYNOMIAL OPTIMIZATION (NONPOPSDP)

The `nonpopsdp.py` module (ported from original Irene in 2026-08-09) applies Lasserre's moment-SOS hierarchy to optimization problems whose objective or constraints involve transcendental functions (`exp`, `sin`, `cos`, ...).

## 18.1 Pipeline

1. **Approximate** each transcendental function by a polynomial surrogate (Taylor or Chebyshev).
2. **Substitute** the surrogates into the objective/constraints, producing a polynomial optimization problem (POP).
3. **Relax** the POP via `SDPRelaxations` (Lasserre hierarchy).
4. **Solve** the SDP (CVXOPT by default; other backends via `SDPRelaxations`).

Per Josz–Henrion [JH16], a redundant ball constraint is ALWAYS added to the relaxation to guarantee strong duality (no primal–dual gap).

## 18.2 Quick Example

```
from math import exp
import sympy as sp
from Irene.nonpopsdp import NonPOPSDP

x = sp.symbols("x")
exp_sym = sp.symbols("exp")            # bare symbol named after the function

pop = NonPOPSDP(
    x,
    {"exp": {"func": exp, "method": "chebyshev",
             "domain": (-1.0, 1.0), "degree": 6}},
    relax_order=2, ball_radius=1.0, verbosity=0,
)
pop.set_objective(exp_sym)             # min exp(x) on [-1, 1]
lb = pop.solve()                       # ~ 0.3679 (true min exp(-1))
```

Function surrogates are referenced by **bare symbols named after the function** (`symbols('sin')`), not by `sp.sin(x)` — `TranscendentalApproximator.substitute` replaces the named symbol with the polynomial surrogate.

## 18.3 API

- `taylor_approx(func, var, center, degree)` — Taylor surrogate with Lagrange remainder bound.
- `chebyshev_approx(func, var, domain, degree)` — Chebyshev surrogate on a domain with empirical max error.
- `TranscendentalApproximator(var, approx_map)` — builds and substitutes several surrogates at once.
- `NonPOPSDP(var, approx_map, relax_order, ball_radius, ...)` — single-variable pipeline (`set_objective`, `add_constraint`, `solve`).
- `NonPOPSDP_Multi(vars, approx_map, ...)` — multi-variable pipeline with per-function `var_idx`.

## 18.4 Numerical Fixes in the Port (vs original Irene)

The original implementation had two latent numerical bugs, both fixed in this port (verified against original Irene):

1. **Chebyshev coefficients** were extracted with an incorrectly scaled raw FFT, producing catastrophically wrong surrogates (max error ~61.5 for `exp` of degree 6 on `[-2, 2]`; the true error is ~5e-4). The port uses `numpy.polynomial.chebyshev.chebfit` and also fixed an off-by-one in the error-evaluation grid (`fine_t = 2(x-mid)/(b-a) - 1` mapped `[a,b]` onto `[-2, 0]`).
2. **Taylor coefficients** were computed with naive central finite differences (error ~1e36 for `exp` at degree 6). The port uses a high-order central-difference stencil with Richardson extrapolation at 60-digit precision (`_mp_derivative`), accurate to ~1e-6 at degree 7.

The pipeline API and SDP construction are otherwise faithful to the original.

## 18.5 Integration Notes

- `NonPOPSDP.solve` accepts an optional `config` (`RelaxationConfig`), so the surrogate POP inherits the quotient-basis and reduction-pipeline options (see *Unified Relaxation API*).
- The module is backend-agnostic: surrogates are SymPy expressions, and the SDP construction routes through the user-selectable symbolic engine.

CHAPTER

# NINETEEN

# DIFFERENTIAL SDP AND MEAN RELAXATIONS

The `dsdp.py` module implements differential semidefinite programming relaxations bridged through SymEngine for symbolic computation. It extends the standard moment/SDP hierarchy to handle optimization problems where polynomial terms include functions that are solutions of algebraic differential equations (ADEs).

## 19.1 Differential SDP Connection

Standard SDP hierarchies for polynomial optimization work with the cone of sums of squares and moment matrices over monomial bases. The **differential SDP** extension handles a broader class of problems by incorporating:

1. **Jet prolongation**: Extending variables to include derivatives $y, y', y'', \ldots$ up to a fixed order
2. **ADE constraints**: Encoding algebraic differential equations as polynomial constraints on the jet space
3. **Differential moment matrices**: Moment structures that respect the derivation operator

The module provides SymEngine-bridged implementations of these constructs, enabling symbolic manipulation of differential polynomials before numerical SDP formulation.

## 19.2 Mean Polynomial Forms

The mean relaxation uses weighted power mean forms as certificates of nonnegativity:

$$M_{q,p}(X, w) = \sum_i w_i X_i^p \left( \sum_j w_j X_j^q \right)^{\frac{p-q}{q}}.$$

These forms generalize both SOS and SONC certificates. The mean polynomial cone $\mathcal{M}_{n,2d}$ contains the SOS cone when $p$ divides $2d$, and strictly contains it for other parameter choices (see the companion manuscript on mean polynomials, Chapters 1–7).

The DSDP module constructs relaxations using:

- **Mean-based moment matrices**: PSD constraints on generalized moment structures derived from power means
- **SymEngine polynomial arithmetic**: Exact symbolic manipulation before numerical evaluation
- **Derivation-aware basis construction**: Monomial bases that respect the derivation operator structure

## 19.3 Convergence Theory: CGIK Framework and Archimedean Conditions

The convergence of the differential SDP hierarchy is grounded in the **Curto–Ghasemi–Infusino–Kuhlmann (CGIK)** framework for the truncated moment problem on unital commutative algebras [CGIK]. In the differential setting, the algebra is the quotient

$$A = \mathbb{R}[S]/\mathcal{I}_{\mathrm{ADE}},$$

where $S$ is the semigroup generated by the original variables together with their jet prolongations (derivative symbols), and $\mathcal{I}_{\mathrm{ADE}}$ is the differential ideal encoding the ADE constraints. The CGIK theorem guarantees that a representing measure exists on the character space $\hat{A}$ provided that the quadratic module generated by the constraints is Archimedean.

**Archimedean Boxing Condition.** For Putinar-type representation theorems to apply on the jet space, the quadratic module must contain an element of the form

$$R^2 - \|x\|^2 - \sum_j y_j^2$$

for some $R > 0$. This is the **Archimedean boxing condition**, enforced by the `archimedean=True` and `box_size` parameters of `DSDPRelaxations`. Geometrically, it restricts the feasible set to a bounded subset of the jet space, guaranteeing that the moment hierarchy produces a monotone non-decreasing sequence of lower bounds converging to the true optimum.

**Stochel's Theorem on Infinite-Dimensional Quotients.** When the ADE encodes transcendental functions (e.g., $y' = y$ for the exponential), the quotient algebra $\mathbb{R}[S]/\mathcal{I}_{\mathrm{ADE}}$ is infinite-dimensional as a real vector space. Stochel's theorem [Stochel2001] ensures that, under the Archimedean condition, every $A$-positive linear functional admits an integral representation via a Borel measure on the character space of $A$. This provides the theoretical foundation for the moment hierarchy to converge on differential-algebraic constraint sets.

**Practical Convergence Guarantees.**

1. With `archimedean=True` (default), the hierarchy produces certified lower bounds that approach the true optimum from below as the relaxation order increases.
2. The boxing constant $R$ (`box_size`, default 10) must be chosen large enough to contain the feasible set; an overly small $R$ may exclude the true optimum.
3. Jet prolongation at order $k$ (`jet_order`) increases the problem dimension by a factor of $(k+1)$ per differentiated variable but tightens the relaxation — the hierarchy is guaranteed to converge in the limit $\min(\text{relaxation order}, \text{jet order}) \to \infty$.

## 19.4 API Overview

```
from Irene.dsdp import DSDPRelaxations

# Construct differential SDP relaxation for a problem with ADE constraints
dsdp = DSDPRelaxations(prog, jet_order=2)

# Solve at specified order
result = dsdp.solve(order=2)
print(f"Differential SDP bound: {result['value']:.6f}")
```

The `DSDPRelaxations` class accepts an `OptimizationProblem` and a `jet_order` parameter controlling the derivative depth. The solve method returns results in the same dictionary format as standard relaxations (keys: `value`, `status`, `order`, `basis_size`, timing fields).

## 19.5 SymEngine Bridging

The module uses SymEngine for symbolic polynomial arithmetic with automatic fallback to SymPy when SymEngine's Poly API lacks a required operation. This dual-engine design ensures correctness while maximizing performance for the common case. Key bridged operations include:

- Polynomial multiplication and addition in the jet space
- Derivation operator application ($d_x, d_y$ as operators, not Leibniz fractions)
- Ideal membership testing via border basis reduction

## 19.6 Practical Notes

1. The DSDP module is research-grade — it implements the theoretical framework from the differential Positivstellensatz rough ideas but should be validated against known benchmarks before production use
2. Jet prolongation increases problem dimension by a factor of $(jet_order + 1)$ per differentiated variable
3. The SymEngine bridge adds minimal overhead for small problems but provides significant speedups for symbolic preprocessing at higher orders

## 19.7 References


- Ghasemi, M. (2026). "Mean Polynomials: Generalizing SOS and SONC via Power Mean Forms." *Companion manuscript*, Chapters 1–7.
- Curto, R. E., Ghasemi, M., Infusino, M., & Kuhlmann, S. (2023). "The truncated moment problem for unital commutative $\mathbb{R}$-algebras." *Journal of Operator Theory*, 90(1), 223–261. DOI: 10.7900/jot.2021nov26.2392.
- Ritt, J. F. (1950). "Differential Algebra." American Mathematical Society Colloquium Publications.
- Kolchin, E. R. (1973). "Differential Algebra and Algebraic Groups." Academic Press.

CHAPTER

# TWENTY

# CVXPY SOLVER LAYER

The `cvxpy_solver.py` module provides a DCP-compliant solver layer that bridges Irene's SDP relaxation constructs to modern convex optimization backends via CVXPY. It supports CLARABEL (default), SCS, and native CVXOPT as solver backends.

## 20.1 Architecture

The CVXPY layer sits between Irene's moment matrix construction and the actual numerical solver. It handles:

1. **DCP formulation**: Translates moment matrix PSD constraints into CVXPY's disciplined convex programming framework
2. **Solver routing**: Selects and configures the backend solver based on problem size and user preference
3. **Result extraction**: Parses solver output back into Irene's result structure with timing metadata

### 20.1.1 Solver Routing Behavior

The default solver is **CLARABEL** (an interior-point method with robust handling of ill-conditioned moment matrices). The routing logic:

```
from Irene.cvxpy_solver import CvxpySDP

sdp = CvxpySDP(moment_matrix, localizing_matrices)
result = sdp.solve(solver='CLARABEL')  # default
result = sdp.solve(solver='SCS')       # ADMM-based, faster for large problems
result = sdp.solve(solver='CVXOPT')    # native fallback
```

### 20.1.2 Infeasibility Detection and Positivstellensatz Duality

An important distinction between backends affects correctness of nonnegativity certificates:

- **Native CVXOPT (C interface)**: Correctly reports `'infeasible'` for SDPs whose primal is infeasible. This is the reliable path for SOS certification: when CVXOPT declares infeasibility, it means the moment matrix cannot be made PSD while satisfying constraints, which is equivalent to a **dual SOS proof of nonnegativity** via Putinar's Positivstellensatz.
- **CLARABEL (via CVXPY)**: May return finite weak bounds with status `'optimal'` for infeasible SDPs, because its interior-point method interprets primal infeasibility differently. CLARABEL's dual unboundedness certificate is mathematically equivalent to a Putinar-type representation, but the solver may terminate with a weak bound rather than a clean `'infeasible'` status.

**Conic Duality Guide.** In the Moment-SOS hierarchy, the primal SDP minimizes $L(f)$ subject to $M_t(y) \succeq 0$ and $M_t(g_i y) \succeq 0$. Its dual maximizes $\gamma$ such that $f - \gamma$ admits a representation

$$f - \gamma = \sigma_0 + \sum_i \sigma_i g_i, \qquad \sigma_0, \ldots, \sigma_m \in \sum \mathbb{R}[x]^2_{\leq 2t}.$$

A **dual unboundedness certificate** from CLARABEL (or an **infeasible** status from native CVXOPT) corresponds exactly to a certified SOS decomposition proving $f \geq \gamma$ on the semialgebraic set $K = \{x : g_i(x) \geq 0\}$. Both solvers therefore produce valid certificates; the difference is only in how they report the status.

**Solver Selection Rule.**

- Use native CVXOPT when **correct infeasibility detection is critical** (e.g., proving a polynomial is NOT SOS, as in the Motzkin and Choi–Lam gallery problems). The Irene engine routes `solver='cvxopt'` through CVXOPT's native C interface for this reason.
- Prefer CLARABEL for **large-scale feasible problems** (up to ~500 moment variables) where its robust interior-point convergence is valuable and infeasibility is not expected.
- Use SCS (`solver='scs'`) when moment matrix dimension exceeds $500 \times 500$; its first-order ADMM method scales better but requires tighter tolerances for certificate-quality bounds.

**Numerical Parameter Recommendations.** For high-order relaxations ($t \geq 3$) where moment matrices become ill-conditioned:

Table 1: Solver tolerance settings for high-order SDP

| Solver | Tolerance parameter(s) | Typical value |
|---|---|---|
| CLARABEL | `tol_gap_abs`, `tol_feas` | `1e-8` |
| SCS | `eps_abs` | `1e-6` (tighten to `1e-7` when $M_t(y) > 500$ $times500$) |
| CVXOPT (native) | `abstol`, `reltol`, `feastol` | defaults adequate up to $t = 3$; raise `feastol` to `1e-7` for $t$ $ge4$ |

## 20.2 API Reference

### 20.2.1 CvxpySDP Class

```
from Irene.cvxpy_solver import CvxpySDP

# Construct from moment matrix and constraint blocks
sdp = CvxpySDP(M, A_blocks, c_vector)

# Solve with default solver (CLARABEL)
result = sdp.solve()

# Solve with specific backend
result = sdp.solve(solver='SCS', verbose=True)
```

The constructor accepts:

- **M** (matrix): The moment matrix template for PSD constraints
- **A_blocks** (list): Linear constraint blocks $\sum_i y_i A_i$
- **c_vector** (array): Objective coefficients

Returns a result dictionary with keys: `value`, `status`, `time_solve`, `solver_used`.

## 20.3 Performance Notes

1. **CLARABEL** is recommended for problems up to ~500 moment variables (orders 2–3 in bivariate settings)
2. **SCS** becomes competitive for larger instances due to its first-order method scaling, but may require tighter tolerances for certificate-quality bounds
3. The CVXPY layer adds ~10–20% overhead vs direct solver calls due to DCP graph construction, but provides uniform API and robust error handling

CHAPTER

# TWENTYONE

# BENCHMARKS AND PERFORMANCE EVALUATION

This chapter documents the benchmarking infrastructure for Irene, including the problem gallery system, performance comparison scripts, and representative examples using the current API.

## 21.1 Benchmark Problem Gallery

The `benchmarks/gallery.yaml` file defines a structured catalog of polynomial optimization problems with known properties, expected relaxation results, and metadata for filtering. Each entry specifies:

- **id**: Unique string identifier (e.g. `motzkin`, `choi_lam`)
- **name**: Human-readable name
- **description**: Mathematical description of the problem
- **variables**: List of variable names
- **degree**: Total degree of objective polynomial(s)
- **category**: One of `unconstrained`, `constrained`, `separating`, `mean_poly`
- **objective**: Polynomial expression in SymPy-compatible syntax
- **constraints**: Optional list of constraint dicts with `expr` and `type` keys
- **true_min**: Known global minimum (or `null` if unknown)
- **relaxations**: Expected relaxation results at various orders
- **tags**: Keywords for filtering

### 21.1.1 Gallery Runner

The `benchmarks/run_gallery.py` script loads the gallery, constructs each problem via Irene's API, runs SOS/SONC/SOSONC relaxations at specified orders, and records structured JSON results for regression tracking:

```
# Run full gallery with default CLARABEL solver
python benchmarks/run_gallery.py

# Filter by tag, use SCS solver, custom tolerance
python benchmarks/run_gallery.py --filter separating --solver scs --tolerance 1e-4

# Custom output directory and timeout
python benchmarks/run_gallery.py --output-dir ./results/ --timeout 600
```

The runner constructs problems using the semigroup algebra pattern:

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem

sg = CommutativeSemigroup(variables)
sga = SemigroupAlgebra(sg)
sym_dict = {v: sga[v] for v in variables}

objective = eval(obj_expr, {"__builtins__": {}}, sym_dict)
prog = OptimizationProblem(sga)
prog.set_objective(objective)
```

### 21.1.2 Representative Gallery Problems

**Motzkin Polynomial**: The canonical separating example. Nonnegative by AM-GM but not SOS: it is itself a single nonnegative circuit polynomial (support {(4,2), (2,4), (0,0)}, total degree 6 — the negative term -3x^2 y^2 sits at (2,2) = 1/3(4,2) + 1/3(2,4) + 1/3(0,0)), so the SONC-vs-SOS separation is stated at total degree rather than at a relaxation order.

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.sosonc import SOSONCRelaxations

sg = CommutativeSemigroup(['x', 'y'])
sga = SemigroupAlgebra(sg)
x, y = sga['x'], sga['y']

prog = OptimizationProblem(sga)
motzkin = x**4 * y**2 + x**2 * y**4 + 1 - 3 * x**2 * y**2
prog.set_objective(motzkin)

# order is set via the constructor kwarg (globalMinSOS takes no arguments)
sosonc = SOSONCRelaxations(prog, relaxation_order=3)
result = sosonc.globalMinSOS()
print(f"SOS lower bound (order 3): {result:.6f}")
```

**Choi-Lam Polynomial**: Nonnegative on $\mathbb{R}^2$, not SOS. Zero at $(0,0), (\pm 1, 0), (0, \pm 1)$. Used in the Mean Polynomial paper as a separating example.

```
choi_lam = x**4 * y**2 + x**2 * y**4 + x**2 * y**2 * (x**2 + y**2 - 1)
prog.set_objective(choi_lam)
result_sonc = sosonc.globalMinSONC(order=3)
print(f"SONC lower bound (order 3): {result_sonc:.6f}")
```

**Robinson Polynomial**: A degree-6 bivariate with known minimum. Frequently used as a stress test for hierarchy convergence.

```
robinson = x**4 * y**2 + x**2 * y**4 + x**4 + y**4 - x**2 - y**2
prog.set_objective(robinson)
result_combined = sosonc.globalMinSOSPSONC(order=3, first='sos')
print(f"SOS+SONC lower bound: {result_combined:.6f}")
```

## 21.2 Phase 3 Optimization Benchmarks

The `benchmarks/p3_vs_baseline.py` script compares the baseline configuration (no reduction pipeline) against the Phase 3 optimized configuration (Newton polytope pruning + border basis + correlative sparsity detection):

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.relaxation_api import RelaxationEngine
from Irene.relaxations import RelaxationConfig

# Build problem (e.g., Motzkin)
sg = CommutativeSemigroup(["x", "y"])
sa = SemigroupAlgebra(sg)
x, y = sa["x"], sa["y"]
prog = OptimizationProblem(sa)
prog.set_objective(x**4 * y**2 + x**2 * y**4 + 1 - 3 * x**2 * y**2)

# Baseline: no reductions
baseline_config = RelaxationConfig(
    reduction_method="none",
    monomial_pruning=False,
    sparsity_detection=False,
)

# Phase 3 optimized: full reduction pipeline
p3_config = RelaxationConfig(
    reduction_method="newton_polytope",
    monomial_pruning=True,
    sparsity_detection=True,
    verbose_reduction=False,
)

engine = RelaxationEngine(prog, order=2, solver="clarabel", config=p3_config)
result = engine.solve("sos")
print(f"Value: {result.value:.8f}, Status: {result.status}")
```

The comparison measures matrix dimension, generation time, solve time, and final bound for each configuration across orders 1–3.

### 21.2.1 Cross-Version Comparison

The `benchmarks/compare_irene_vs_rewrite.py` script runs the same problems through both the original Irene package and Irene to validate numerical consistency:

```
python benchmarks/compare_irene_vs_rewrite.py
```

This verifies that Phase 3 optimizations (Newton pruning, border basis, sparsity) produce bounds within tolerance of the baseline while reducing matrix dimensions.

## 21.3 Constrained Optimization Examples

The following examples demonstrate constrained polynomial optimization using the current Irene API pattern with semigroup algebras.

### 21.3.1 Bounded Quartic Minimization

Minimize $x^2 - 4x$ subject to $4 - x^2 \geq 0$:

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.relaxations import SDPRelaxations

sg = CommutativeSemigroup(['x'])
sga = SemigroupAlgebra(sg)
x = sga['x']

prog = OptimizationProblem(sga)
prog.set_objective(x**2 - 4 * x)
prog.add_constraints([4 - x**2])

sdp = SDPRelaxations(prog, verbosity=0)
for t in range(1, 5):
    result = sdp.solve(order=t)
    print(f"Order {t}: bound = {result['value']:.6f}, "
          f"time = {result['time_solve']:.3f}s, "
          f"basis = {result['basis_size']}")
```

### 21.3.2 Trigonometric Polynomial via Algebraic Substitution

Minimize $\cos^2(x) + \sin^2(y)$ over $-5 \leq x, y \leq 5$ by substituting $s_1 = \sin(x), c_1 = \cos(x), s_2 = \sin(y), c_2 = \cos(y)$ and adding the algebraic relations $s_i^2 + c_i^2 = 1$:

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem
from Irene.relaxations import SDPRelaxations

sg = CommutativeSemigroup(['s1', 'c1', 's2', 'c2'])
sga = SemigroupAlgebra(sg)
s1, c1, s2, c2 = sga['s1'], sga['c1'], sga['s2'], sga['c2']

prog = OptimizationProblem(sga)
prog.set_objective(c1**2 + s2**2)

# Algebraic relations: sin^2 + cos^2 = 1 (encoded as equality via pair of inequalities)
rel1 = 1 - s1**2 - c1**2
rel2 = 1 - s2**2 - c2**2
prog.add_constraints([rel1, -rel1, rel2, -rel2])

# Box constraints: x^2 <= 25, y^2 <= 25 (approximated)
prog.add_constraints([25 - s1**2, 25 - s2**2])

sdp = SDPRelaxations(prog, verbosity=0)
```

(continues on next page)

(continued from previous page)

```
result = sdp.solve(order=2)
print(f"Lower bound: {result['value']:.8f}")
```

## 21.4 Performance Profiling Tools

Irene includes several profiling scripts for diagnosing performance bottlenecks:

- **``instrument_relaxation_v2.py``**: Instruments the relaxation pipeline with per-stage timing (basis construction, matrix assembly, solver call)
- **``profile_symengine_overhead.py``**: Measures SymEngine vs SymPy overhead in polynomial arithmetic operations
- **``stage_bc_detailed.py``**: Fine-grained breakdown of border basis and sparsity detection stages
- **``p3_diagnose.py``**: Diagnostic script for Phase 3 reduction pipeline behavior

## 21.5 Solver Routing Behavior

Irene routes SDP solves through multiple backends:

1. **CVXPY + CLARABEL** (default): Robust handling of ill-conditioned moment matrices and reliable infeasibility detection
2. **Native CVXOPT**: Fallback path; note that infeasibility detection can differ from CLARABEL due to tolerance handling
3. **External solvers** (DSDP, SDPA, CSDP): For very large instances via CLI invocation

The solver is selected via the `solver` parameter:

```
result = sdp.solve(order=4, solver='clarabel')    # default
result = sdp.solve(order=4, solver='cvxopt')      # native fallback
result = sdp.solve(order=4, solver='dsdp')        # external CLI
```

### 21.5.1 Return Structure

The `solve()` method returns a dictionary with these keys:

- `value` (float): Primal objective value (lower bound on minimum)
- `status` (str): Solver status string ('optimal', 'infeasible', etc.)
- `order` (int): Relaxation order used
- `basis_size` (int): Number of moment variables
- `time_init` (float): SDP construction time in seconds
- `time_solve` (float): Solver runtime in seconds

## 21.6 References


- Lasserre, J.-B. (2001). "Global optimization with polynomials and the problem of moments." *SIAM Journal on Optimization*, 11(3), 796–817.
- Parrilo, P. A. (2000). "Structured semidefinite programs and semialgebraic geometry methods in robustness and optimization." *Caltech PhD Thesis*.

CHAPTER

# TWENTYTWO

# EXAMPLES AND VALIDATION

This chapter lists runnable entry points that exercise the three method families. All example scripts live in the `examples/` directory at the repository root (except the benchmark runners, which live in `benchmarks/`).

## 22.1 Recommended Example Sequence

1. `examples/Rosenbrock.py` — SDP hierarchy on a classic benchmark.
2. `examples/GPExample.py` — geometric relaxation flow via GP.
3. `examples/SONCExample.py` and `examples/SONCExample33.py` — SONC circuit-polynomial relaxations.

## 22.2 API Quick Reference

| Script | Primary classes | Solver dependency |
|---|---|---|
| `examples/Rosenbrock.py` | `SDPRelaxations` | CVXPY/CLARABEL (default) or CVXOPT |
| `examples/GPExample.py` | `OptimizationProblem`, `GPRelaxations` | GP solver backend |
| `examples/SONCExample.py` | `OptimizationProblem`, `SONCRelaxations` | GP solver backend |
| `examples/SONCExample33.py` | `OptimizationProblem`, `SONCRelaxations` | GP solver backend |

## 22.3 SDP Example

Run from the repository root with the virtual environment activated:

```
source .venv/bin/activate
python examples/Rosenbrock.py
```

Expected behavior:

1. Initializes an `SDPRelaxations` object with semigroup-algebra expressions.
2. Solves an SDP lower-bound problem via the selected solver (CLARABEL by default).
3. Prints solver summary, objective values, and timing information.

## 22.4 Geometric Programming Example

Run:

```
python examples/GPExample.py
```

Expected behavior:

1. Builds an `OptimizationProblem` from semigroup-algebra expressions.
2. Constructs a `GPRelaxations` model with transformation matrix $H$.
3. Prints transformation matrix information and GP solution details.

## 22.5 SONC Examples

Run:

```
python examples/SONCExample.py
python examples/SONCExample33.py
```

Expected behavior:

1. Builds constrained SONC models from semigroup-algebra expressions.
2. Prints a certified lower bound when the solver succeeds.
3. Reports runtime and solver status if GP solving is unavailable in the current environment.

## 22.6 Example 3.3 Traceability

The script `examples/SONCExample33.py` is aligned with the Section 3.3 benchmark used in the repository and is paired with checks in `tests/test_sonc_section3.py`.

## 22.7 Benchmark Gallery System

Irene includes a gallery-based benchmark runner for systematic comparison across relaxation families:

- `benchmarks/gallery.yaml` — YAML configuration defining problem sets, parameters, and solver options.
- `benchmarks/run_gallery.py` — Entry point that loads the gallery config and runs all configured benchmarks.

Run:

```
python benchmarks/run_gallery.py
```

This exercises SDP, GP, SONC, and SOSONC relaxations on a curated set of problems and writes structured results to `benchmarks/results/`.

## 22.8 Backend Comparison Benchmark

The cross-backend benchmark (SymEngine vs SymPy) has been archived to `benchmarks/archive/` since the rewrite branch was merged into master. The gallery runner remains the primary benchmark entry point:

```
python benchmarks/run_gallery.py
```

## 22.9 Additional Examples

The following scripts exercise mean polynomial forms, separating polynomials, and other specialized relaxation techniques:

- `examples/pqforms.py` — Power-mean form certificates for nonnegativity.
- `examples/SOSONCSchickSeparating.py` — Schick's SOS+SONC separating example.
- `examples/Rosenbrock.py`, `examples/Giunta.py`, `examples/Parsopoulos.py` — Classic benchmark problems.
- `examples/McCormick.py` — McCormick relaxation on non-convex objective.

## 22.10 Regression Validation

Run the test suite from the repository root with the virtual environment activated:

```
source .venv/bin/activate
python -m pytest Irene/tests/ tests/ -q
```

This is the recommended consistency check after modifying optimization modules or documentation examples. Individual test files can be run separately, e.g.:

```
python -m pytest tests/test_sosonc.py -v
```

CHAPTER

# TWENTYTHREE

# PYPROXIMATION (LEAN)

This is a brief documentation for using *pyProximation*. pyProximation provides tools for function approximation in Hilbert spaces. It uses orthogonal systems of functions built via the Gram-Schmidt procedure over measures on arbitrary domains.

## 23.1 Requirements and dependencies

Since this is a python package, so clearly one needs have python available. pyProximation relies on the following packages:

- **for numerical calculation:**
  - numpy,
  - scipy ,
- **for symbolic computation:**
  - sympy

## 23.2 Download

*pyProximation* can be obtained from https://github.com/mghasemi/pyProximation.

## 23.3 Installation

To install *pyProximation*, run the following in terminal:

```
sudo python setup.py install
```

### 23.3.1 Documentation

The documentation of *pyProximation* is prepaqred via sphinx.

## 23.4 License

*pyProximation* is distributed under MIT license:

### 23.4.1 MIT License

Copyright (c) 2016 Mehdi Ghasemi

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

CHAPTER

# TWENTYFOUR

# MEASURES

## 24.1 Density function

**The *pyProximation* implements two different scenarios for measure spaces:**

1. Continuous case, where support of the measure is given as a compact subspace (box) of $\mathbb{R}^n$, and
2. Discrete case, where a finite set of points and their weights are given.

### 24.1.1 Continuous measure spaces

For the continuous case, generally assume that the support of the measure is a product of closed interval, i.e., $\prod_{i=1}^{n}[a_i, b_i]$, where for each $i$, $a_i < b_i$. Such a set can be defined as a list of ordered pairs of real numbers as `[(a1, b1), (a2, b2), ..., (an, bn)]`. Moreover, when we speak about a subset of the support, we always refer to a box, defined as a list of 2-tuples.

In this case, the measure is given implicitly as a density function $w(x)$. So the measure of a set $S$ is given by

$$\mu(S) = \int_S w(x)dx.$$

For example, the following code defines the Lebesgue measure on $[-1, 1] \times [-1, 1]$ and finds the measure of the set $[0, 1] \times [-1, 0]$:

```
# import the Measure class
from pyProximation import Measure
# define the support of the measure
D = [(-1, 1), (-1, 1)]
# define the measure with the constant density 1
M = Measure(D, 1)
# define a set called S
S = [(0, 1), (-1, 0)]
# find the measure of the set S
print(M.measure(S))
```

### 24.1.2 Discrete measure spaces

In this case, the measure is basically a convex combination of Dirac measure. Given a set $X = \{x_1, \ldots, x_n\}$ and corresponding non-negative weights $w_1, \ldots, w_n$, one defines a measure as $\mu = \sum_{i=1}^{n} w_i \delta_{x_i}$. Then the measure of a subset $S = \{x_{i_1}, \ldots, x_{i_k}\}$ of $X$ is given by

$$\mu(S) = \int_S d\mu = \sum_{j=1}^{k} w_{i_j}$$

The following is a sample code for discrete case:

```
# import the Measure class
from pyProximation import Measure
# define the support and density
D = {'x1':1, 'x2':.5, 'x3':1.1, 'x4':.6}
# define the measure
M = Measure(D)
# define a set called S
S = ['x2', 'x3']
# find the measure of the set S
print(M.measure(S))
```

## 24.2 Integrals

Suppose that a measure space $(X, \mu)$ and a measurable function $f$ on $X$ are given. The method `integral` computes $\int_X f d\mu$. If $\mu$ is discrete, then $f$ can be a dictionary with keys as points of domain and values as evaluation at each point. Otherwise, $f$ is simply a numerical function:

```
from pyProximation import Measure
from numpy import sqrt
# define the density function
w = lambda x:1./sqrt(1.-x**2)
# define the support
D = [(-1, 1)]
# initiate the measure space
M = Measure(D, w)
# set f(x) = x^2
f = lambda x: x**2
# integrate f(x) w.r.t. w(x)
print(M.integral(f))
```

Or in two dimensions:

```
from pyProximation import Measure
from numpy import sqrt
# define the density function
w = lambda x, y:y**2/sqrt(1.-x**2)
# define the support
D = [(-1, 1), (-1, 1)]
# initiate the measure space
M = Measure(D, w)
# set f(x, y) = x^2 + y
f = lambda x, y: x**2 + y
# integrate f(x, y) w.r.t. w(x, y)
print(M.integral(f))
```

## 24.3 *p*-norms

Given a measure space $(X, \mu)$ and a measurable function $f$, the $p$-norm of $f$, for a positive $p$ is defined as:

$$\|f\|_p = \left( \int_X |f|^p d\mu \right)^{1/p}.$$

The method `norm(p, f)` calculates the above quantity.

## 24.4 Drawing samples

Suppose that $(X, \mu)$ is a measure space and one wishes to draw a sample of size $n$ from $X$ according to the distribution $\mu$. This can be done by the method `sample(n)` which returns a list of $n$ random points from the support, according to $\mu$.

CHAPTER

# TWENTYFIVE

# HILBERT SPACES

## 25.1 Orthonormal system of functions

Let $X$ be a topological space and $\mu$ be a finite Borel measure on $X$. The bilinear function $\langle \cdot, \cdot \rangle$ defined on $L^2(X, \mu)$ as $\langle f, g \rangle = \int_X fg d\mu$ is an inner product which turns $L^2(X, \mu)$ into a Hilbert space.

Let us denote the family of all continuous real valued functions on a non-empty compact space $X$ by $\mathrm{C}(X)$. Suppose that among elements of $\mathrm{C}(X)$, a subfamily $A$ of functions are of particular interest. Suppose that $A$ is a subalgebra of $\mathrm{C}(X)$ containing constants. We say that an element $f \in \mathrm{C}(X)$ can be approximated by elements of $A$, if for every $\epsilon > 0$, there exists $p \in A$ such that $|f(x) - p(x)| < \epsilon$ for every $x \in X$. The following classical results guarantees when every $f \in \mathrm{C}(X)$ can be approximated by elements of $A$.

**Note**

**Stone-Weierstrass:**

Every element of $\mathrm{C}(X)$ can be approximated by elements of $A$ if and only if for every $x \neq y \in X$, there exists $p \in A$ such that $p(x) \neq p(y)$.

Despite the strong and important implications of the Stone-Weierstrass theorem, it leaves every computational details out and does not give an specific algorithm to produce an estimator for $f$ with elements of $A$, given an error tolerance $\epsilon$, and the search for a such begins.

Define $\|f\|_\infty$ (the sup norm of $f$) of a given function $f \in \mathrm{C}(X)$ by

$$\|f\|_\infty = \sup_{x \in X} |f(x)|,$$

Then the above argument can be read as: For every $f \in \mathrm{C}(X)$ and every $\epsilon > 0$, there exists $p \in A$ such that $\|f - p\|_\infty < \epsilon$.

Let $(V, \langle \cdot, \cdot \rangle)$ be an inner product space with $\|v\|_2 = \langle v, v \rangle^{\frac{1}{2}}$. A basis $\{v_\alpha\}_{\alpha \in I}$ is called an orthonormal basis for $V$ if $\langle v_\alpha, v_\beta \rangle = \delta_{\alpha\beta}$, where $\delta_{\alpha\beta} = 1$ if and only if $\alpha = \beta$ and is equal to $0$ otherwise. Every given set of linearly independent vectors can be turned into a set of orthonormal vectors that spans the same sub vector space as the original. The following well-known result gives an algorithm for producing such orthonormal from a set of linearly independent vectors:

**Note**

**Gram–Schmidt**

Let $(V, \langle\cdot,\cdot\rangle)$ be an inner product space. Suppose $\{v_i\}_{i=1}^n$ is a set of linearly independent vectors in $V$. Let

$$u_1 := \frac{v_1}{\|v_1\|_2}$$

and (inductively) let

$$w_k := v_k - \sum_{i=1}^{k-1} \langle v_k, u_i\rangle u_i \text{ and } u_k := \frac{w_k}{\|w_k\|_2}.$$

Then $\{u_i\}_{i=1}^n$ is an orthonormal collection, and for each $k$,

$$span\{u_1, u_2, \cdots, u_k\} = span\{v_1, v_2, \cdots, v_k\}.$$

Note that in the above note, we can even assume that $n = \infty$.

Let $B = \{v_1, v_2, \dots\}$ be an ordered basis for $(V, \langle\cdot,\cdot\rangle)$. For any given vector $w \in V$ and any initial segment of $B$, say $B_n = \{v_1, \dots, v_n\}$, there exists a unique $v \in \text{span}(B_n)$ such that $\|w - v\|_2$ is the minimum:

**Note**

Let $w \in V$ and $B$ a finite orthonormal set of vectors (not necessarily a basis). Then for $v = \sum_{u\in B}\langle u, w\rangle u$

$$\|w - v\|_2 = \min_{z\in\text{span}(B)} \|w - z\|_2.$$

Now, let $\mu$ be a finite measure on $X$ and for $f, g \in \mathrm{C}(X)$ define $\langle f, g\rangle = \int_X fgd\mu$. This defines an inner product on the space of functions. The norm induced by the inner product is denoted by $\|\cdot\|_2$. It is evident that

$$\|f\|_2 \leq \|f\|_\infty \mu(X),\ \forall f \in \mathrm{C}(X),$$

which implies that any good approximation in $\|\cdot\|_\infty$ gives a good $\|\cdot\|_2$-approximation. But generally, our interest is the other way around. Employing Gram-Schmidt procedure, we can find $\|\cdot\|_2$ within any desired accuracy, but this does not guarantee a good $\|\cdot\|_\infty$-approximation. The situation is favorable in finite dimensional case. Take $B = \{p_1, \dots, p_n\} \subset \mathrm{C}(X)$ and $f \in \mathrm{C}(X)$, then there exists $K_f > 0$ such that for every $g \in \text{span}(B \cup \{f\})$,

$$K_f\|g\|_\infty \leq \|g\|_{2\leq}\|g\|_\infty\mu(X).$$

Since $X$ is assumed to be compact, $\mathrm{C}(X)$ is separable, i.e., $\mathrm{C}(X)$ admits a countable dimensional dense subvector space (e.g. polynomials for when $X$ is a closed, bounded interval). Thus for every $f \in \mathrm{C}(X)$ and every $\epsilon > 0$ one can find a big enough finite $B$, such that the above inequality holds. In other words, good enough $\|\cdot\|_2$-approximations of $f$ give good $\|\cdot\|_\infty$-approximations, as desired.

## 25.2 OrthSystem

Given a measure space, the `OrthSystem` class implements the described procedure, symbolically. Therefore, it relies on a symbolic environment. Currently, the supported symbolic environment is:

1. *sympy*

### 25.2.1 Legendre polynomials

Let $d\mu(x) = dx$, the regular Lebesgue measure on $[-1, 1]$ and $B = \{1, x, x^2, \ldots, x^n\}$. The orthonormal polynomials constructed from $B$ are called *Legendre* polynomials. The $n^{th}$ Legendre polynomial is denoted by $P_n(x)$.

The following code generates Legendre polynomials up to a given order:

```
# the symbolic package
from sympy import *
from pyProximation import Measure, OrthSystem
# the symbolic variable
x = Symbol('x')
# set a limit to the order
n = 6
# define the measure
D = [(-1, 1)]
M = Measure(D, 1)
S = OrthSystem([x], D, 'sympy')
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^n}
B = S.PolyBasis(n)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# print the result
print(S.OrthBase)
```

### 25.2.2 Chebyshev polynomials

Let $d\mu(x) = \frac{dx}{\sqrt{1-x^2}}$ on $[-1, 1]$ and $B$ as in Legendre polynomials. The orthonormal polynomials associated to this setting are called *Chebyshev* polynomials and the $n^{th}$ one is denoted by $T_n(x)$.

The following code generates Chebyshev polynomials up to a given order:

```
# the symbolic package
from sympy import *
from numpy import sqrt
from pyProximation import Measure, OrthSystem
# the symbolic variable
x = Symbol('x')
# set a limit to the order
n = 6
# define the measure
D = [(-1, 1)]
w = lambda x: 1./sqrt(1. - x**2)
M = Measure(D, w)
S = OrthSystem([x], D, 'sympy')
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^n}
B = S.PolyBasis(n)
```

(continues on next page)

(continued from previous page)

```
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# print the result
print(S.OrthBase)
```

## 25.3 Approximation

Let $(X, \mu)$ be a compact Borel measure space and $\mathcal{O} = \{u_1, u_2, \dots\}$ an orthonormal basis of function whose span is dense in $L^2(X, \mu)$. Given a function $f \in L^2(X, \mu)$, then $f$ can be approximated as

$$f = \lim_{n\to\infty} \sum_{i=1}^{n} \langle f, u_i \rangle u_i$$

`OrthSystem.Series` calculates the coefficients $\langle f, u_i \rangle$:

### 25.3.1 Truncated Fourier series

Let $d\mu(x) = dx$, the regular Lebesgue measure on $[c, c + 2l]$ and $B = \{1, \sin(\pi x), \cos(\pi x), \sin(2\pi x), \cos(2\pi x), \dots, \sin(n\pi x), \cos(n\pi x), \}$. The following code calculates the Fourier series approximation of $f(x) = \sin(x)e^x$:

```
from sympy import *
from numpy import sqrt
from pyProximation import Measure, OrthSystem
# the symbolic variable
x = Symbol('x')
# set a limit to the order
n = 4
# define the measure
D = [(-1, 1)]
w = lambda x: 1./sqrt(1. - x**2)
M = Measure(D, w)
S = OrthSystem([x], D, 'sympy')
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^n}
B = S.FourierBasis(n)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# number of elements in the basis
m = len(S.OrthBase)
# set f(x) = sin(x)e^x
f = sin(x)*exp(x)
# extract the coefficients
Coeffs = S.Series(f)
# form the approximation
f_app = sum([S.OrthBase[i]*Coeffs[i] for i in range(m)])
print(f_app)
```

### 25.3.2 Fitting Data

Suppose that a set of data points $(x_1, y_1), \ldots, (x_m, y_m)$ are given, where $x_i \in \mathbb{R}^n$. To fit a curve or surface to data with minimum average square error, one can employ `OrthSystem` with an arbitrary system of functions to do so. The following example generates random data and fits two surfaces using polynomials and trigonometric functions:

```
from random import uniform
from sympy import Function, Symbol
from sympy.utilities.lambdify import implemented_function, lambdify
# import the module
from pyProximation import Measure, OrthSystem
x = Symbol('x')
y = Symbol('y')
# define dirac function symbolically and numerically
dirac = implemented_function(
    Function('dirac'), lambda a, b: 1 * (abs(a - b) < .000001))
# list of random points
points = [(uniform(0, 1), uniform(0, 1)) for _ in range(50)]
# container for the support of discrete measure
D = {}
# a symbolic function to represent the random points
g = 0
# Random values of the function
vals = []
for p in points:
    t = uniform(-.8, 1)
    vals.append(t)
    g += t * dirac(x, p[0]) * dirac(y, p[1])
    D[p] = 1
# The discrete measure supported at random points
M = Measure(D)
# Orthonormal systems of functions
S1 = OrthSystem([x, y], [(0, 1), (0, 1)])
S1.SetMeasure(M)

S2 = OrthSystem([x, y], [(0, 1), (0, 1)])
S2.SetMeasure(M)
# polynomial basis
B1 = S1.PolyBasis(4)
# trigonometric basis
B2 = S2.FourierBasis(3)
# link the basis to the orthogonal systems
S1.Basis(B1)
S2.Basis(B2)
# form the orthonormal basis
S1.FormBasis()
S2.FormBasis()
# calculate coefficients
cfs1 = S1.Series(g)
cfs2 = S2.Series(g)
# orthogonal approximation
aprx1 = sum([S1.OrthBase[i] * cfs1[i] for i in range(len(S1.OrthBase))])
aprx2 = sum([S2.OrthBase[i] * cfs2[i] for i in range(len(S2.OrthBase))])
```

(continues on next page)

(continued from previous page)

```
# inspect the symbolic approximations
print(aprx1)
print(aprx2)
# compare the first sample numerically
f1 = lambdify((x, y), aprx1, 'numpy')
f2 = lambdify((x, y), aprx2, 'numpy')
print(vals[0], f1(*points[0]), f2(*points[0]))
```

## 25.4 Rational Approximation

Let $f : X \to \mathbb{R}$ be a continuous function and $\{b_0, b_1, \dots\}$ an orthonormal system of functions over $X$ with respect to a measure $\mu$. Suppose that we want to approximate $f$ with a function of the form $\frac{p(x)}{q(x)}$ where $p = \sum_{j=0}^{m} \alpha_j b_j$ and $q = 1 + \sum_{j=1}^{n} \beta_j b_j$. Let $r(x) = p(x) - q(x)f(x)$ be the residual. We can find the coefficients $\alpha_i, \beta_j$ such that $\|r\|_2$ be minimum.

Let $L(\alpha, \beta) = r(x) \cdot r(x)$. Then the solution satisfies the following equations:

$$\frac{\partial}{\partial \alpha_i} L = 0, \quad \frac{\partial}{\partial \beta_i} L = 0,$$

which is a system of linear equations. This is implemented in `rational.RationalApprox.RatLSQ`:

```
from sympy import Symbol, cos, exp, pi, sin, sinh
from pyProximation import OrthSystem, RationalApprox

x = Symbol('x')
f = sinh(x) * cos(3 * x) + sin(3 * x) * exp(x)
D = [(-pi, pi)]
S = OrthSystem([x], D)
B = S.PolyBasis(5)

# link B to S
S.Basis(B)

# generate the orthonormal basis
S.FormBasis()

# extract the coefficients of approximations
Coeffs = S.Series(f)

# form the approximation
h = sum([S.OrthBase[i] * Coeffs[i] for i in range(len(S.OrthBase))])
T = RationalApprox(S)
g = T.RatLSQ(5, 5, f)

print(h)
print(g)
```

CHAPTER

# TWENTYSIX

# INTERPOLATION

## 26.1 Lagrange interpolation

Suppose that a list of $m+1$ points $\{(x_0, y_0), \ldots, (x_m, y_m)\}$ in $\mathbb{R}^2$ is given, such that $x_i \neq x_j$, if $i \neq j$. Then

$$p(\mathbf{x}) = \sum_{i=0}^{m} y_i \ell_i(\mathbf{x}),$$

is a polynomial of degree $m$ which passes through all the given points. Here

$$\ell_i(\mathbf{x}) = \prod_{j \neq i} \frac{\mathbf{x} - x_j}{x_i - x_j},$$

are *Lagrange Basis Polynomials*.

This procedure can be extended to multivariate case as well.

Suppose that a list of points $\{x_1, \ldots, x_\rho\}$ in $\mathbb{R}^n$ and a list of corresponding values $\{y_1, \ldots, y_\rho\}$ are given. Let's denote by $\mathbf{X}$ the tuple $(\mathbf{X}_1, \ldots, \mathbf{X}_n)$ of variables and $\mathbf{X}_1^{e_1} \cdots \mathbf{X}_n^{e_n}$ by $\mathbf{X}^{\mathbf{e}}$, where $\mathbf{e} = (e_1, \ldots, e_n)$.

If for some $m > 0$, we have $\rho = \binom{m+n}{m}$, then the number of given points matches the number of monomials of degree at most $m$ in the polynomial basis. Denote the exponents of these monomials by $\mathbf{e}_i$, $i = 1, \ldots, \rho$ and let

$$D = \begin{pmatrix} x_1^{\mathbf{e}_1} \ldots & x_1^{\mathbf{e}_\rho} \\ \vdots & \vdots \\ x_\rho^{\mathbf{e}_1} \ldots & x_\rho^{\mathbf{e}_\rho} \end{pmatrix}$$

and for $1 \leq j \leq \rho$:

$$D_j = \begin{pmatrix} x_1^{\mathbf{e}_1} \ldots & x_1^{\mathbf{e}_\rho} \\ \vdots & \vdots \\ x_{j-1}^{\mathbf{e}_1} \ldots & x_{j-1}^{\mathbf{e}_\rho} \\ \mathbf{X}^{\mathbf{e}_1} \ldots & \mathbf{X}^{\mathbf{e}_\rho} \\ x_{j+1}^{\mathbf{e}_1} \ldots & x_{j+1}^{\mathbf{e}_\rho} \\ \vdots & \vdots \\ x_\rho^{\mathbf{e}_1} \ldots & x_\rho^{\mathbf{e}_\rho} \end{pmatrix}.$$

Then the polynomial

$$p(\mathbf{X}) = \sum_{i=1}^{\rho} y_i \frac{|D_i|}{|D|},$$

interpolates the given list of points and their corresponding values. Here $|M|$ denotes the determinant of $M$.

The above procedure is implemented in the `Interpolation` module. The following code provides an example in 2 dimensional case:

```
from sympy import *
from pyProximation import Interpolation
# define symbolic variables
```

(continues on next page)

(continued from previous page)

```
x = Symbol('x')
y = Symbol('y')
# initiate the interpolation instance
Inter = Interpolation([x, y], 'sympy')
# list of points
points = [(-1, 1), (-1, 0), (0, 0), (0, 1), (1, 0), (1, -1)]
# corresponding values
values = [-1, 2, 0, 2, 1, 0]
# interpolate
p = Inter.Interpolate(points, values)
# print the result
print(p)
```

## 26.2 Lagrange interpolation as an $L^2$-orthogonal projection

The `Interpolation` and `OrthSystem` modules are two views of the same operator. Let $X = \{x_0, \dots, x_m\}$ be $m+1$ distinct nodes and let

$$\mu = \sum_{k=0}^{m} w_k \, \delta_{x_k}, \qquad w_k > 0,$$

be a discrete measure (the example below uses $w_k = 1$). On the polynomials, define the associated inner product

$$(f, g)_\mu = \int f \, g \, d\mu = \sum_{k=0}^{m} w_k \, f(x_k) \, g(x_k),$$

and let $V = \Pi_m$ be the space of polynomials of degree at most $m$, so that $\dim V = m+1$ equals the number of atoms of $\mu$.

### 26.2.1 Lagrange interpolation is the $L^2(\mu)$-orthogonal projection

The $L^2(\mu)$-orthogonal projection $P_V f$ of $f$ onto $V$ is the unique $p \in V$ satisfying $p(x_k) = f(x_k)$ for $k = 0, \dots, m$ — that is, the Lagrange interpolant.

*Proof sketch.* The form $(\cdot, \cdot)_\mu$ is a genuine inner product on $V$: if $v \in V$ and $(v, v)_\mu = 0$, then $v(x_k) = 0$ for every $k$, so $v$ has $m+1$ roots and hence $v = 0$. Thus $P_V$ is well-defined. By the projection characterization, $p = P_V f$ iff $(f - p, v)_\mu = 0$ for all $v \in V$, i.e.

$$\sum_{k=0}^{m} w_k \big(f(x_k) - p(x_k)\big) \, v(x_k) = 0 \qquad \forall \, v \in V.$$

Because the evaluation functionals $v \mapsto v(x_k)$ span $V^*$ (the Lagrange basis $\{\ell_k\}_{k=0}^m$ is the dual basis to the point evaluations), this orthogonality is equivalent to $f(x_k) - p(x_k) = 0$ for each $k$, i.e. $p(x_k) = f(x_k)$. Uniqueness follows from $\dim V = m+1$.

Equivalently, in the orthonormal basis $\{u_0, \dots, u_m\}$ obtained from $\{1, x, \dots, x^m\}$ by Gram–Schmidt with respect to $(\cdot, \cdot)_\mu$, the interpolant is the truncated Hilbert series

$$p = \sum_{i=0}^{m} \langle f, u_i \rangle_\mu \, u_i = \sum_{i=0}^{m} \Big( \sum_{k=0}^{m} w_k \, f(x_k) \, u_i(x_k) \Big) \, u_i,$$

which is exactly what `OrthSystem.FormBasis` / `OrthSystem.Series` compute. The following example builds the same interpolant two ways and shows they coincide:

```
# symbolic variable
x = Symbol('x')
# function to be approximated
g = sin(x)*exp(sin(x)*x)#x*sin(x)
# its numerical equivalent
g_ = lambdify(x, g, 'numpy')
# number of approximation terms
n = 6
# half interval length
l = 3.1
# interpolation points and values
Xs = [[-3], [-2], [-1], [0], [1], [2], [3]]
Ys = [g_(Xs[i][0]) for i in range(7)]
# a discrete measure
supp = {-3:1, -2:1, -1:1, 0:1, 1:1, 2:1, 3:1}
M = Measure(supp)
# orthogonal system
S = OrthSystem([x], [(-l, l)])
# link the measure
S.SetMeasure(M)
# polynomial basis
B = S.PolyBasis(n)
# link the basis to the orthogonal system
S.Basis(B)
# form the orthonormal basis
S.FormBasis()
# calculate coefficients
cfs = S.Series(g)
# orthogonal approximation
aprx = sum([S.OrthBase[i]*cfs[i] for i in range(len(B))])
# interpolate
Intrp = Interpolation([x])
intr = Intrp.Interpolate(Xs, Ys)
print(intr)
```

The two constructions agree to machine precision: the `OrthSystem` truncated series and the `Interpolation` Lagrange form both return the same degree-6 polynomial passing through the seven nodes (verified: max node difference $\leq 10^{-15}$).

### 26.2.2 Interpolation and least-squares are one projection in two regimes

The same operator $P_V f$ changes character with the relation between $\dim V$ and the number $m+1$ of atoms of $\mu$. The form $(\cdot,\cdot)_\mu$ is a genuine (non-degenerate) inner product on $V = \Pi_n$ exactly when $n+1 \leq m+1$, i.e. $\dim V \leq$ the number of atoms.

| $\dim V$ vs atoms | What $P_V f$ is | Characterizing property |
|---|---|---|
| $\dim V < m+1$ (underdetermined) | weighted least-squares fit | minimizes $\sum w_k(f-p)^2$; normal equations; residual $\neq 0$ |
| $\dim V = m+1$ (exact fit) | Lagrange interpolant | interpolates exactly; residual $= 0$ |
| $\dim V > m+1$ (overdetermined) | degenerate (no unique projection) | $m+1$ interpolants; the minimum-norm (Moore–Penrose) interpolant is canonical |

In the underdetermined regime the projection is the weighted least-squares fit: it is the unique $p \in V$ minimizing $\sum_k w_k \big(f(x_k) - p(x_k)\big)^2$, equivalently the solution of the normal equations $(E^o pW E)\, c = E^o pW y$ with $E$ the Vandermonde matrix. The transition is sharp: the monomial Gram matrix $E^o pW E$ is nonsingular for $\dim V \le m+1$ and singular for $\dim V > m+1$.

### 26.2.3 Taylor / Maclaurin series lie outside the ordinary $L^2$ framework

Unlike interpolation and least-squares, the Taylor expansion is **not** an $L^2(\mu)$-orthogonal projection for any ordinary positive measure $\mu$ on an interval. Its coefficient functionals are the point derivatives $f \mapsto f^{(k)}(a)$, which are *unbounded* on $L^2$: the sequence $f_n(x) = n\, e^{-n^2(x-a)^2}(x-a)^2$ satisfies $\|f_n\|_{L^2} o 0$ while $f_n''(a) o \infty$, so no $L^2$-inner product can represent them. Taylor series are recovered only after extending "measure" to *distributions*: with $\langle f, \delta_a^{(k)} \rangle = (-1)^k f^{(k)}(a)$, the degree-$n$ Taylor polynomial is the truncated expansion

$$T_n(f)(x) = \sum_{k=0}^{n} \langle f,\, (-1)^k \delta_a^{(k)}/k! \rangle\, (x-a)^k,$$

which requires $f$ to be $C^k$ (a distributional pairing, not an $L^2$ inner product). This is why `OrthSystem` naturally produces interpolants and least-squares fits, while Taylor expansions are built separately in `approx.rst`.

### 26.2.4 Verification

The identities above are checked numerically in `verify_unified_approx.py` (Lagrange $= L^2$ projection, 1D and 2D; Taylor $\neq L^2$ projection; distributional Taylor) and `verify_least_squares.py` (least-squares $= L^2$ projection in the underdetermined regime, and the $\dim V$-vs-atoms boundary). Both run in the `Irene` virtual environment.

CHAPTER

# TWENTYSEVEN

# CODE DOCUMENTATION

`class pyProximation.base.Foundation`

This class contains common features of all modules.

`DetSymEnv()`

Returns `['sympy']` when sympy is available.

`class pyProximation.measure.Measure(`*dom*, *w=None*`)`

**An instance of this class is a measure on a given set *supp*. The support is either**

- a python variable of type *set*, or
- a list of tuples which represents a box in euclidean space.

**Initializes a measure object according to the inputs:**

- ***dom* must be either**
  - a list of 2-tuples
  - a non-empty dictionary
- ***w* must be a**
  - a function if *dom* defines a region
  - left blank (None) if *dom* is a dictionary

`boxCheck(`*B*`)`

Checks the structure of the box *B*. Returns *True* id *B* is a list of 2-tuples, otherwise it returns *False*.

`check(`*dom*, *w*`)`

Checks the input types and their consistency, according to the ___*init*___ arguments.

`integral(`*f*`)`

Returns the integral of $f$ with respect to the currwnt measure over the support.

`measure(`*S*`)`

Returns the measure of the set $S$. $S$ must be a list of 2-tuples.

`norm(`*p*, *f*`)`

Computes the norm-$p$ of the $f$ with respect to the current measure.

`sample(`*num*`)`

Samples from the support according to the measure.

class pyProximation.orthsys.**OrthSystem**(*variables*, *var_range*, *env='sympy'*)

`OrthogonalSystem` class produces an orthogonal system of functions according to a suggested basis of functions and a given measure supported on a given region.

This basically performs a 'Gram-Schmidt' method to extract the orthogonal basis. The inner product is obtained by integration of the product of functions with respect to the given measure (more accurately, the distribution).

To initiate an instance of this class one should provide a list of symbolic variables *variables* and the range of each variable as a list of lists `var_range`.

To initiate an orthogonal system of functions, one should provide a list of symbolic variables `variables` and the range of each these variables as a list of lists `var_range`.

**Basis**(*base_set*)

To specify a particular family of function as a basis, one should call this method with a list `base_set` of linearly independent functions.

**FormBasis**()

Call this method to generate the orthogonal basis corresponding to the given basis via `Basis` method. The result will be stored in a property called `OrthBase` which is a list of function that are orthogonal to each other with respect to the measure `measure` over the given range `Domain`.

**FourierBasis**(*n*)

Generates a Fourier basis from variables consisting of all *sin* & *cos* functions with coefficients at most *n*.

**PolyBasis**(*n*)

Generates a polynomial basis from variables consisting of all monomials of degree at most `n`.

**Series**(*f*)

Given a function *f*, this method finds and returns the coefficients of the series that approximates *f* as a linear combination of the elements of the orthogonal basis.

**SetMeasure**(*M*)

To set the measure which the orthogonal system will be computed, simply call this method with the corresponding distribution as its parameter *dm*; i.e, the parameter is *d(m)* where *m* is the original measure.

**SetOrthBase**(*base*)

Sets the orthonormal basis to be the given *base*.

**TensorPrd**(*Bs*)

Takses a list of symbolic bases, each one a list of symbolic expressions and returns the tensor product of them as a list.

**inner**(*f*, *g*)

Computes the inner product of the two parameters with respect to the measure `measure`.

**project**(*f*, *g*)

Finds the projection of `f` on `g` with respect to the inner product induced by the measure `measure`.

pyProximation.rational.**RationalApprox**

alias of *RationalAprox*

class pyProximation.rational.**RationalAprox**(*orth*)

`RationalAprox` calculates a rational approximation for a given function. `RationalApprox` is the preferred public alias; this class name is retained for backward compatibility. It takes one argument *orth* which is an instance of `OrthSystem` and does all the computations in the scope of this object.

`RatLSQ`(*m*, *n*, *f*)

Calculates a rational function :math:`

**rac{p}{q}` where**
*p, q* consists of the first *m, n* elements of the orthonormal basis :math:`||f-

rac{p}{q}||_2` is minimized.

`class pyProximation.interpolation.Interpolation`(*var*, *env='sympy'*)

The `Interpolation` class provides polynomial interpolation routines in multi variate case.

*var* is the list of symbolic variables and *env* is the the symbolic tool.

`Delta`(*idx=-1*)

Construct the matrix corresponding to *idx*'th point, if *idx>0* Otherwise returns the discriminant.

`Interpolate`(*points*, *vals*)

Takes a list of points *points* and corresponding list of values *vals* and return the interpolant.

Since in multivariate case, there is a constraint on the number of points, it checks for the valididty of the input. In case of failure, describes the type of error occured according to the inputs.

`MinNumPoints`()

Returns the minimum number of points still required.

`Monomials`()

Generates the minimal set of monomials for interpolation.

CHAPTER

# TWENTYEIGHT

# CODE DOCUMENTATION

This is the base module for all other objects of the package.

- *LaTeX* returns a LaTeX string out of an *Irene* object.
- *base* is the parent of all *Irene* objects.

`Irene.base.LaTeX(`*obj*`)`

Returns LaTeX representation of Irene's objects.

`class Irene.base.base`

All the modules in *Irene* extend this class which perform some common tasks such as checking existence of certain software.

`AvailableSDPSolvers()`

find the existing sdp solvers.

`static which(`*program*`)`

Check the availability of the *program* system-wide. Returns the path of the program if exists and returns 'None' otherwise.

This module is responsible for conversion of a given symbolic optimization problem into semidefinite optimization problems. The main classes included in this module are:

- *SDPRelaxations*
- *SDRelaxSol*
- *Mom*

`Irene.relaxations.Calpha_(`*expn*, *Mmnt*`)`

Given an exponent *expn*, this function finds the corresponding $C_{expn}$ matrix which can be used for parallel processing.

`Irene.relaxations.Calpha__(`*expn*, *Mmnt*, *ii*, *q*`)`

Given an exponent *expn*, this function finds the corresponding $C_{expn}$ matrix which can be used for parallel processing.

`class Irene.relaxations.Mom(`*expr*`)`

This is a simple interface to define moment constraints to be used via *SDPRelaxations.MomentConstraint*. It takes a sympy expression as input and initiates an object which can be used to force particular constraints on the moment sequence.

**Example:** Force the moment of $x^2f(x) + f(x)^2$ to be at least $.5$:

```
Mom(x**2 * f + f**2) >= .5
# OR
Mom(x**2 * f) + Mom(f**2) >= .5
```

class Irene.relaxations.RelaxationConfig(*reduction_method: str = 'none', monomial_pruning: bool = False, sparsity_detection: bool = False, sparsity_block_sdp: bool = False, border_basis_degree: int = 2, verbose_reduction: bool = False, quotient_basis: str = 'groebner'*)

Configuration for the SDP relaxation monomial-reduction pipeline.

The default reduction strategy is `'newton_polytope'` based on Phase 3 benchmark results (bench_phase3_reductions.py, 2026-08-08): Newton pruning achieves 64–67 % basis reduction on sparse gallery problems with zero overhead on dense ones. Sparsity detection only helps 2/12 problems, and border basis shows no conditioning advantage at low degrees.

`reduction_method`

Which reduction strategy to apply before building the moment matrix. One of `'none'`, `'newton_polytope'`, `'border_basis'`, or `'sparsity'` (correlative sparsity decomposition). Default `'none'`.

**Type**
str

`monomial_pruning`

Enable Newton-polytope pruning of the monomial basis via the newton_polytope module (Minkowski hull of the objective + constraint supports). Until a complete SOS-support certificate exists, this path retains the full degree-bounded support (hull membership alone does not certify that an exponent is removable from the moment/Gram basis – see `_pruned_basis_from_module`), so enabling it currently changes no SDP; the option is kept so a certified reduction can be activated later. Default `False`.

**Type**
bool

`sparsity_detection`

Run correlative-sparsity analysis on the problem polynomials and decompose the moment matrix into independent blocks when possible.

**Type**
bool

`sparsity_block_sdp`

When True and sparsity detection finds disconnected variable cliques, build and solve one SDP per clique instead of a single monolithic SDP. This can give exponential savings for problems whose variables split into truly independent sub-problems (e.g. block-diagonal objectives). Default `False`.

**Type**
bool

`border_basis_degree`

Degree bound for the border-basis quotient algebra representation (only used when `reduction_method='border_basis'`). Default 2.

**Type**
int

`verbose_reduction`

Print diagnostics (basis sizes, reduction ratios) during setup.

**Type**
bool

`class Irene.relaxations.SDPRelaxations(`*gens*, *relations=()*, *name='SDPRlx'*, *config=None*`)`

This class defines a function space by taking a family of sympy symbolic functions and relations among them. Simply, it initiates a commutative free real algebra on the symbolic functions and defines the function space as the quotient of the free algebra by the ideal generated by the given relations. It takes three arguments:

- *gens* which is a list of `sympy` symbols and function symbols,
- *relations* which is a set of `sympy` expressions in terms of *gens* that defines an ideal.
- *name* is a given name which is used to save the state of the instant at break.

`AddConstraint(`*cnst*`)`

Takes an (in)equality as an algebraic combination of the generating functions that defines the feasibility region. It reduces the defining (in)equalities according to the given relations.

`Calpha(`*expn*, *Mmnt*`)`

Given an exponent *expn*, this method finds the corresponding $C_{expn}$ matrix.

P5.7: When `Mmnt` is a precomputed dict-of-dicts (from `_precompute_entry_dicts`), uses O(1) dict lookups instead of per-entry _poly() conversions. Falls back to the original path for raw symbolic matrices.

`Commit(`*blk*, *c*, *idx*`)`

Sets the latest computed values for the final SDP and saves the current state.

`Decompose()`

Returns a dictionary that associates a list to every constraint, $g_i \geq 0$ for $i = 0, \ldots, m$, where $g_0 = 1$. Each list consists of elements of algebra whose sums of squares is equal to $\sigma_i$ and $f - f_* = \sum_{i=0}^{m} \sigma_i g_i$. Here, $f_*$ is the output of the `SDPRelaxation.Minimize()`.

`ExponentsVec(`*deg*`)`

Returns all the exponents that appear in the reduced basis of all monomials of the auxiliary symbols of degree at most *deg*.

`InitSDP()`

Initializes the SDP based on the value of `self.Parallel`. If it is `True`, it runs in parallel mode, otherwise in serial.

P5.8: When `config.sparsity_block_sdp` is True (or `reduction_method='sparsity'`), routes through the correlative sparsity decomposition path instead of a monolithic SDP.

`LocalizedMoment(`*p*`)`

Computes the reduced symbolic moment generating matrix localized at *p*.

P5.6: The large polynomial product `p * m * m.T` is routed through SymEngine's C++ backend when available. Each entry of the resulting matrix is expanded with `se.expand()` (C++) before being converted back to SymPy for Groebner reduction via `ReduceExp`.

`LocalizedMoment_(`*p*`)`

Computes the reduced symbolic moment generating matrix localized at *p*.

`Minimize(**kwargs)`

Finds the minimum of the truncated moment problem which provides a lower bound for the actual minimum.

Includes stability diagnostics for high-order relaxations (Risk 1).

Telemetry: when enabled, records total pipeline time, solver status, primal/dual objectives, and moment-matrix condition number.

`MomentConstraint(cnst)`

Takes constraints on the moments. The input must be an instance of *Mom* class.

`MomentMat()`

Returns the numerical moment matrix resulted from solving the SDP.

P5.7: Precomputes Poly dicts for all entries once, then uses them directly instead of re-running _poly() per entry.

`MomentsOrd(ordr)`

Sets the order of moments to be considered.

`PolyCoefFullVec()`

return the vector of coefficient of the reduced objective function as an element of the vector space of elements of degree up to the order of moments.

`ReduceExp(expr)`

Takes an expression *expr*, either in terms of internal free symbolic variables or generating functions and returns the reduced expression in terms of internal symbolic variables, if a relation among generators is present, otherwise it just substitutes generating functions with their corresponding internal symbols.

**The reduction engine is selected by `config.quotient_basis`:**

- 'groebner' (default): classical Groebner-basis reduction, the original Irene behavior.
- 'border': BorderBasis quotient-algebra reduction (Irene), using the numerically computed multiplication tables.

`ReducedMonomialBase(deg)`

Returns a reduce monomial basis up to degree *d*.

Phase 3 integration: the reduction pipeline is controlled via `self.config`. When `config.reduction_method == 'newton_polytope'`, only exponent tuples that appear in the Minkowski sum of objective + constraint supports are generated, which can cut the basis size by 60–90 % for sparse problems. When `config.reduction_method == 'border_basis'`, the quotient-algebra border basis is used to replace the full monomial basis.

`RelaxationDeg()`

Finds the minimum required order of moments according to user's request, objective function and constraints.

`Resume()`

Resumes the process of a previously saved program.

`SaveState()`

Saves the current state of the relaxation object to the file *self.Name+'.rlx'*.

`SetMonoOrd(ordr)`

Changes the default monomial order to *ord* which mustbe among *lex*, *grlex*, *grevlex*, *ilex*, *igrlex*, *igrevlex*.

`SetNumCores(num)`

Sets the maximum number of workers which cannot be bigger than number of available cores.

`SetObjective(obj)`

Takes the objective function *obj* as an algebraic combination of the generating symbolic functions, replace the symbolic functions with corresponding auxiliary symbols and reduce them according to the given relations.

`SetSDPSolver(solver)`

**Sets the default SDP solver. The followings are currently supported:**

- CVXOPT (legacy, via CVXPY or native)
- DSDP (via CVXPY or native)
- SDPA (legacy text writer)
- CSDP (legacy text writer)
- CLARABEL (CVXPY – recommended default)
- SCS (CVXPY – first-order, fast for large problems)

The selected solver must be installed otherwise it cannot be called. CVXPY-family solvers (CLARABEL, SCS, CVXOPT) are routed through the direct DCP formulation layer and bypass legacy text-file I/O entirely. When a legacy solver is specified but CVXPY is available, CVXPY is tried first as a fast path with fallback to the legacy backend.

`State()`

Returns the latest state of the object at last break and save.

`classmethod from_problem(optim_prob: OptimizationProblem, name='SDPRlx', config=None)`

Creates an SDPRelaxations instance from an OptimizationProblem.

This method acts as an alternative constructor to bridge compatibility with problems defined using SemigroupAlgebra.

**Parameters**

- **optim_prob** (OptimizationProblem) – The optimization problem defined with SemigroupAlgebra.
- **name** (*str*) – A name for the relaxation instance.
- **config** – RelaxationConfig instance controlling Phase 3 reduction pipeline. When None, defaults to no reduction.

**Returns**

An instance of SDPRelaxations.

`getConstraint(idx)`

Returns the constraint number *idx* of the problem after reduction modulo the relations, if given.

`getMomentConstraint(idx)`

Returns the moment constraint number *idx* of the problem after reduction modulo the relations, if given.

`getObjective()`

Returns the objective function of the problem after reduction modulo the relations, if given.

`pInitSDP`(***kwargs*)

Initializes the semidefinite program (SDP), in parallel, whose solution is a lower bound for the minimum of the program.

Telemetry: when enabled, records wall-clock init time, monomial basis sizes, block dimensions, and relaxation order.

`sInitSDP`(***kwargs*)

Initializes the semidefinite program (SDP), in serial mode, whose solution is a lower bound for the minimum of the program.

Telemetry: when enabled, records wall-clock init time, monomial basis sizes, block dimensions, and relaxation order.

`class Irene.relaxations.SDRelaxSol`(*X*, *symdict={}*, *err_tol=1e-05*)

Instances of this class carry information on the solution of the semidefinite relaxation associated to an optimization problem. It includes various pieces of information:

- `SDRelaxSol.TruncatedMmntSeq` a dictionary of resulted moments
- `SDRelaxSol.MomentMatrix` the resulted moment matrix
- `SDRelaxSol.Primal` the value of the SDP in primal form
- `SDRelaxSol.Dual` the value of the SDP in dual form
- `SDRelaxSol.RunTime` the run time of the sdp solver
- `SDRelaxSol.InitTime` the total time consumed for initialization of the sdp
- `SDRelaxSol.Solver` the name of sdp solver
- `SDRelaxSol.Status` final status of the sdp solver
- `SDRelaxSol.RelaxationOrd` order of relaxation
- `SDRelaxSol.Message` the message that maybe returned by the sdp solver
- `SDRelaxSol.ScipySolver` the scipy solver to extract solutions
- `SDRelaxSol.err_tol` the minimum value which is considered to be nonzero
- `SDRelaxSol.Support` the support of discrete measure resulted from `SDPRelaxation.Minimize()`
- `SDRelaxSol.Weights` corresponding weights for the Dirac measures

`ExtractSolution`(*mthd='LH'*, *card=0*)

Extract support of the solution measure from `SDPRelaxations`:

`-mthd` should be either 'LH' or 'Scipy', where 'LH' stands for 'Lasserre-Henrion' and 'Scipy' employs a Scipy solver to find points matching the moments,

`-card` restricts the number of points of the support.

`ExtractSolutionLH`(*card=0*)

Extract solutions based on Lasserre–Henrion's method.

`ExtractSolutionScipy`(*card=0*)

This method tries to extract the corresponding values for generators of the `SDPRelaxation` class. Number of points is the rank of the moment matrix which is computed numerically according to the size of its eigenvalues. Then the points are extracted as solutions of a system of polynomial equations using a *scipy* solver. The following solvers are currently acceptable by `scipy`:

- `hybr`,

- `lm` (default),
- `broyden1`,
- `broyden2`,
- `anderson`,
- `linearmixing`,
- `diagbroyden`,
- `excitingmixing`,
- `krylov`,
- `df-sane`.

`NumericalRank()`

Finds the rank of the moment matrix based on the size of its eigenvalues. It considers those with absolute value less than `self.err_tol` to be zero.

`Pivot(`*arr*`)`

Get the leading term of each column.

`SetScipySolver(`*solver*`)`

Sets the `scipy.optimize.root` solver to *solver*.

`StblRedEch(`*A*`)`

Compute the stabilized row reduced echelon form.

`Term2Mmnt(`*trm*, *rnk*, *X*`)`

Converts a moment object into an algebraic equation.

`class Irene.sdp.sdp(`*solver='cvxopt'*, *solver_path=None*`)`

This is the class which intends to solve semidefinite programs in primal format:

$$\begin{cases} \min & \sum_{i=1}^{m} b_i x_i \\ \text{subject to} & \\ & \sum_{i=1}^{m} A_{ij} x_i - C_j \succeq 0 \quad j = 1, \ldots, k. \end{cases}$$

**For the argument *solver* following sdp solvers are supported (if they are installed):**

- *CVXOPT*,
- *CSDP*,
- *SDPA*,
- *DSDP*.

`AddConstantBlock(`*C*`)`

$C$ must be a list of `numpy` matrices that represent $C_j$ for each $j$. This method sets the value for $C = [C_1, \ldots, C_k]$.

`AddConstraintBlock(`*A*`)`

This takes a list of square matrices which corresponds to coefficient of $x_i$. Simply, $A_i = [A_{i1}, \ldots, A_{ik}]$. Note that the $i^{th}$ call of `AddConstraintBlock` fills the blocks associated with $i^{th}$ variable $x_i$.

`CvxOpt()`

This calls *CVXOPT* and *DSDP* to solve the initiated semidefinite program.

`Option`(*param*, *val*)

Sets the *param* option of the solver to *val* if the solver accepts such an option. The following options are supported by solvers:

- `CVXOPT`:
  - `show_progress`: `True` or `False`, turns the output to the screen on or off (default: `True`);
  - `maxiters`: maximum number of iterations (default: 100);
  - `abstol`: absolute accuracy (default: 1e-7);
  - `reltol`: relative accuracy (default: 1e-6);
  - `feastol`: tolerance for feasibility conditions (default: 1e-7);
  - `refinement`: number of iterative refinement steps when solving KKT equations (default: 0 if the problem has no second-order cone or matrix inequality constraints; 1 otherwise).
- `SDPA`:
  - `maxIteration`: Maximum number of iterations. The SDPA stops when the iteration exceeds `maxIteration`;
  - `epsilonStar`, `epsilonDash`: The accuracy of an approximate optimal solution of the SDP;
  - `lambdaStar`: This parameter determines an initial point;
  - `omegaStar`: This parameter determines the region in which the SDPA searches an optimal solution;
  - `lowerBound`: Lower bound of the minimum objective value of the primal problem;
  - `upperBound`: Upper bound of the maximum objective value of the dual problem;
  - `betaStar`: Parameter controlling the search direction when current state is feasible;
  - `betaBar`: Parameter controlling the search direction when current state is infeasible;
  - `gammaStar`: Reduction factor for the primal and dual step lengths; $0.0 <$ `gammaStar` $< 1.0$.

`SetObjective`(*b*)

Takes the coefficients of the objective function.

`static VEC`(*M*)

Converts the matrix M into a column vector acceptable by *CVXOPT*.

`csdp()`

Calls *CSDP* to solve the initiated semidefinite program.

Deprecated since version This: method uses the legacy text-file I/O interface. The CVXPY DCP layer is now preferred and tried first in `solve()`.

`static parse_solution_matrix`(*iterator*)

Parses and returns the matrices and vectors found by *SDPA* solver. This was taken from *ncpol2sdpa* and customized for *Irene*.

`read_csdp_out`(*filename*, *txt*)

Takes a file name and a string that are the outputs of *CSDP* as a file and command line outputs of the solver and extracts the required information.

`read_sdpa_out`(*filename*)

Extracts information from *SDPA*'s output file *filename*. This was taken from *ncpol2sdpa* and customized for *Irene*.

`sdpa`()

Calls *SDPA* to solve the initiated semidefinite program.

Deprecated since version This: method uses the legacy text-file I/O interface. The CVXPY DCP layer is now preferred and tried first in `solve()`.

`sdpa_param`()

Produces sdpa.param file from `SolverOptions`.

`solve`(***kwargs*)

Solves the initiated semidefinite program.

Tries CVXPY first (no text I/O, direct DCP formulation). Falls back to the legacy solver specified by `self.solver` if CVXPY is unavailable or fails.

Telemetry: when enabled, records wall-clock solve time, block dimensions, variable count, and solver name in a structured dict accessible via `Irene.telemetry.get_telemetry()`.

`write_sdpa_dat`(*filename*)

Writes the semidefinite program in the file *filename* with dense SDPA format.

`write_sdpa_dat_sparse`(*filename*)

Writes the semidefinite program in the file *filename* with sparse SDPA format. Uses sparse iteration over non-zero entries to avoid dense nested loops.

`class Irene.program.OptimizationProblem`(*sga:* SemigroupAlgebra | *None* = *None*, *relations:* *list*[SemigroupAlgebraElement] | *None* = *None*)

This class represents an optimization problem.

`sga`

The semigroup algebra of the optimization problem.

**Type**
*SemigroupAlgebra*

`relations`

The relations of the optimization problem.

**Type**
list[*SemigroupAlgebraElement*]

`semigroup`

The semigroup of the optimization problem.

**Type**
*CommutativeSemigroup*

`objective`

The objective function of the optimization problem.

**Type**
*SemigroupAlgebraElement*

`constraints`

The constraints of the optimization problem.

**Type**
list[*SemigroupAlgebraElement*]

`objective_degree`
The degree of the objective function.

**Type**
int

`objective_half_degree`
The half degree of the objective function.

**Type**
int

`constraints_degree`
The degrees of the constraints.

**Type**
list[int]

`constraints_half_degree`
The half degrees of the constraints.

**Type**
list[int]

`objective_trms_with_positive_coefficient`
The terms of the objective function with positive coefficients.

**Type**
list[*SemigroupAlgebraElement*]

`objective_trms_with_negative_coefficient`
The terms of the objective function with negative coefficients.

**Type**
list[*SemigroupAlgebraElement*]

`objective_terms_with_even_exponent`
The terms of the objective function with even exponents.

**Type**
list[*SemigroupAlgebraElement*]

`objective_terms_with_odd_exponent`
The terms of the objective function with odd exponents.

**Type**
list[*SemigroupAlgebraElement*]

`constraint_trms_with_positive_coefficient`
The terms of the constraints with positive coefficients.

**Type**
list[*SemigroupAlgebraElement*]

`constraint_trms_with_negative_coefficient`
The terms of the constraints with negative coefficients.

**Type**
list[*SemigroupAlgebraElement*]

**constraint_terms_with_even_exponent**

The terms of the constraints with even exponents.

**Type**
list[*SemigroupAlgebraElement*]

**constraint_terms_with_odd_exponent**

The terms of the constraints with odd exponents.

**Type**
list[*SemigroupAlgebraElement*]

**total_degree**

The total degree of the optimization problem.

**Type**
int

**add_constraints**(*const: list[SemigroupAlgebraElement]*) → None

Adds polynomial inequality or equality constraints to the optimization problem.

This method extends the optimization problem with additional constraints, typically representing feasibility conditions like bounds or domain restrictions. Each constraint is analyzed for its degree to assist in later relaxation construction.

Place in code structure: Complements set_objective() to fully define the constrained optimization problem. Called during problem setup before relaxation methods.

**Parameters**
**const** (*list[SemigroupAlgebraElement]*) – List of polynomial constraint expressions, each represented as a SemigroupAlgebraElement.

**Returns**
None

**Side effects:**

- Appends each constraint to self.constraints
- Updates self.semigroup if any constraint uses a different semigroup
- Computes and stores degree and half-degree for each constraint

**analyse_program**() → None

Performs structural analysis of the optimization problem's polynomial terms.

This method categorizes all terms from the objective function and constraints based on two key properties: (1) sign of coefficients, and (2) parity of exponents. This analysis is essential for specialized polynomial optimization techniques like SONC (sums of non-negative circuit polynomials) and conditional relaxations.

Place in code structure: Called after problem setup to preprocess the polynomial structure. Populates classification lists used by relaxation algorithms (relaxations.py) and decomposition methods (sonc.py).

**Returns**
None

**Side effects:**
Populates the following attributes: - objective_trms_with_positive/negative_coefficient: Objective terms by sign - constraint_trms_with_positive/negative_coefficient: Constraint

terms by sign - objective_terms_with_even/odd_exponent: Objective terms by exponent parity - constraint_terms_with_even/odd_exponent: Constraint terms by exponent parity

`convex_combination`(*point: Sequence[float]*) → ndarray | None

Finds the representation of a point as a convex combination of Newton polytope vertices.

This method formulates and solves a linear program to express a point as a convex combination of the polytope vertices, enforcing both non-negativity of coefficients and that they sum to 1. This is more restrictive than linear_combination() but guarantees the point lies within the convex hull.

Place in code structure: Advanced geometric utility for Newton polytope analysis. Uses scipy.optimize to solve the constrained optimization problem. Results can guide sparse relaxation strategies by identifying how to express new terms using existing ones.

**Parameters**

**point** – A numeric sequence representing coordinates in the exponent space that should be expressed as a convex combination of self.vertices.

**Returns**

**Array of non-negative coefficients summing to 1 that express the point**
as a convex combination of vertices, or None if no such combination exists (i.e., point is outside the Newton polytope).

**Return type**
numpy.ndarray

`delta`(*xprsn:* SemigroupAlgebraElement, *deg: int*) → dict[str, set]

Identifies problematic terms in polynomial expressions for relaxation construction.

The delta function categorizes terms that cannot be directly represented in a sum-of-squares (SOS) form: terms with odd exponents or negative coefficients. These terms are grouped by whether their total degree equals the target degree ('=d') or is less than it ('<d'), which affects how they're handled in relaxations.

Place in code structure: Core analysis method used by relaxation algorithms (relaxations.py) to identify which terms need special treatment through auxiliary variables or alternative decomposition strategies.

**Parameters**

- **xprsn** (SemigroupAlgebraElement) – The polynomial expression to analyze.
- **deg** (*int*) – The target degree for comparison.

**Returns**

**Dictionary with two keys:**
'=d': Set of monomials with degree exactly equal to deg '<d': Set of monomials with degree less than deg Both contain only non-square or negative coefficient terms.

**Return type**
dict[str, set[*SemigroupAlgebraElement*]]

`delta_vertex`(*xprsn:* SemigroupAlgebraElement, *vertices: list*) → None

Computes delta relative to Newton polytope vertices (currently unimplemented).

This method is intended to analyze terms based on their relationship to the vertices of the Newton polytope, likely for geometric programming or sparsity exploitation.

Place in code structure: Placeholder for advanced geometric analysis that would work with the newton() and related polytope methods for exploiting problem structure.

**Parameters**

- **xprsn** (SemigroupAlgebraElement) – The expression to analyze.
- **vertices** (*list*) – List of vertices from the Newton polytope.

**Returns**
None (not yet implemented)

**static has_symbol**(*symb: str*, *mono:* SemigroupAlgebraElement) → tuple[bool, int]

Searches for a specific variable (symbol) in a monomial and returns its position.

This method determines whether a given variable name appears in a monomial's factorization, which is useful for variable-specific operations like substitution, elimination, or targeted relaxation strategies.

Place in code structure: Static utility method for monomial inspection. Used by various algorithms that need to check variable presence or extract variable-specific information from polynomial terms.

**Parameters**

- **symb** (*str*) – The name of the variable/symbol to search for (e.g., 'x', 'y').
- **mono** (SemigroupAlgebraElement) – The monomial to search within.

**Returns**

**A tuple where the first element is True if the symbol is found,**
False otherwise. The second element is the index of the symbol in the monomial's array form if found, or -1 if not found.

**Return type**
tuple[bool, int]

**in_newton**(*point: Sequence[float]*) → bool

Tests whether a point lies inside the Newton polytope.

This method uses Delaunay triangulation to efficiently determine if a given point (typically representing a monomial's exponents) is within the convex hull of the problem's support. This is useful for determining which monomials are representable as convex combinations of existing terms.

Place in code structure: Geometric query method that depends on newton() being called first. Used in sparsity analysis and to determine valid monomial spaces for relaxations.

**Parameters**
**point** – A numeric sequence (tuple, list, or array) representing exponent coordinates to test for inclusion in the Newton polytope.

**Returns**

**True if the point is inside or on the boundary of the Newton polytope,**
False otherwise.

**Return type**
bool

**Raises**
ValueError – If vertices are empty or if Delaunay triangulation fails due to degenerate geometry.

`linear_combination`(*point: Sequence[float]*) → ndarray

Expresses a point as a linear combination of Newton polytope vertices.

This method solves a linear system to find coefficients that express the given point as a linear combination of polytope vertices. Note that unlike convex_combination(), this method doesn't enforce non-negativity or that coefficients sum to 1, so it's suitable for affine (not necessarily convex) combinations.

Place in code structure: Geometric utility for expressing points in terms of vertices. Complements convex_combination() but with less restrictive constraints. Used when affine relationships are needed rather than strictly convex ones.

**Parameters**
`point` – A numeric sequence representing coordinates to express in terms of vertices.

**Returns**

**Array of coefficients for the linear combination. Note that these** coefficients may be negative or sum to values other than 1.

**Return type**
numpy.ndarray

**Note**

Excludes the origin vertex [0, 0, ...] if present to avoid singularity issues.

`mono2ord_tuple`(*mono: Any*) → tuple[int, ...]

Converts a monomial from semigroup representation to a tuple of exponents.

This method transforms a symbolic monomial into a numeric tuple where each position corresponds to a generator in the semigroup, containing that generator's exponent. This standardized format is essential for numerical computations like convex hull calculations and geometric analysis.

Place in code structure: Conversion utility used by newton() and related geometric methods to transform symbolic polynomial data into numeric arrays suitable for scipy geometric algorithms.

**Parameters**
`mono` (`SemigroupAlgebraElement`) – The monomial to convert. Can also be a scalar (int or float) representing a constant term.

**Returns**

**A tuple of integers representing the exponents of each generator in order.** For constants, returns a tuple of zeros.

**Return type**
tuple

`newton`() → None

Computes the Newton polytope of the optimization problem.

The Newton polytope is the convex hull of all exponent vectors appearing in the objective function and constraints. This geometric object encodes the sparsity structure of the problem and can be exploited to construct more efficient relaxations.

Place in code structure: Geometric analysis method that should be called after problem setup. Its results (stored in newton_polytope and vertices) are used by in_newton(), convex_combination(), and potentially specialized relaxation strategies.

**Returns**
None

**Side effects:**

- Collects all exponent tuples from objective and constraints
- Computes and stores the convex hull in self.newton_polytope
- Extracts and stores sorted vertex list in self.vertices
- Raises ValueError if polytope is degenerate (insufficient points or dimension mismatch)

`static omega(`*xprsn:* SemigroupAlgebraElement, *deg:* *int*`)` → list[tuple[float, Any]]

Extracts terms from a polynomial that don't match a specific univariate degree pattern.

The omega function filters out univariate monomials of exactly the specified degree, keeping all multivariate terms, constant terms, and univariate terms of other degrees. This is used in conditional optimization where certain degree-specific terms need special treatment.

Place in code structure: Static helper method used by delta() to identify terms that require special handling in relaxation construction, particularly for conditional constraints and geometric programming.

**Parameters**

- **xprsn** (SemigroupAlgebraElement) – The polynomial expression to filter.
- **deg** (*int*) – The specific univariate degree to exclude.

**Returns**

**List of (coefficient, monomial) tuples**
representing terms that don't match the degree criterion.

**Return type**
list[tuple[int, *SemigroupAlgebraElement*]]

`program_degree()` → int

Computes the overall degree of the optimization problem.

This method determines the maximum degree among all polynomials (objective and constraints) in the problem, rounded up to the nearest even number. This degree is critical for determining the size of sum-of-squares (SOS) relaxations.

Place in code structure: Utility method used during relaxation construction to determine the appropriate degree for SOS decompositions and moment matrices.

**Returns**

**The maximum degree among all polynomials, adjusted to be even (adds 1 if odd).**
This ensures compatibility with SOS relaxations which require even degrees.

**Return type**
int

`set_objective(`*obj:* SemigroupAlgebraElement`)` → None

Sets the objective function for the optimization problem.

This method is central to defining the optimization problem. It assigns the objective function that will be minimized or maximized, and automatically updates the semigroup and computes the degree information needed for polynomial optimization.

Place in code structure: Core setup method that must be called before relaxation generation. Works with add_constraints() to fully specify the optimization problem.

**Parameters**

**obj** (SemigroupAlgebraElement) – A polynomial expression to be optimized, represented as an element in a semigroup algebra.

**Returns**

None

**Side effects:**

- Updates self.objective with the provided expression
- Updates self.semigroup if different from current
- Computes and stores objective_degree (total degree of the leading monomial)
- Computes and stores objective_half_degree (ceiling of degree/2)

**static square_exponent**(*xpnt: Any*) → bool

Determines if a monomial has only even exponents (is a perfect square).

This method checks whether all variables in a monomial appear with even powers, meaning the monomial can be expressed as a square of another monomial. This property is fundamental for identifying terms that are inherently non-negative and for constructing sum-of-squares decompositions.

Place in code structure: Static utility method used throughout analysis and relaxation construction. Called by analyse_program(), delta(), and other methods that need to identify square monomials for optimization techniques.

**Parameters**

**xpnt** (SemigroupAlgebraElement) – A monomial whose exponents are to be checked.

**Returns**

**True if all exponents in the monomial are even (divisible by 2),**
False if any exponent is odd.

**Return type**

bool

**to_sympy**(*expr:* SemigroupAlgebraElement, *sym_map: dict[str, Symbol]*)

Converts a SemigroupAlgebraElement polynomial to a SymPy symbolic expression.

This method provides interoperability with the SymPy computer algebra system, enabling use of SymPy's extensive symbolic manipulation capabilities (differentiation, integration, simplification, etc.) on polynomials defined in the semigroup algebra framework.

Place in code structure: Interface method for exporting to SymPy. Useful for visualization, symbolic analysis, or interfacing with other tools that expect SymPy expressions. Complements the internal semigroup algebra representation.

**Parameters**

- **expr** (SemigroupAlgebraElement) – The polynomial expression to convert.
- **sym_map** (*dict*) – Dictionary mapping generator name strings to SymPy Symbol objects, e.g., {'x': engine.Symbol('x'), 'y': engine.Symbol('y')}.

**Returns**

**A SymPy expression algebraically equivalent to the input, using the** symbols provided in sym_map.

**Return type**
sympy.Expr

`tuple2mono`(*xpnt: Sequence[int]*)

Converts a tuple of exponents back to a monomial in semigroup representation.

This method performs the inverse operation of mono2ord_tuple, constructing a symbolic monomial from numeric exponents. Each position in the input tuple corresponds to the power of the respective generator in the semigroup.

Place in code structure: Inverse conversion utility that works with mono2ord_tuple to enable round-trip conversions between symbolic and numeric representations. Used when translating geometric analysis results back to polynomial terms.

**Parameters**
`xpnt` – A sequence (tuple or list) of integers representing exponents for each generator in the semigroup, in the same order as semigroup.generators.

**Returns**

**The monomial product of generators raised to their** corresponding powers from xpnt.

**Return type**
*SemigroupAlgebraElement*

A module for working with commutative semigroups and their differential algebras.

This module provides classes and functions for working with commutative semigroups and their differential algebras.

A semigroup is a set S together with an associative binary operation on S. A commutative semigroup is a semigroup in which the binary operation is commutative.

A semigroup algebra is a vector space over a field with a basis consisting of the elements of a semigroup equipped with multiplication compatible with the semigroup's operation.

This module provides the following classes:

- CommutativeSemigroup: A class representing a commutative semigroup.
- AtomicSGElement: A class representing an atomic element of a semigroup algebra.
- SemigroupAlgebraElement: A class representing an element of a semigroup algebra.
- SemigroupAlgebra: A class representing a semigroup algebra.

This module also provides the following functions:

- _degree: Computes the degree of a given expression.
- diff: Computes the derivative of an expression in a semigroup algebra.

`class Irene.grouprings.AtomicSGElement`(*semigroup:* CommutativeSemigroup, *element: str*)

An atomic element of a semigroup algebra.

An atomic element is an element of the semigroup algebra that is not a sum of other elements.

`semigroup`

The semigroup of the element.

**Type**
*CommutativeSemigroup*

`symbol`

The symbol of the element.

**Type**
str

`content`

The content of the element.

**Type**
list

`LC()` → float

Returns the leading coefficient of the element.

The leading coefficient of an element is the coefficient of the term with the highest degree.

**Returns**
The leading coefficient of the element.

**Return type**
float

`LM()` → Expr

Returns the leading monomial of the element.

The leading monomial of an element is the monomial with the highest degree.

**Returns**
The leading monomial of the element.

**Return type**
sympy.Expr

`LT()` → *SemigroupAlgebraElement*

Returns the leading term of the element.

The leading term of an element is the term with the highest degree.

**Returns**
The leading term of the element.

**Return type**
*SemigroupAlgebraElement*

`divide`(*fs: list*) → tuple[list, *SemigroupAlgebraElement*]

Divides the element by a list of elements or expressions.

**Parameters**
**fs** (*list*) – The list of elements or expressions to divide by.

**Returns**
A tuple containing the quotient and remainder of the division.

**Return type**
tuple

`lt_divisible_by`(*expr: Any*) → bool

Checks if the leading term of the element is divisible by the leading term of another element or expression.

**Parameters**
**expr** (*int*, *float*, AtomicSGElement, SemigroupAlgebraElement) – The element or expression to check divisibility by.

**Returns**
True if the leading term of the element is divisible by the leading term of the other element or expression, False otherwise.

**Return type**
bool

**class** Irene.grouprings.**CommutativeSemigroup**(*gens: list*, *is_semigroup: bool = True*, *is_abelian: bool = True*)

A class representing a commutative semigroup.

A semigroup is a set S together with an associative binary operation on S. A commutative semigroup is a semigroup in which the binary operation is commutative.

**gens**
The generators of the semigroup.

**Type**
list

**is_semigroup**
True if the semigroup is a semigroup, False otherwise.

**Type**
bool

**is_abelian**
True if the semigroup is abelian, False otherwise.

**Type**
bool

**rels**
The relations of the semigroup.

**Type**
list

**aux_rels**
The auxiliary relations of the semigroup.

**Type**
list

**inverses**
The inverses of the generators of the semigroup.

**Type**
dict

**max_deg**
The maximum degree of the relations of the semigroup.

**Type**
int

`edges`

The edges of the lattice of the semigroup.

**Type**
list

`vertices`

The vertices of the lattice of the semigroup.

**Type**
dict

`symbols`

The symbols of the semigroup.

**Type**
list

`add_relations`(*rels: list*)

Adds relations to the semigroup.

**Parameters**
**rels** (*list*) – The relations to add to the semigroup.

`degree`(*expr*) → int

Computes the degree of an expression in the semigroup.

The degree of an expression is the sum of the absolute values of the exponents of its terms.

**Parameters**
**expr** – The expression whose degree is to be computed.

**Returns**
The degree of the expression.

**Return type**
int

`element_sub_lattice`(*elm: Expr*, *ex: set = None*) → set

Computes the sublattice of the lattice of the semigroup generated by an element.

**Parameters**

- **elm** (*sympy.Expr*) – The element of the semigroup to generate the sublattice from.
- **ex** (*set*) – The set of elements to exclude from the sublattice.

**Returns**
The sublattice of the lattice of the semigroup generated by the element.

**Return type**
set

`lattice_edges`(*degree: int*) → None

Computes the edges of the lattice of the semigroup.

**Parameters**
**degree** (*int*) – The degree of the lattice.

`lattice_vertices()` → None

Computes the vertices of the lattice of the semigroup.

The vertices of the lattice are the elements of the semigroup that are generated by the edges of the lattice.

`positive_exp(expr: Expr)` → bool

Checks if an expression in the semigroup has only positive exponents.

**Parameters**
**expr** (*sympy.Expr*) – The expression to check.

**Returns**
True if the expression has only positive exponents, False otherwise.

**Return type**
bool

`class Irene.grouprings.SemigroupAlgebra(semigroup: CommutativeSemigroup)`

A semigroup algebra.

A semigroup algebra is a vector space over a field with a basis consisting of the elements of a semigroup.

`gens`

The generators of the semigroup.

**Type**
list

`semigroup`

The semigroup of the algebra.

**Type**
*CommutativeSemigroup*

`derivatives`

A list of dictionaries mapping the elements of the semigroup to their derivatives.

**Type**
list

`add_derivative(base_map: dict)` → None

Adds a derivative to the algebra.

**Parameters**
**base_map** (*dict*) – A dictionary mapping the elements of the semigroup to their derivatives.

`derivative(expr: Any, idx: int)`

Computes the derivative of an expression in the algebra.

**Parameters**

- **expr** (SemigroupAlgebraElement, AtomicSGElement, *int*, *float*) – The expression to differentiate.
- **idx** (*int*) – The index of the derivative to use.

**Returns**
The derivative of the expression.

**Return type**
*SemigroupAlgebraElement*

`diff(`*expr: Any*, *base_map: dict*`)`

Computes the derivative of an expression in the algebra.

**Parameters**

- **expr** (SemigroupAlgebraElement, AtomicSGElement, *int*, *float*) – The expression to differentiate.
- **base_map** (*dict*) – A dictionary mapping the elements of the semigroup to their derivatives.

**Returns**
The derivative of the expression.

**Return type**
*SemigroupAlgebraElement*

`class Irene.grouprings.SemigroupAlgebraElement(`*terms: list*, *semigroup:* CommutativeSemigroup`)`

An element of a semigroup algebra.

A semigroup algebra is a vector space over a field with a basis consisting of the elements of a semigroup.

`content`

The content of the element.

**Type**
list

`semigroup`

The semigroup of the element.

**Type**
*CommutativeSemigroup*

`LC() →` float

Returns the leading coefficient of the element.

The leading coefficient of an element is the coefficient of the term with the highest degree.

**Returns**
The leading coefficient of the element.

**Return type**
float

`LM() →` Expr

Returns the leading monomial of the element.

The leading monomial of an element is the monomial with the highest degree.

**Returns**
The leading monomial of the element.

**Return type**
sympy.Expr

`LT()` → *SemigroupAlgebraElement*

Returns the leading term of the element.

The leading term of an element is the term with the highest degree.

**Returns**
The leading term of the element.

**Return type**
*SemigroupAlgebraElement*

`divide`(*fs: list*) → tuple[list, *SemigroupAlgebraElement*]

Divides the element by a list of elements or expressions.

**Parameters**
**fs** (*list*) – The list of elements or expressions to divide by.

**Returns**
A tuple containing the quotient and remainder of the division.

**Return type**
tuple

`lt_divisible_by`(*expr: Any*) → bool

Checks if the leading term of the element is divisible by the leading term of another element or expression.

**Parameters**
**expr** (*int*, *float*, AtomicSGElement, SemigroupAlgebraElement) – The element or expression to check divisibility by.

**Returns**
True if the leading term of the element is divisible by the leading term of the other element or expression, False otherwise.

**Return type**
bool

This module provides a framework for polynomial optimization using the techniques introduced by Ghasemi, Lasserre, and Marshall, using Geometric Programming.

`class Irene.geometric.GPRelaxations`(*prog:* OptimizationProblem, ***kwargs*)

This class aims to provide a framework for polynomial optimization using the techniques introduced by Ghasemi, Lasserre, and Marshall, using Geometric Programming.

`auto_transform_matrix()` → ndarray

Compute the automatic transformation matrix H.

**Returns**
The transformation matrix H.

`static compare_diags`(*vec: list[float]*) → tuple[int, list[float]]

Compare two vectors by the number of non-zero elements.

**Parameters**
**vec** – The first vector.

**Returns**
The number of non-zero elements in the vector.

`h_plus`(*xprsn: Any*, *idn: Any = None*) → float

Compute the h_plus function for a given expression.

**Parameters**

- **xprsn** – The expression to evaluate.
- **idn** – The identity element of the semigroup.

**Returns**
The value of h_plus(xprsn).

`solve`(***kwargs*)

Form the geometric program relaxation.

**Returns**
The optimal value of the relaxation.

`transform_program`() → None

Transform the program using the transformation matrix H.

`class Irene.sonc.SONCRelaxations`(*prog: OptimizationProblem*, ***kwargs*)

Framework for constrained SONC relaxations formulated as geometric programs.

`solve`(***kwargs*)

Build and solve the constrained SONC geometric program.

## 28.1 SOS+SONC Relaxation Framework

Implements the SOS+SONC two-step optimization algorithms from Moritz Schick's PhD thesis (*Sums of squares plus sums of nonnegative circuit polynomials*, Universität Konstanz), translated from MATLAB to Python and integrated into the Irene framework.

### 28.1.1 Classes

**SOSONCRelaxations**
Combined SOS+SONC lower-bound computation for unconstrained polynomial optimization.

**SOSONCRelaxSol**
Container for relaxation results (optimal value, certificates, timing, status).

`class Irene.sosonc.SOSONCRelaxSol`

Carries optimisation results for SOS+SONC relaxations.

`val`

Optimal $\lambda^*$, a lower bound on $f^*$.

**Type**
float

`method`

`'sos'`, `'sonc'`, `'sos-first'`, or `'sonc-first'`.

**Type**
str

`f_sos`

SOS summand of `f - val` (if applicable).

**Type**
optional

`f_sonc`
SONC summand of `f - val` (if applicable).

**Type**
optional

`status`
`'optimal'`, `'infeasible'`, or `'error'`.

**Type**
str

`error_code`
0 = success, 1 = infeasible, 2 = solver error.

**Type**
int

`runtime`
Wall-clock time in seconds.

**Type**
float

`message`
Human-readable status message.

**Type**
str

`class Irene.sosonc.SOSONCRelaxations(`*prog:* OptimizationProblem, ***kwargs*`)`
Combined SOS+SONC relaxation framework.

Implements the algorithms from Schick's SOS+SONC toolbox:

- `globalMinSOS` – pure SOS relaxation (SDP via Gram matrix)
- `globalMinSONC` – pure SONC relaxation (signomial GP)
- `globalMinSOSPSONC` – two-step SOS+SONC (Algorithms 4 & 5)

**Parameters**

- **`prog`** (OptimizationProblem) – An Irene `OptimizationProblem` with an objective set via `prog.Minimize(f)`. Constraints are optional.
- **`error_bound`** (*float*) – Numerical zero tolerance (default `1e-10`).
- **`verbosity`** (*int*) – Log level (0 = silent, 1 = verbose; default 1).
- **`solver`** (*str*) – SDP solver name forwarded to `SDPRelaxations` (`'cvxopt'`, `'csdp'`, `'sdpa'`, `'dsdp'`).
- **`use_local_solve`** (*bool*) – Use signomial GP local solve for the SONC portion (default `True`).
- **`relaxation_order`** (*int*) – Relaxation order for SDP (default `1`, i.e., the first Lasserre moment relaxation).

`globalMinSONC()` → *SOSONCRelaxSol*

Compute a lower bound using the SONC relaxation.

Solves $\sup\{\lambda : f - \lambda \in C\}$ via a signomial geometric program using Irene's `SONCRelaxations`.

**Return type**
*SOSONCRelaxSol*

`globalMinSOS()` → *SOSONCRelaxSol*

Compute a lower bound using the SOS relaxation.

Solves $\sup\{\lambda : f - \lambda \in \Sigma\}$ via a Gram-matrix SDP using Irene's `SDPRelaxations`.

**Return type**
*SOSONCRelaxSol*

`globalMinSOSPSONC`(*first: str = 'sos'*) → *SOSONCRelaxSol*

Two-step SOS+SONC lower bound (Algorithms 4 & 5).

**Parameters**
**first** (*str*) – `'sos'` (Algorithm 4: SOS preprocess, then SONC) or `'sonc'` (Algorithm 5: SONC preprocess, then SOS).

**Returns**
Lower bound $f^*_{Sigma+C}$ together with decomposed certificate.

**Return type**
*SOSONCRelaxSol*

`Irene.sosonc.sosonc_bounds`(*prog:* OptimizationProblem, ***kwargs*) → dict[str, float]

Compute SOS, SONC, and SOS+SONC lower bounds.

Returns a dict with keys `'sos'`, `'sonc'`, `'sos_first'`, `'sonc_first'`.

Border basis module for quotient algebra representations.

Border bases generalize Gröbner bases to zero-dimensional ideals and provide numerically stable bases for the quotient algebra K[x]/I. Unlike Gröbner bases, which depend on a monomial ordering and can be ill-conditioned, border bases work with any finite K-basis of the quotient space and maintain numerical stability through orthogonalization.

**Key references:**

- Traverso (1990), "A new algorithm for computing in algebraic extensions"
- Greuel & Pfister (2002), "A Border Basis Algorithm"
- Becker et al. (2005), "Border Bases and the Moment Problem"

**The border basis representation consists of:**

1. A monomial basis B (standard monomials spanning K[x]/I)
2. The border partial B = {bcdotx_i : b in B, i=1..n} B
3. Multiplication tables: for each f in partial B, the representation of f mod I as a linear combination of elements in B

`class Irene.border_basis.BorderBasis`(*variables*, *generators*, *degree*)

Border basis for the quotient algebra K[x_{1},. . . ,x_n]/I.

Given an ideal I generated by polynomials g_{1},. . . ,g_m in K[x_{1},. . . ,x_n], this class computes a border basis representation of the quotient algebra up to a specified degree bound. The border basis provides:

- A monomial basis B for the quotient space (degree leqslant d)
- The border partial B (monomials of degree d+1 reachable from B)
- Multiplication tables expressing each border element as a linear combination of basis elements modulo I

**Parameters**

- **variables** – List of symbolic variables [x_{1}, . . . , x_n].
- **generators** – List of polynomials generating the ideal I.
- **degree** – Maximum degree for the monomial basis.

**nvars**

Number of variables.

**degree**

Degree bound of the basis.

**basis**

List of exponent tuples forming the monomial basis B.

**border**

List of exponent tuples forming the border partial B.

**mult_tables**

Dict mapping each border element to its coefficient vector over the basis (i.e., f equiv Sigma c_b cdot b^alpha mod I).

**conditioning_diagnostic()**

Compute numerical conditioning diagnostics for the border basis.

**Returns**

'condition_number': condition number of the multiplication table matrix 'basis_conditioning': condition number of the basis monomial evaluation matrix 'is_well_conditioned': boolean flag (True if cond < 1e10)

**Return type**

dict with keys

**moment_matrix_structure()**

Return the block structure of the moment matrix induced by the border basis.

The moment matrix M_alpha has entries M_{alpha,beta} = y_{alpha+beta} where the moment variables. The border basis partitions this into blocks based on which monomials appear in the basis vs the border.

**Returns**

'basis_size': number of basis elements 'border_size': number of border elements 'block_structure': list of (row_start, row_end, col_start, col_end) tuples

**Return type**

dict with keys

**reduce(*expr*)**

Reduce an expression using the border basis multiplication tables.

**Parameters**

**expr** – A polynomial expression to reduce modulo the ideal.

**Returns**
The reduced expression as a linear combination of basis monomials.

Correlative sparsity detection for SDP relaxation decomposition.

Correlative sparsity exploits the fact that not all variables appear together in every polynomial constraint. By building a variable dependency graph where edges connect variables that co-occur in the same term, we can find connected components and decompose a large moment matrix into smaller independent blocks.

**Key references:**

- Hall & Sastry (2015), "Exploiting sparsity in polynomial optimization"
- Kurzhanskiy et al. (2017), "Sparse semidefinite programming relaxations"
- Louveaux et al. (2018), "Correlative and term-at-a-time sparsity"

The implementation uses a union-find data structure for efficient connected component detection, then returns the clique decomposition needed to partition moment matrices.

`class Irene.sparsity.CorrelativeSparsity(`*num_vars: int*, *var_names: List | None = None*`)`

Detect and exploit correlative sparsity in polynomial optimization problems.

Given an optimization problem with variables x_1, . . . , x_n and polynomials (objective + constraints), this class builds a variable dependency graph where an edge (i, j) exists if variables i and j appear together in some monomial term. The connected components of this graph define the correlative sparsity pattern.

**Parameters**

- **num_vars** – Number of optimization variables.
- **var_names** – Optional list of variable names/symbols for debugging.

`adjacency`

Adjacency list representation of the dependency graph.

`components`

Connected components as lists of variable indices.

`is_sparse`

True if sparsity detected (more than one component).

`add_poly_terms(`*exponent_dict: Dict[Tuple[int, ...], object]*`)` → None

Add edges from a polynomial's term dictionary.

**Parameters**
**exponent_dict** – Mapping from exponent tuple -> coefficient (as returned by engine.Poly(. . . ).as_dict()). Each key represents one monomial.

`add_term(`*var_indices: List[int]*`)` → None

Add edges for a monomial term involving the given variables.

For a term like x_1^2 * x_3, var_indices would be [0, 2]. All pairs of co-occurring variables get connected.

**Parameters**
**var_indices** – Indices of variables that appear in this term.

`finalize()` → List[List[int]]

Compute connected components and return them sorted by size (descending).

`moment_matrix_partition`(*deg: int*) → Dict[int, List[Tuple[int, ...]]]

Partition the moment matrix basis by sparsity components.

For each connected component of variables, compute which exponent tuples belong exclusively to that component (i.e., only use variables from that component). This allows decomposing the large moment matrix into smaller independent blocks.

**Parameters**
**deg** – Maximum degree for moment basis generation.

**Returns**
Dict mapping component index -> list of exponent tuples belonging to that component's moment block. Exponents that span multiple components are assigned to a 'cross' block (key=-1).

`reduction_factor`(*deg: int*) → float

Estimate the moment matrix size reduction from sparsity.

Returns the ratio of total work with sparsity vs without. A value $< 1$ means sparsity helps. For a problem decomposed into k components of sizes n_1, . . . , n_k, the reduction is roughly: `sum_i (2*deg choose n_i) / (2*deg choose n)`

**Parameters**
**deg** – Relaxation degree.

**Returns**
Reduction factor ($< 1$ means improvement).

`summary`() → Dict

Return a human-readable summary of the sparsity pattern.

`class Irene.sparsity.UnionFind`(*n: int*)

Disjoint-set union-find with path compression and rank.

`get_components`() → Dict[int, List[int]]

Return mapping from root -> list of members.

`Irene.sparsity.detect_sparsity_from_polys`(*polynomials*, *num_vars: int*) → *CorrelativeSparsity*

Detect correlative sparsity from a list of polynomial expressions.

**Parameters**

- **polynomials** – List of symbolic polynomial expressions.
- **num_vars** – Number of variables in the problem.

**Returns**
Configured CorrelativeSparsity instance with finalized components.

`Irene.sparsity.detect_sparsity_from_problem`(*prog*) → *CorrelativeSparsity*

Detect correlative sparsity from an OptimizationProblem instance.

Inspects the objective and constraints, extracts variable co-occurrence patterns from each polynomial's term structure, and builds the dependency graph.

**Parameters**
**prog** – An OptimizationProblem with set_objective() and add_constraint() already called.

**Returns**
Configured CorrelativeSparsity instance with finalized components.

Newton polytope monomial pruning for moment matrix dimension reduction.

For a polynomial optimization problem, the Newton polytopes of the objective and constraints identify exponent candidates that are *potentially* relevant: an exponent outside `2\cdotNewt(f)` can never occur as a term of `g*h` with `supp(g), supp(h)` inside the Minkowski-sum support, so it is never needed in the objective coefficient vector. Filtering on this hull reduces the moment matrix size – sometimes by orders of magnitude for sparse problems.

Safety contract: hull membership alone is *not* a certificate that an exponent may be removed from the moment/Gram basis of a Lasserre SDP. Removing exponents drops PSD coupling entries and can introduce recession directions (or force invalid objective values) even when every objective-support exponent is kept; the Motzkin polynomial at order 3 is such a counterexample (the pruned SDP has an unbounded rank-one recession ray while the full one is bounded). Until a complete SOS-support certificate exists, callers must pass `required_support` with the exponents that must be retained regardless of hull membership; the certified safe choice at present is the full degree-bounded support.

**Key references:**

- Parrilo (2000), "Structured Semidefinite Programs and Semialgebraic Geometry"
- Lasserre (2006), "Moments, Positive Polynomials and Their Applications"
- Kim & Kojima (2014), "Sparse SOS decompositions via Newton polytopes"

The implementation computes the Minkowski sum of scaled Newton polytopes, then filters the full monomial basis to those inside the hull, always retaining `required_support`.

**class** Irene.newton_polytope.**NewtonPruner**(*num_vars: int*, *max_degree: int*, *polytope_vertices: ndarray | None = None*, *required_support: Set[Tuple[int, ...]] | None = None*)

Filter monomial basis using Newton polytope pruning.

Given the combined Newton polytope of an optimization problem's polynomials, this class filters the full monomial basis to those exponent vectors that lie within the convex hull of `2\cdotNewt(f)`, always retaining every exponent in `required_support` (see module docstring for why hull membership alone is not a removal certificate).

**Parameters**

- **num_vars** – Number of variables in the problem.
- **max_degree** – Maximum degree for moment matrix construction.
- **polytope_vertices** – Precomputed vertices of 2cdotcombined Newton polytope. If None, will be computed from polynomials later.
- **required_support** – Exponent tuples that must appear in the returned basis regardless of hull membership (degree-bounded to `max_degree`). Pass the full degree-bounded support until a complete SOS-support certificate is available; pass a smaller certified set once one exists.

**full_basis_size**

Size of unpruned monomial basis.

**pruned_basis_size**

Size after Newton polytope filtering (plus required support).

**reduction_ratio**

Fraction of basis retained (lower = more pruning).

`compute_pruned_basis`(*vars_list=None*)

Compute the pruned monomial basis.

Iterates over all monomials up to `max_degree`, filters those outside the Newton polytope, and then retains every exponent of `required_support` (degree-bounded) regardless of hull membership.

**Parameters**
`vars_list` – Optional variable list (for compatibility).

**Returns**
List of exponent tuples forming the pruned basis; always a superset of `{e in required_support : sum(e) <= max_degree}`.

`moment_matrix_dimension_reduction()` → Dict

Estimate the moment matrix size reduction from Newton pruning.

The moment matrix has dimension RtimesR where R is the basis size. Pruning reduces this to R'timesR', so the reduction factor is (R'/R)^2.

**Returns**
Dict with full_size, pruned_size, matrix_reduction, and savings.

`summary()` → Dict

Return a human-readable summary of the pruning results.

`Irene.newton_polytope.combined_newton_polytope`(*polynomials*, *vars_list=None*)

Compute the Minkowski sum of Newton polytopes of multiple polynomials.

For moment matrix construction, we need 2cdot(Newt(f_0) oplus Newt(g_1) oplus ...), where f_0 is the objective and g_i are constraint polynomials.

**Parameters**

- `polynomials` – List of symbolic polynomial expressions.
- `vars_list` – Shared variable list for consistent exponent ordering.

**Returns**
Combined Newton polytope vertices (deduplicated).

**Return type**
np.ndarray

`Irene.newton_polytope.minkowski_sum`(*polytope_a*, *polytope_b*)

Compute the Minkowski sum of two point sets.

A oplus B = {a + b | a in A, b in B}

**Parameters**

- `polytope_a` – Arrays of shape (n, d) and (m, d).
- `polytope_b` – Arrays of shape (n, d) and (m, d).

**Returns**
All pairwise sums, shape (n*m, d). Deduplicated.

**Return type**
np.ndarray

Irene.newton_polytope.**newton_polytope**(*expr*, *vars_list=None*)

Compute the Newton polytope of a polynomial expression.

The Newton polytope is the convex hull of exponent vectors of all terms with nonzero coefficients in the polynomial.

**Parameters**

- **expr** – A symbolic polynomial expression (SymPy or SymEngine), a scalar, or a SemigroupAlgebra element (converted to SymPy automatically).
- **vars_list** – List of variables to extract exponents for. If None, inferred.

**Returns**
Array of shape (num_terms, num_vars) of exponent vectors.

**Return type**
np.ndarray

**Raises**
ValueError – If the input cannot be parsed as a polynomial. This function never returns a silently degraded (origin-only) polytope; unparseable input is an explicit error instead.

Irene.newton_polytope.**prune_basis_from_polys**(*polynomials*, *num_vars: int*, *max_degree: int*, *required_support: Set[Tuple[int, ...]] | None = None*) → *NewtonPruner*

Convenience function: compute pruned basis from a list of polynomials.

**Parameters**

- **polynomials** – List of symbolic polynomial expressions (objective + constraints). SymPy expressions, scalars, and SemigroupAlgebra elements are all accepted; ring elements are converted to SymPy automatically (see *newton_polytope()*).
- **num_vars** – Number of variables in the problem.
- **max_degree** – Maximum degree for moment matrix construction (usually 2*d).
- **required_support** – Exponents that must be retained regardless of hull membership (see *NewtonPruner*). Default None keeps only hull-interior exponents – callers building relaxation bases should pass the certified safe set.

**Returns**
Configured NewtonPruner with pruned basis computed.

**Raises**
ValueError – If any polynomial cannot be parsed (no silent fallback).

Irene.newton_polytope.**prune_basis_from_problem**(*prog*, *max_degree: int*, *required_support: Set[Tuple[int, ...]] | None = None*) → *NewtonPruner*

Convenience function: compute pruned basis from an OptimizationProblem.

The objective and constraint polynomials may be SymPy expressions or SemigroupAlgebra elements; ring elements are converted to SymPy automatically (see *newton_polytope()*), so problems built via Irene's documented construction path work directly.

**Parameters**

- **prog** – An OptimizationProblem with set_objective() and add_constraint().
- **max_degree** – Maximum degree for moment matrix construction.

- **required_support** – Exponents that must be retained regardless of hull membership (see *NewtonPruner*). Default `None` keeps only hull-interior exponents – callers building relaxation bases should pass the certified safe set.

**Returns**
Configured NewtonPruner with pruned basis computed.

**Raises**
`ValueError` – If any polynomial cannot be parsed (no silent fallback).

`Irene.newton_polytope.scale_polytope(`*polytope*, *factor*`)`
Scale all exponent vectors by an integer factor.

## 28.2 Unified Relaxation API

A single entry point for all relaxation methods (SOS, SONC, SOS+SONC).

### 28.2.1 Design goals

1. **One constructor** – `RelaxationEngine(prog)` wraps any `OptimizationProblem`.
2. **Consistent return type** – every solve call returns a `RelaxResult` with the same attributes (value, status, timing, solver metadata).
3. **Method dispatch** – the user picks `'sos'`, `'sonc'`, or `'sosonc'`; the engine routes to the correct backend class internally.
4. **Backward compatibility** – the old classes (`SDPRelaxations`, etc.) still work; this module is additive, not destructive.

### 28.2.2 Usage

```
>>> from Irene.program import OptimizationProblem
>>> from Irene.relaxation_api import RelaxationEngine, RelaxResult
>>> engine = RelaxationEngine(prog)
>>> result: RelaxResult = engine.solve(method='sos', order=2, solver='cvxopt')
>>> print(result.value)          # lower bound
>>> print(result.status)         # 'optimal' | 'infeasible' | 'error'
```

For a full comparison across all methods in one call:

```
>>> results = engine.compare(order=1)
>>> for m, r in results.items():
...     print(f"{m}: {r.value:.6f}")
```

`class Irene.relaxation_api.RelaxMethod(`*value*`)`
Supported relaxation backends.

`class Irene.relaxation_api.RelaxResult(`*value: float = -inf*, *method: str = ''*, *status: str = 'error'*, *error_code: int = 2*, *runtime: float = 0.0*, *init_time: float | None = None*, *message: str = ''*, *certificate: Any = None*, *solver_info: dict = <factory>*`)`
Uniform result container for all relaxation methods.

**value**
Lower bound on the global minimum (`-inf` on failure).

**Type**
float

**method**
Which relaxation was used.

**Type**
str

**status**
`'optimal'`, `'infeasible'`, or `'error'`.

**Type**
str

**error_code**
0 = success, 1 = infeasible, 2 = solver/computation error.

**Type**
int

**runtime**
Wall-clock seconds for the solve phase (excludes matrix setup).

**Type**
float

**init_time**
Time spent building moment/localizing matrices (if available).

**Type**
Optional[float]

**message**
Human-readable status or error description.

**Type**
str

**certificate**
SOS/SONC polynomial certificate if the backend exposes one.

**Type**
Optional[Any]

**solver_info**
Backend-specific metadata (solver name, iterations, etc.).

**Type**
dict

**property success: bool**
True if the relaxation returned a finite bound without error.

**class Irene.relaxation_api.RelaxationEngine**(*prog:* OptimizationProblem, *order: int = 1*, *solver: str = 'cvxopt'*, *error_bound: float = 1e-10*, *verbosity: int = 1*, *use_local_solve: bool = True*, *config=None*)

Single entry point for SOS, SONC, and combined relaxations.

**Parameters**

- **prog** (OptimizationProblem) – An Irene optimization problem with an objective set via `prog.Minimize(f)` or `prog.set_objective(f)`.
- **order** (*int*) – Relaxation/hierarchy order (default 1).
- **solver** (*str*) – SDP solver name forwarded to the SOS backend (`'cvxopt'`, `'csdp'`, `'sdpa'`, `'dsdp'`).
- **error_bound** (*float*) – Numerical zero tolerance for SONC GP.
- **verbosity** (*int*) – Log level: 0 = silent, 1 = normal, 2+ = verbose.
- **use_local_solve** (*bool*) – Use signomial GP local solver for the SONC portion.
- **config** (RelaxationConfig, *optional*) – Phase 3 reduction pipeline configuration (Newton polytope pruning, border basis, correlative sparsity). Defaults to no reduction.

### Examples

```
>>> engine = RelaxationEngine(prog, order=2, solver='cvxopt')
>>> res = engine.solve('sos')
>>> print(res.value)
```

Compare all methods at once:

```
>>> results = engine.compare()
>>> best_method = max(results, key=lambda m: results[m].value)
```

**compare**(*order: int | None = None*, *solver: str | None = None*) → dict[str, *RelaxResult*]

Run all four relaxation variants and return results keyed by method.

Parameters are the same as `solve()`. Returns a dict mapping method name -> `RelaxResult`.

**solve**(*method: Literal['sos', 'sonc', 'sosonc_sos_first', 'sosonc_sonc_first'] | RelaxMethod = 'sos'*, *order: int | None = None*, *solver: str | None = None*) → *RelaxResult*

Run a single relaxation and return a `RelaxResult`.

**Parameters**

- **method** (*str or* RelaxMethod) – Which relaxation to use. Accepted values:

```
'sos'                  -- pure SOS (Gram-matrix SDP)
'sonc'                 -- pure SONC (signomial GP)
'sosonc_sos_first'     -- two-step: SOS -> SONC residual
'sosonc_sonc_first'    -- two-step: SONC -> SOS residual
```

- **order** (*int*, *optional*) – Override the hierarchy order for this call.
- **solver** (*str*, *optional*) – Override the SDP solver name for this call.

**Return type**
*RelaxResult*

Irene.relaxation_api.**compare_all**(*prog:* OptimizationProblem, ***kwargs: Any*) → dict[str, *RelaxResult*]

Run all four relaxation variants and return results keyed by method.

Convenience wrapper around `RelaxationEngine.compare()`.

Irene.relaxation_api.relax(*prog: OptimizationProblem, method: Literal['sos', 'sonc', 'sosonc_sos_first', 'sosonc_sonc_first'] | RelaxMethod = 'sos', **kwargs: Any*) → *RelaxResult*

Quick one-liner for solving a relaxation.

**Parameters**

- **prog** (OptimizationProblem) – The optimization problem to relax.
- **method** (*str or* RelaxMethod) – Which relaxation backend to use.
- ****kwargs** – Passed through to RelaxationEngine constructor (order, solver, error_bound, verbosity, etc.).

**Return type**
*RelaxResult*

### Examples

```
>>> from Irene.relaxation_api import relax
>>> res = relax(prog, method='sos', order=2)
>>> print(res.value)
```

Sparse moment method for real roots of sparse polynomial systems.

Implements the algorithm of Baumbach & Bender, "The Moment Method for Computing Real Points of Sparse Polynomial Systems" (arXiv:2609.33313): polynomial systems solved on the semigroup ring R[A] = R[x^{alpha_1}, ..., x^{alpha_r}] by exploiting the toric ideal T_A to keep moment matrices small, with a termination guarantee via sparse flat-extension rank conditions (Theorem 3.27 of the paper).

This module currently provides S51.0 – the semigroup-ring setup:

- ToricSetup – exponent set A, ambient dimension, toric generators
- toric_ideal(A) – a Groebner basis G_A of T_A = ker(psi_A) in R[z]
- rho_A – max total degree of the generators (0 if T_A < 0>)
- deg_A(f, A) – the A-graduation: min{s : f in R[A]^s}

LATER SUBTASKS (Vikunja #51): S51.2 Theorem 3.27 rank conditions and termination loop, S51.3 sparse border basis, S51.4 sparse Stickelberger eigenvalue recovery, S51.5 Algorithm 2 (randomized single root).

S51.1 provides: semigroup_level_map, moment_indices (R[A]^t enumeration), prolongations (H_t, eq. (9)) and SparseMomentSDP – the K_t spectrahedron of eq. (10) as a standard-form SDP on CvxpySDPSolver.

## 28.3 NOTATION (paper sections referenced)

A = {alpha_1, ..., alpha_r} subset N^m finite set of exponents R[A]^s = < x^{sum_{i<=s} alpha_i} > s-truncation (Def 2.4) psi_A : R[z] -> R[x], z_i |-> x^{alpha_i} the semigroup map (1) T_A := ker(psi_A), G_A a generating set, rho_A := max{deg(g) : g in G_A}, rho_A = 0 if T_A < 0> (3)

class Irene.sparse_moment.Algorithm1Result(*converged: bool = False, no_real_roots: bool = False, t: int = 0, iterations: int = 0, rank_test: RankTestResult | None = None, kernel_polys: List[Expr] = <factory>, quotient_basis: Tuple[int, ...]]=<factory>, lambda_vec: ndarray | None = None, moment_matrix: ndarray | None = None, infeasible_at_t: int | None = None*)

Outcome of the Algorithm-1 termination loop (paper, end of Section 3.4).

**converged**

V_{R[A]}(I) is empty – a Stengle certificate), or when `max_iterations` was exceeded without rank stabilization.

**Type**
False when K_t was infeasible at some t (Lemma 3.29(i): then

**no_real_roots**

**Type**
True iff the loop stopped on an infeasible K_t.

**t**

**Type**
truncation degree at which the loop stopped.

**iterations**

**Type**
number of K_t SDPs solved (t increments by 1 each iteration).

**rank_test**

**Type**
final RankTestResult (None when not converged).

**kernel_polys**

ker(M_{s_star}(Lambda)) – a generating set for J = <ker M_s(Lambda)>. When converged and Lambda generic, J is exactly the real radical. Variable names follow the `deg_A` convention (sorted free symbols of the input system). Coefficients are snapped to 0/integers when close.

**Type**
sympy expressions p = sum v_i x^{beta_i} spanning

**quotient_basis**

M_{s_star - 1}(Lambda); by Corollary 3.19 their residue classes form a basis of R[A]/J (Remark 3.28: the greedy-in-order choice gives a connected-to-one border basis). len(quotient_basis) = n_real when converged and Lambda generic.

**Type**
exponent tuples indexing a monomial column basis B of

**lambda_vec, moment_matrix**

(diagnostics; None until convergence).

**Type**
the optimizer and its flat matrix M_{s_star}

**infeasible_at_t**

**Type**
t at which K_t was reported empty (None otherwise).

### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> r = solve_algorithm1([x**3 - x], A=[(1,)])
```

(continues on next page)

(continued from previous page)

```
>>> (r.converged, r.t, sorted(str(p) for p in r.kernel_polys))
(True, 6, ['x**3 - x'])
```

### Notes

The constant-objective SDP returns a generic element of K_t with interior-point solvers (Lemma 3.23 / eq. (11): all relint points share the same kernel space N_t); first-order solvers may land on the boundary, in which case pass `solver='CLARABEL'` explicitly.

**class** Irene.sparse_moment.**RankTestResult**(*converged: bool = False*, *condition: str | None = None*, *s_star: int | None = None*, *D: int = 0*, *d: int = 1*, *rho_A: int = 0*, *ranks: Dict[int, int]=<factory>*, *n_real: int | None = None*, *t: int = 0*, *tol: float = 1e-06*)

Outcome of the Theorem 3.27 convergence test at one truncation t.

**converged**

**Type**
True iff condition (i) or (ii) holds for some admissible s.

**condition**

cases carry different proof obligations in Thm 3.27, so the distinction is kept even though both imply a flat extension at level s.

**Type**
'flat' (case i), 'interpolation' (case ii), or None. The two

**s_star**

**Type**
smallest admissible s at which convergence was detected.

**D**

**Type**
max_i deg_A(f_i) (0 when I < 0>); d = max(1, ceil(D/2)); rho_A from T_A.

**ranks**

**Type**
{k: rank(M_k(Lambda))} for all 0 <= k <= floor(t/2).

**n_real**

converged and Lambda generic (Lemma 3.14 / Thm 3.27).

**Type**
rank(M_{s_star}(Lambda)) – the number of real toric roots when

**t, tol**

**Type**
truncation degree and eigenvalue threshold used.

### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> sdp = SparseMomentSDP([x**3 - x], [(1,)], 6)
>>> res = sdp.solve()
>>> rt = test_theorem_327(res.lambda_vec, [x**3 - x], [(1,)], 6,
...                       sdp.index, sdp.levels, sdp.monoms_t)
>>> (rt.converged, rt.condition, rt.s_star, rt.n_real)
(True, 'flat', 3, 3)
```

class Irene.sparse_moment.**SparseBorderBasis**(*lam_vec*, *basis: Sequence[Sequence[int]]*, *A*, *t: int*, *s_star: int*, *monoms: List[Tuple[int, ...]]*, *index: Dict[Tuple[int, ...], int]*, *levels: dict*, *tol: float = 1e-06*)

Sparse border basis B of the quotient algebra $R[A]/J$, J = $\langle \ker M_{s^*}(\Lambda) \rangle$, built from the moment data of a converged Algorithm-1 run (Theorem 3.27 + Corollary 3.19 / Remark 3.28).

B is taken to be the quotient basis of the *Algorithm1Result* – the greedy-in-deg_A-order column basis of $M_{s^*-1}(\Lambda)$ – which Remark 3.28 guarantees connected-to-one (Def A.2); this class *verifies* that property by BFS rather than assuming it.

For every exponent e with $\deg_A(e) \le s^*$ the normal form $[x^e] = \sum_j d_j b_j$ (i.e. $x^e - \sum_j d_j b_j \in J$) is recovered from the moment equalities of flat extension: under the rank condition, Lambda admits a representing measure with n_real atoms on V_{R[A]}(J), hence for all u with $\deg_A(u) + \deg_A(e) \le t$

Lambda(x^{u+e}) = sum_j d_j Lambda(x^{u+b_j}),

an exactly consistent linear system solved in least squares (residuals are exposed for diagnostics). The multiplication table of each semigroup generator x^{a_i} is then the matrix whose k-th column is [x^{a_i} b_k] in B; these tables are self-adjoint on V, commute, and respect every toric relation (Def A.3) – verified numerically by *check_relations()* (Appendix A.1 normal-form criterion).

**Parameters**

- **lam_vec** (*moment vector Lambda(x^alpha) over R[A]^t (position-ordered), t >= 2*s_star.*)
- **basis** (*exponent tuples of B, with basis[0] == (0, ..., 0); normally*) – Algorithm1Result.quotient_basis.
- **A** (*exponent set (sequence or ToricSetup).*)
- **t** (*truncation degree; levels must cover R[A]^{s_star}.*)
- **s_star** (*level at which flat extension was detected.*)
- **monoms** (basis data from moment_indices(A, t).)
- **index** (basis data from moment_indices(A, t).)
- **levels** (basis data from moment_indices(A, t).)
- **tol** (*least-squares / residual threshold (default 1e-6).*)

**basis, r_**

**Type**
the border basis and its cardinality (= n_real on convergence).

**tables**

**Type**
list of r_ x r_ arrays; column k = [x^{a_i} b_k] in B.

`normal_forms[e]()`

**Type**
coefficient vector d over R[A]^{s_star}, x^e - sum d_j b_j in J.

`nf_residuals[e]()`

**Type**
lstsq residual of the moment system for e (0 on exact flatness).

`connected_to_one`

**Type**
Def A.2 flag, verified by BFS from 1 inside B.

### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> r = solve_algorithm1([x**3 - x], A=[(1,)])
>>> monoms, index, levels = moment_indices([(1,)], r.t)
>>> bb = SparseBorderBasis(r.lambda_vec, r.quotient_basis, [(1,)], r.t,
...                        r.rank_test.s_star, monoms, index, levels)
>>> (bb.r_, tuple(bb.basis), bb.connected_to_one)
(3, ((0,), (1,), (2,)), True)
```

### Notes

Semigroup-aware twin of *Irene.border_basis.BorderBasis* (dense ring): here the extra constraint is Def A.3 – every g in G_A must annihilate V under M(g), which `check_relations` tests.

`check_relations()` → float

Max over g in G_A of $\|g(M_1, \ldots, M_r)\|_2$ – the Def A.3 / Appendix A.1 compatibility residual (0 when every toric relation holds on V).

#### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> r = solve_algorithm1([x**4 - 5*x**2 + 4], A=[(2, 0), (3, 0)])
>>> monoms, index, levels = moment_indices([(2, 0), (3, 0)], r.t)
>>> bb = SparseBorderBasis(r.lambda_vec, r.quotient_basis, [(2, 0), (3, 0)],
...                        r.t, r.rank_test.s_star, monoms, index, levels)
>>> bool(bb.check_relations() < 1e-6)      # z1^3 = z2^2 holds on V
True
```

`normal_form(e)`

Coefficient vector d over R[A]^{s_star}: x^e - sum_j d[b_j] b_j in J.

`class Irene.sparse_moment.SparseMomentResult(`*status: str, lambda_vec: ndarray | None = None, M_s: ndarray | None = None, rank_Ms: int | None = None, rank_Ms_minus_1: int | None = None, n_monomials_t: int = 0, n_monomials_s: int = 0, h_count: int = 0, objective_value: float | None = None, wall_time: float = 0.0, solver_info: dict = <factory>*`)`

Structured output of `SparseMomentSDP.solve`.

**status**

Infeasible means K_t = empty, hence by Lemma 3.29(i) the system has no real toric solution (1 is an SOS modulo I – a Stengle certificate).

**Type**
'Optimal' | 'Infeasible' | ... (solver verdict).

**lambda_vec**

**Type**
moment vector Lambda(x^alpha), alpha in R[*A*]_t (position-ordered).

**M_s**

**Type**
the sparse truncated moment matrix M_{floor(t/2)}(Lambda).

**rank_Ms, rank_Ms_minus_1**

submatrix on R[A]^{s-1} – the flat-extension pair (Theorem 3.27).

**Type**
*ranks* of M_s and its top-left principal

**n_monomials_t**

**Type**
**|R[A]_t|**, number of SDP variables.

**n_monomials_s**

benchmark vs the dense monomial count binom(n+s-1, s)).

**Type**
**|R[A]_{floor(t/2)}|**, moment-matrix dimension (the sparsity

**h_count**

**Type**
number of prolongation constraints in H_t after deduplication.

**objective_value**

**Type**
min c^T Lambda over K_t (constant 1 for Algorithm 1).

**wall_time**

**Type**
seconds spent inside CvxpySDPSolver.solve().

**solver_info**

**Type**
the legacy `Info` dict from CvxpySDPSolver.

`class Irene.sparse_moment.SparseMomentSDP(`*f_system: Sequence[Basic]*, *A: Sequence[Sequence[int]]*, *t: int*, *objective: str | Sequence[float] = 'constant'*, *solver=None*`)`

The K_t spectrahedron of eq. (10), built on `CvxpySDPSolver`.

$$K_t = \{\Lambda \in R[A]_t^* : \Lambda(1) = 1,\ M_{\lfloor t/2 \rfloor}(\Lambda) \succeq 0,\ \Lambda(f) = 0\ \forall f \in H_t\}.$$

One SDP variable per monomial of R[A]_t (enumerated by A-degree, not total degree – this is exactly where the sparsity pays off); one PSD block for M_{floor(t/2)}; affine equalities for normalization and prolongations.

**Parameters**

- **f_system** (*generators f_i of I in R[A].*)
- **A** (exponent set, or a `ToricSetup`.)
- **t** (*truncation degree >= max_i deg_A(f_i).*)
- **objective** (*'constant' (Algorithm 1: min Lambda(1) -- feasibility probe)*) – or an array-like c over R[A]_t (one entry per monomial position, as in `moment_indices`); a random Gaussian choice realizes Algorithm 2.
- **solver** (*SDP backend name passed to CvxpySDPSolver.*)

### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> sdp = SparseMomentSDP([x*(x-1)*(x+1)], [(1,)], 4)   # I < x^3-x >
>>> res = sdp.solve()
>>> bool(abs(res.lambda_vec[sdp.index[(0,)]] - 1.0) < 1e-6)   # Lambda(1) = 1
True
```

**solve**() → *SparseMomentResult*

Solve min b^T Lambda over K_t and return structured moment data.

**class** Irene.sparse_moment.**SparseRootsResult**(*roots: Dict[str, float]]=<factory>, bwe: List[float] = <factory>, bwe_components: List[List[float]] = <factory>, eigenvalues: List[float] = <factory>, coefficients: Tuple[int, ...] | None=None, trials: int = 0, kernel_polys: List[Expr] = <factory>, connected_to_one: bool = False*)

Output of the sparse Stickelberger eigenvalue method (Appendix A.2).

**roots**

z_i = x^{a_i} the semigroup generators (values m^{a_i}).

**Type**
one dict per real toric root, {'z1': v1, ..., 'z_r': vr} with

**bwe**

(sum_alpha |c_{alpha,f} Lambda_m(x^alpha)| + 1). The paper reports log10(BWE) in [-7, -3] for its random systems.

**Type**
backward error per root, eq. (12): (1/m) sum_i |Lambda_m(f_i)| /

**bwe_components**

**Type**
per-root list of the m summands above (diagnostics).

**eigenvalues**

Stickelberger functional f = c_1 x^{a_1} + … + c_r x^{a_r}.

**Type**
the **r_** distinct real values Lambda_m(f) of the successful

**coefficients**

**Type**
integer coefficient vector c of that functional.

**trials**

**Type**
number of random functionals tried (genericity, Theorem A.4).

**kernel_polys**

**Type**
generators of J = <ker M_{s_star}(Lambda)> (Algorithm 1 step 5).

**connected_to_one**

**Type**
Def A.2 flag from the border basis used.

### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> r = solve_algorithm1([x**3 - x], A=[(1,)])
>>> out = recover_all_sparse_roots(r, [(1,)], [x**3 - x])
>>> sorted(round(v['z1'], 6) for v in out.roots)
[-1.0, 0.0, 1.0]
```

**class** Irene.sparse_moment.**ToricSetup**(*A: Tuple[Tuple[int, …], …]*, *r: int*, *z: Tuple[Symbol, …]*, *G_A: Tuple[Expr, …]*, *rho_A: int*)

Semigroup-ring data for the sparse moment method.

**A**

**Type**
tuple of exponent vectors (validated, deduplicated)

**r**

**Type**
number of generators = len($A$); ambient dimension m = len($A$[0])

**z**

**Type**
the generator symbols z_1..z_r of R[$z$] ~= R[$A$] (isomorphic)

**G_A**

**Type**
Groebner basis of the toric ideal T_A in R[$z$] (empty if none)

`rho_A`

**Type**
max total degree over G_A, or 0 when T_A = <0> (paper eq. 3)

`classmethod from_exponents`(*A: Sequence[Sequence[int]]*) → *ToricSetup*

Build the setup for an exponent set A subset of N^m.

### Example

```
>>> import sympy as sp
>>> ts = ToricSetup.from_exponents([(2, 0), (3, 0)])   # R[x^2, x^3]
>>> ts.rho_A
3
>>> [sp.simplify(g) for g in ts.G_A]
[z1**3 - z2**2]
```

`property is_full_ring: bool`

True iff A is a monomial basis of the full polynomial ring (T_A < 0>); then rho_A = 0 and no toric reduction applies.

`Irene.sparse_moment.deg_A`(*f: Expr*, *A: Sequence[Sequence[int]]*) → int

A-graduation of f in R[A] (paper Section 2):

deg_A(f) = min{ s in N : f in R[A]^s },

where R[A]^s is spanned by the monomials x^{alpha_{i_1} + ... + alpha_{i_s}} with at most s summands. Subadditive: deg_A(fg) <= deg_A(f)+deg_A(g), deg_A(f+g) <= max{deg_A(f), deg_A(g)}.

**Parameters**

- **f** (*SymPy polynomial (or 0); its free symbols are taken as the ambient*) – variables x_1..x_m in sorted-name order, so len(free_symbols) must equal the ambient dimension of A. All monomials must lie in R[A].
- **A** (exponent set; may also be a `ToricSetup`.)

**Raises**

- **ValueError if f contains monomials outside R[A] or its number of free** –
- **symbols does not match len(A[0]).** –

### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> deg_A(x**7, [(2, 0), (3, 0)])   # x^7 in R[x^2, x^3]
3
```

`Irene.sparse_moment.moment_indices`(*A: Sequence[Sequence[int]]*, *t: int*)

Enumerate the truncated ring $R[A]^t$.

Returns `(monoms, index, levels)` where `monoms` is the sorted list of exponent tuples in R[A]^t (by A-degree, then lexicographic), `index` maps each exponent to its position in that list, and `levels`

gives `deg_A(x^beta)` for every enumerated beta. The length of `monoms` is the number of SDP variables in K_t; the sub-list with level <= floor(t/2) indexes the rows/columns of the moment matrix M_{floor(t/2)}.

### Example

```
>>> from Irene.sparse_moment import moment_indices
>>> monoms, index, levels = moment_indices([(1,)], 4)   # R[x], t = 4
>>> len(monoms), index[(0,)]
(5, 0)
```

`Irene.sparse_moment.prolongations`(*f_system: Sequence[Basic]*, *A: Sequence[Sequence[int]]*, *t: int*, *index: Dict[Tuple[int, ...], int]*)

The set $H_t$ of eq. (9): all products `x^beta f_i` with `deg_A(x^beta) <= t - deg_A(f_i)`, expressed in the moment basis.

Returns one dict per distinct polynomial of H_t, each mapping monomial *positions* (in `index`) to float coefficients – i.e. the row vector of the linear constraint $\Lambda(h) = 0$ over R[A]_t.

**Parameters**

- **f_system** (*polynomials f_i in R[A] (sympy expressions).*)
- **A** (exponent set, or a `ToricSetup`.)
- **t** (*truncation degree; must be >= deg_A(f_i) for every i.*)
- **index** (monomial -> position map from `moment_indices(A, t)`.)

**Raises**

**ValueError if some f_i is not in R[A] or has A-degree > t.** –

### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> monoms, index, levels = moment_indices([(1,)], 3)
>>> H = prolongations([x**2 - 1], [(1,)], 3, index)   # {x^2-1, x^3-x}
>>> sorted(H[0].items()), sorted(H[1].items())
([(0, -1.0), (2, 1.0)], [(1, -1.0), (3, 1.0)])
```

`Irene.sparse_moment.recover_all_sparse_roots`(*result: Algorithm1Result*, *A*, *f_system: Sequence | None = None*, ** (Keyword-only parameters separator (PEP 3102))*, *seed: int = 2026*, *max_trials: int = 50*, *tol: float = 1e-08*) → *SparseRootsResult*

Recover all points of $V_{R[A]}(I)$ from a converged Algorithm-1 run.

Implements the sparse Stickelberger eigenvalue method (paper Appendix A.2, Theorem A.4): pick random integer coefficients c = (c_1,...,c_r), form the multiplication matrix of $f = \sum_i c_i x^{a_i}$ on the border basis B, take its left eigenvectors w normalized by their 1-coordinate (eq. 14: $w \propto (\Lambda_m(x^{b_1}), \ldots, \Lambda_m(x^{b_r}))$), and read off each generator value – directly from the W-row when $x^{a_i} \in B$, else via the normal form of x^{a_i} (eq. 15). A trial is generic iff it yields exactly **r_** distinct real eigenvalues; random integer coefficients are distinct w.p. 1, so a single retry suffices with overwhelming probability.

**Parameters**

- **result** (CONVERGED `Algorithm1Result` (lambda_vec and quotient_basis set).)

- **A** (*the exponent set used for the run ((sequence of tuples) or* ToricSetup*);*) – must match what produced **result**.
- **f_system** (*generators of I, used only for the BWE diagnostic (eq. 12);*) – defaults to [] (BWE = 0).

:param Returns a *SparseRootsResult*.:

Irene.sparse_moment.**semigroup_level_map**(*A: Sequence[Sequence[int]]*, *max_level: int*) → dict

Minimal summand count for every exponent reachable within **max_level**.

Returns the map $\beta \mapsto d_A(\beta)$ for all $\beta \in \bigcup_{s \le max_level} A^s$, where $d_A(x^\beta) = min\{s : x^\beta \in R[A]^s\}$ is the minimal number of summands of $A$ adding up to $\beta$. BFS over the exponent lattice with steps in **A**; exponents whose total degree already exceeds $max_level \cdot max|a_i|$ are pruned (all relevant paths stay below that bound).

**Parameters**

- **A** (sequence of exponent vectors in N^m, or a **ToricSetup**.)
- **max_level** (*int >= 0; the largest A-degree to enumerate.*)

Irene.sparse_moment.**solve_algorithm1**(*f_system: Sequence[Basic]*, *A: Sequence[Sequence[int]] | None = None*, *t0: int | None = None*, *tol: float = 1e-06*, *max_iterations: int = 8*, *solver=None*) → *Algorithm1Result*

Algorithm 1 (all real roots): the K_t SDP + Theorem 3.27 termination loop.

Solves **min Lambda(1)** over $K_t$ (eq. (10)) at truncation t, tests the rank conditions of Theorem 3.27, and on failure increments $t \leftarrow t + 1$ and re-solves (paper step 4). Stopping cases:

- K_t infeasible -> by Lemma 3.29(i) there is no real toric solution;
- rank stabilization at some admissible s -> J = <ker M_s(Lambda)> is the real radical for generic Lambda (Thm 3.27);
- **max_iterations** exceeded -> RuntimeError with the last rank profile.

**Parameters**

- **f_system** (*generators of I, each in R[A] (sympy expressions).*)
- **A** (*exponent set; optional only for single-variable systems, where it is*) – inferred as [(1,)] from the free symbols.
- **t0** (starting truncation degree; defaults to **2*max{D, rho_A}** per) – Algorithm 1 step 1 (with a floor of max(4, D) so K_t contains H_t).
- **tol** (*eigenvalue threshold for numerical ranks (paper practice: 1e-3. .1e-7).*)
- **max_iterations** (*guard against Lemma 3.29's "sufficiently large t".*)
- **solver** (SDP backend name passed to **CvxpySDPSolver**; interior-point) – methods are preferred so the optimizer lies in relint(K_t) (Lemma 3.23).

:param Returns an *Algorithm1Result*.:

### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> r = solve_algorithm1([x**3 - x], A=[(1,)])
>>> (r.converged, r.t, sorted(str(p) for p in r.kernel_polys))
(True, 6, ['x**3 - x'])
```

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> r = solve_algorithm1([x**2 + 1], A=[(1,)])
>>> (r.converged, r.no_real_roots)
(False, True)
```

Irene.sparse_moment.**solve_sparse_real_roots**(*f_system: Sequence[Basic]*, *A: Sequence[Sequence[int]] | None = None*, *t0: int | None = None*, *tol: float = 1e-06*, *max_iterations: int = 8*, *seed: int = 2026*, *solver=None*) → *SparseRootsResult*

Top-level pipeline: Algorithm 1 (S51.2) + sparse border basis (S51.3) + Stickelberger recovery (S51.4).

Returns a *SparseRootsResult* with one dict per real toric root of the system in the semigroup generators z_i = x^{a_i}, backward-error diagnostics (eq. 12), and the J-generators from ker M_{s_star}. When K_t is infeasible (Lemma 3.29(i): no real toric solution) an empty result is returned with `no_real_roots` recorded on it via `roots == []` and a non-empty kernel list left untouched.

### Example

```
>>> import sympy as sp
>>> x = sp.Symbol('x')
>>> out = solve_sparse_real_roots([x*(x-1)*(x+1)], A=[(1,)])
>>> sorted(round(v['z1'], 6) for v in out.roots)
[-1.0, 0.0, 1.0]
```

Irene.sparse_moment.**test_theorem_327**(*lam_vec*, *f_system: Sequence[Basic]*, *A: Sequence[Sequence[int]]*, *t: int*, *index: Dict[Tuple[int, ...], int]*, *levels: dict*, *monoms: List[Tuple[int, ...]]*, *tol: float = 1e-06*) → *RankTestResult*

Theorem 3.27 convergence test for an optimizer $\Lambda \in K_t$.

With $D = \max_i \deg_A(f_i)$ and $d = \lceil D/2 \rceil$, check for admissible s (case i: $\max\{D, \rho_A\} \le s \le \lfloor t/2 \rfloor$; case ii: $\max\{d, \rho_A\} \le s \le \lfloor t/2 \rfloor$):

(i) rank(M_s(Lambda)) = rank(M_{s-1}(Lambda)), or

(ii) rank(M_s(Lambda)) = rank(M_{s-d}(Lambda)).

Either condition implies :math:`sqrt R I subseteq J := langle ker M_s(Lambda)

angle`; for generic Lambda, equality (paper Thm 3.27).

The full rank profile is always returned so a non-convergent truncation can still report which ranks were computed. Note: for small t the admissible range may be empty – then `converged` is False and the caller must increment t (Algorithm 1 step 4).

lam_vec : moment vector Lambda(x^alpha) over R[A]_t (position-ordered). f_system : generators of I (sympy expressions in R[A]). A : exponent set, or a ToricSetup. t : the truncation degree K_t was solved at; must be >= 2*max admissible s. index, levels, monoms : the basis data from `moment_indices(A, t)`. tol : absolute eigenvalue threshold for numerical ranks.

Returns a *RankTestResult*.

Irene.sparse_moment.toric_ideal(*A: Sequence[Sequence[int]]*) → List[Expr]

Compute a Groebner basis G_A of the toric ideal T_A = ker(psi_A).

Uses standard elimination (Sturmfels, "Grobner Bases and Convex Polytopes", Theorem 2.3): introduce u_1..u_m for x_1..x_m and z_1..z_r for the r monomials x^{alpha_i}, then eliminate u from

< z_i - prod_j u_j^{alpha_{i,j}} : i = 1..r > (lex, u first)

The ideal G_A in R[z] alone is exactly T_A.

**Parameters**

**A** (*sequence of exponent vectors in N^m*)

**Returns**

- list of SymPy expressions in z_1..z_r generating T_A; [] iff the
- *monomials are algebraically independent (T_A = <0>).*

symbolic_engine.py – user-selectable symbolic backend (SymEngine or SymPy).

**Design:**

- The default backend is SymEngine (C++ backend) for expansion, matrix and symbol creation; operations SymEngine does not implement (Groebner basis, Poly.as_dict(), reduced(), lambdify, DomainMatrix/PolyMatrix, ...) always run on SymPy, with transparent to_sympy() casting.
- Users can select the backend at runtime:

  from Irene.symbolic_engine import engine, set_symbolic_backend set_symbolic_backend('sympy') # or 'symengine' / 'auto'

  or via the environment variable IRENE_SYMBOLIC_BACKEND:

  IRENE_SYMBOLIC_BACKEND=sympy python3 my_script.py IRENE_SYMBOLIC_BACKEND=symengine python3 my_script.py

  Accepted values: 'symengine' (default when installed), 'sympy', 'auto' (prefer SymEngine when available, else SymPy).
- If SymEngine is not installed, the engine automatically falls back to SymPy and *engine.backend* reports 'sympy'.

**Usage in existing modules (e.g., relaxations.py):**

# OLD: from sympy import groebner, Poly, Matrix, expand, lambdify, Symbol, QQ, zeros, reduced, sympify # NEW: from Irene.symbolic_engine import engine x = engine.Symbol('x') g = engine.groebner([f1, f2], x) # always SymPy (SymEngine lacks Groebner) p = engine.Poly(expr, x) # always SymPy (SymEngine lacks full Poly API) M = engine.Matrix([[x**2, 1], [0, x]]) # DenseMatrix (SymEngine) or sp.Matrix (SymPy) result = engine.expand(p * q) # backend-native expand

class Irene.symbolic_engine.SymbolicEngine(*use_symengine=None*)

Unified symbolic computation engine.

Primary path: SymEngine (C++ backend) when *use_symengine* is True. Fallback: SymPy for operations SymEngine doesn't support.

The backend can be selected at construction, at runtime via *set_backend()*, or globally via the IRENE_SYMBOLIC_BACKEND environment variable (read once at import time for the default engine).

**use_symengine**

bool – True uses SymEngine primary path (default True)

**fallback_log**

list – records which calls fell back to SymPy

**Abs**(*expr*)

Absolute value in the selected backend.

**DomainMatrix**(*matrix*, *domain*)

Domain matrix – SymPy only.

**Function**(*name*)

Symbolic function – SymPy only (SymEngine lacks this).

**Matrix**(**args*, ***kwargs*)

Construct a matrix in the selected backend.

SymEngine DenseMatrix when the backend is SymEngine and entries are compatible; SymPy Matrix otherwise.

**Poly**(*expr*, **gens*)

Construct polynomial. SymPy fallback (SymEngine lacks full Poly API).

**PolyMatrix**(*matrix*, **gens*)

Polynomial matrix – SymPy only.

**property QQ**

Rational field – SymPy only.

**Symbol**(*name*, ***kwargs*)

Create a symbolic variable in the selected backend.

**static available_backends**()

Backends that can be selected on this installation.

**clear_fallback_log**()

Clear the fallback log.

**expand**(*expr*)

Expand a polynomial expression using the selected backend.

**fallback_stats**()

Return dict of how many times each operation fell back to SymPy.

**get_backend**()

Current backend name: 'symengine' or 'sympy'.

**groebner**(*polys*, **gens*, *order='lex'*)

Groebner basis – always SymPy (SymEngine doesn't support this).

**lambdify**(*symbols*, *expressions*, *modules='numpy'*)

Compile to numerical function – SymPy lambdify (SymEngine lacks this).

**latex**(*expr*)

LaTeX string representation (SymPy printer for both backends).

**reduced**(*expr*, *groebner_basis*)

Reduce expression modulo Groebner basis – always SymPy.

**set_backend**(*backend*)

Select the symbolic backend.

**Parameters**
**backend** – 'symengine', 'sympy', or 'auto' (prefer installed).

**Returns**
self (chainable).

**Raises**

- **ValueError** – unknown backend name.
- **ImportError** – 'symengine' requested but not installed.

**sqrt**(*expr*)

Square root in the selected backend.

**symbols**(*names*, ***kwargs*)

Create multiple symbolic variables in the selected backend.

**sympify**(*obj*)

Convert Python/SymEngine object to symbolic expression.

**zeros**(*rows*, *cols*)

Zero matrix in the selected backend.

Irene.symbolic_engine.**fallback_to_sympy**(*func*)

Decorator: run func with SymEngine; on failure, cast inputs->SymPy->run native->cast result.

Irene.symbolic_engine.**get_symbolic_backend**()

Module-level helper: name of the default engine's backend.

Irene.symbolic_engine.**set_symbolic_backend**(*backend*)

Module-level helper: select the backend of the default engine.

Irene.symbolic_engine.**to_symengine**(*obj*)

Cast a SymPy object (or list/matrix of them) to SymEngine.

Returns the object unchanged when SymEngine is unavailable or the object cannot be converted.

Irene.symbolic_engine.**to_sympy**(*obj*)

Cast a SymEngine object (or list/matrix of them) to SymPy.

CVXPY-based solver abstraction layer for Irene SDP relaxations.

Replaces text-based solver writers (SDPA/CSDP dat files) with a DCP-compliant formulation that routes to Clarabel, SCS, or CVXOPT through CVXPY's unified API.

The SDP is formulated in primal standard form:

min b^T x s.t. sum_i A_{i,j} * x[i] - C_j >= 0 for j = 1..k (PSD blocks)

where each block j has dimension BlockStruct[j].

**class** Irene.cvxpy_solver.**CvxpySDPSolver**(*solver: str | None = None*)

CVXPY-based SDP solver with the same API as Irene.sdp.sdp.

Usage mirrors the legacy interface:

```
solver = CvxpySDPSolver(solver='CLARABEL')
solver.SetObjective(b)
solver.AddConstraintBlock(A_i)   # for each variable x_i
solver.AddConstantBlock(C_j)     # constant PSD blocks
result = solver.solve()
```

**Parameters**

**solver** (*str, optional*) – Solver backend. One of `'CLARABEL'`, `'SCS'`, `'CVXOPT'`. Defaults to the first available solver from that list.

`AddConstantBlock(`*C*`)`

Set constant PSD blocks.

**Parameters**

**C** (*list of ndarray, length k*) – `C[j]` is the constant matrix (d_j x d_j) for block j.

`AddConstraintBlock(`*A*`)`

Add constraint matrices for one primal variable.

**Parameters**

**A** (*list of ndarray, length k*) – `A[j]` is the coefficient matrix (d_j x d_j) for block j.

`AddEquality(`*a*, *b*`)`

Add one affine equality constraint `a^T x = b`.

Needed by moment-spectrahedron formulations (e.g. the K_t set of Baumbach–Bender: normalization `Lambda(1) = 1` and prolongation constraints `Lambda(f) = 0`), which the PSD blocks alone do not express.

**Parameters**

- **a** (*array-like of shape (m,)*) – Coefficient vector; `m` must match the objective length set by `SetObjective`.
- **b** (*float*) – Right-hand side.

`CvxOpt()`

Legacy compatibility – delegates to `solve()`.

`Option(`*param: str*, *val*`)`

Set a solver option.

**Parameters**

- **param** (*str*) – Option name (solver-specific).
- **val** (*any*) – Option value.

`SetObjective(`*b*`)`

Set objective coefficient vector `b`.

**Parameters**

**b** (*array-like of shape (m,)*) – Coefficients of the linear objective `min b^T x`.

`solve()` → *SDPResult*

Solve the SDP and return structured results.

**Returns**
Container with status, objectives, primal/dual variables, etc.

**Return type**
*SDPResult*

`class Irene.cvxpy_solver.SDPResult`

Container for SDP solution data.

Attributes mirror the legacy `sdp.Info` dictionary keys so that downstream code in `relaxations.py` can consume results without change.

`to_info_dict()` → dict

Return legacy-compatible Info dictionary.

`Irene.cvxpy_solver.available_solvers()` → list[str]

Return list of SDP-capable solvers currently installed.

Differential Semidefinite Programming (DSDP) relaxations.

Extends the Lasserre SDP hierarchy to handle optimization problems where polynomial terms include functions satisfying algebraic differential equations (ADEs). Uses Ritt-Woodin differential algebra to encode ADE constraints as algebraic relations in the moment hierarchy.

**Key features:**

- ADE lift: encode transcendental functions via algebraic relations (e.g., y*z=1 for exp, y' = y for dy/dx = y)
- Differential KKT: inject stationarity conditions derived via Leibniz rule
- Mean polynomial certificates: M_{q,p}(X,w) nonnegativity via SDP
- Archimedean (boxing) constraints for convergence guarantees

**Integration:**

- Inherits SDPRelaxations (sympy-based moment hierarchy)
- Compatible with OptimizationProblem via from_problem()
- Works with SemigroupAlgebra derivation support

`class Irene.dsdp.DSDPKKTRelaxation(`*gens*, *relations=()*, *name='DSDPKKTRlx'*, *diff_map=None*, ***kwargs*`)`

DSDP relaxation with differential KKT condition injection.

Injects stationarity conditions derived from differentiating the Lagrangian, which can tighten bounds significantly (equivalent to one higher Lasserre order).

`solve_kkt(`*order=None*`)`

Solve with KKT stationarity injection.

**Parameters**
**order** – Relaxation order.

**Returns**
Tightened lower bound.

`class Irene.dsdp.DSDPMeanRelaxation(`*gens*, *weights*, *q=1*, *p=0*, *relations=()*, *name='DSDPMeanRlx'*, ***kwargs*`)`

Specialized DSDP relaxation using mean polynomial certificates.

Focuses on the $M_{q,p}(X, w)$ nonnegativity certificate as the primary relaxation mechanism, with ADE relations as secondary constraints.

The mean polynomial cone $\mathcal{M}_{n,2d}$ contains both SOS and SONC cones, providing a potentially tighter relaxation.

**construct_mean_moment_matrix()**

Construct the moment matrix for the mean polynomial certificate.

**Returns**
Block-diagonal moment matrix encoding M_{q,p} nonnegativity.

**solve_mean**(*order=None*)

Solve using mean polynomial relaxation.

**Parameters**
**order** – Relaxation order.

**Returns**
Lower bound from mean relaxation.

**class Irene.dsdp.DSDPRelaxations**(*gens*, *relations=()*, *name='DSDPRlx'*, ***kwargs*)

Differential SDP relaxation framework.

Extends `SDPRelaxations` to handle problems with ADE-constrained variables. The ADE is encoded as algebraic relations in the Groebner basis, and differential KKT conditions are injected as additional linear moment constraints.

**The relaxation constructs a moment hierarchy where:**

1. ADE relations are enforced as equality constraints on moment variables
2. Differential KKT conditions tighten the relaxation (degree-lift equivalent)
3. Mean polynomial certificates $M_{q,p}(X, w)$ provide nonnegativity proofs

### Example

```
>>> from sympy import symbols, exp
>>> from Irene import DSDPRelaxations
>>> x, y, z = symbols('x y z')
>>> # Minimize e^x - x^2 via ADE lift y = e^x, z = e^{-x}, y*z = 1
>>> dsdp = DSDPRelaxations([x, y, z], relations=[y*z - 1])
>>> dsdp.SetObjective(y - x**2)
>>> dsdp.solve()
```

**add_ade_moment_constraint**(*expr*, *rhs=0*)

Add an ADE-derived moment constraint directly.

**Parameters**

- **expr** – Sympy polynomial expression for the constraint.
- **rhs** – Right-hand side value (default: 0 for equality).

**build_ade_relations**(*diff_map: dict*, *prefix='d'*, *wrt=None*)

Build ADE relations from a derivation map by introducing derivative symbols.

For each generator g in diff_map, creates a new symbol d_g representing d_x(g), then adds the relation d_g - expr to the quotient ring. This encodes the ADE constraint algebraically via Groebner reduction.

CRITICAL: Derivative symbols MUST be prepended to the generator list so they become leading terms in the lex-ordered Groebner basis. Without this, ReduceExp() cannot substitute them back to polynomial expressions.

**Parameters**

- **diff_map** – Dictionary mapping generators to their derivative expressions.
- **prefix** – Prefix for generated derivative symbols (default: "d").
- **wrt** – Derivation variable name for symbol prefix. When provided, symbols are named "{prefix}{wrt}_{gen}" (e.g., "dx_y", "dy_v"). When None, uses "{prefix}_{gen}" (backward compatible).

**Returns**

- derivative_syms: Dict mapping original generators to their derivative symbols.
- relations: List of relation expressions (d_g - expr).
- new_gens: Updated generator list with derivative symbols prepended.

**Return type**
Tuple (derivative_syms, *relations*, new_gens)

### Example

```
>>> # tan(x) ADE: D_x(u) = 1 + u^2
>>> dm = {x: 1, u: 1 + u**2}
>>> dsyms, rels, gens = dsdp.build_ade_relations(dm)
>>> # dsyms = {x: dx, u: du}
>>> # rels = [dx - 1, du - (1 + u**2)]
>>> # gens = [dx, du, x, u]  (derivatives first!)
```

```
>>> # Multi-derivation: D_y(L) = v, D_y(v) = -v^2
>>> dsyms, rels, gens = dsdp.build_ade_relations({y:1, L:v, v:-v**2}, wrt='y')
>>> # dsyms = {y: dy_y, L: dy_L, v: dy_v}
```

**differentiate**(*expr*, *var=None*)

Compute the formal derivative of an expression using the registered derivation map.

Applies the Leibniz rule: d(f*g) = d(f)*g + f*d(g). If no derivation map is registered, falls back to sympy's diff().

**Parameters**

- **expr** – Sympy expression to differentiate.
- **var** – Variable to differentiate with respect to (default: first generator).

**Returns**
Formal derivative of expr.

**set_derivation**(*diff_map: dict*) → None

Register a derivation map for differential KKT conditions.

**Parameters**
**diff_map** – Dictionary mapping sympy generators to their derivatives. E.g., {x: y, y: -y} encodes dy/dx = -y.

`solve`(*order=None*)

Solve the DSDP relaxation with automatic solver routing.

Builds the relaxation and routes to the appropriate solver based on certificate structure: - Posynomial certificates -> GP/SONC (convex log-domain optimization) - Mixed-sign certificates -> SDP (general moment hierarchy) - Depth >= 2 -> SDP (alternating-sign expansion terms)

**Parameters**

**order** – Relaxation order (default: auto from problem degree).

**Returns**

Lower bound on the optimal value.

`property sp_auxsyms`

Return AuxSyms converted to pure SymPy (engine.Symbol -> sympy.Symbol).

Execution telemetry for Irene SDP pipeline.

Provides zero-overhead phase timing, structured diagnostics collection, and JSON export for benchmarking. Controlled by the environment variable `IRENE_TELEMETRY` (default `"1"` – enabled). Set to `"0"` to disable all telemetry with no runtime cost.

## 28.4 Public API

- `@timed(phase)` – decorator that logs wall-clock time for a named phase.
- `TelemetryContext` – context manager that collects structured metrics.
- `get_telemetry()` – retrieve the current session's telemetry dict.
- `clear_telemetry()` – reset the session state.
- `export_json(path)` – write telemetry to a JSON file.

## 28.5 Usage example

```
>>> from Irene.telemetry import timed, TelemetryContext, export_json
>>> @timed("basis_construction")
... def build_basis(deg): ...
>>> with TelemetryContext("sdp_solve", monomial_count=42):
...     solve_sdp()
>>> export_json("benchmark.json")
```

`class Irene.telemetry.PhaseTiming`(*wall_clock_s: float = 0.0*)

Wall-clock timing for a single pipeline phase.

`class Irene.telemetry.TelemetryContext`(*phase: str*, ***kwargs*)

Context manager that collects structured metrics for one SDP run.

**Parameters**

- **phase** (*str*) – Top-level phase name (e.g. `"sdp_solve"`, `"init_sdp_serial"`).
- ****kwargs** – Arbitrary metadata key-value pairs attached to this record.

### Example

```
>>> with TelemetryContext("solve", monomial_count=42, block_dims=[10, 5]):
...     sdp.solve()
```

**set**(*key: str*, *value: Any*)

Attach or update a metadata field on the active record.

**class** Irene.telemetry.**TelemetryRecord**(*phase: str = ''*, *timings: Dict[str, ~Irene.telemetry.PhaseTiming]=<factory>*, *metadata: Dict[str, ~typing.Any]=<factory>*)

Structured telemetry record for one SDP relaxation run.

**phase**

Human-readable phase name (e.g. `"basis_construction"`).

**Type**
str

**timings**

Per-sub-phase wall-clock times accumulated via the `@timed` decorator.

**Type**
dict[str, *PhaseTiming*]

**metadata**

Arbitrary key-value pairs set by the caller (monomial counts, block dims, etc.).

**Type**
dict[str, Any]

Irene.telemetry.**clear_telemetry**()

Reset all session-level telemetry state.

Irene.telemetry.**export_json**(*path: str*) → str

Write the full session telemetry to a JSON file.

**Parameters**
**path** (*str*) – File path for the output JSON.

**Returns**
The absolute path of the written file.

**Return type**
str

Irene.telemetry.**get_active_record**() → dict | None

Return the currently active telemetry record (if any) as a dict.

Irene.telemetry.**get_telemetry**() → list[dict]

Return all telemetry records as a list of plain dicts.

Each dict has keys `phase`, optionally `timings` and `metadata`.

Irene.telemetry.**timed**(*phase: str*)

Decorator that logs wall-clock time for a named sub-phase.

When telemetry is disabled (`IRENE_TELEMETRY=0`), this becomes a no-op wrapper with zero overhead beyond the function call itself.

**Parameters**
**phase** (*str*) – Name of the pipeline phase to time (e.g. "basis_construction", "matrix_assembly", "solve", "post_processing").

### Example

```
>>> @timed("basis_construction")
... def build_basis(deg):
...     return [monomials]
```

Irene.matrices.**find_psd_gram_matrix**(*polynomial*)

Uses Convex Optimization (SDP) to find a Positive Semidefinite (PSD) Gram matrix for the given polynomial.

Irene.matrices.**get_gram_matrix**(*polynomial*)

Computes a symmetric Gram matrix Q for a given polynomial p such that p = Z.T * Q * Z, where Z is the vector of monomials.

Uses SymEngine for polynomial expansion and SymPy fallback for Poly operations.

**Parameters**
**polynomial** – A symbolic polynomial expression (SymEngine or SymPy).

**Returns**
The Gram matrix as float64 array. Q_sym: The symbolic Gram matrix.

**Return type**
Q_np (np.ndarray)

Irene.matrices.**is_psd_numeric**(*matrix*, *tol=1e-08*)

Checks if a matrix is Positive Semidefinite using NumPy.

**Parameters**

- **matrix** (*np.ndarray*) – The input matrix.
- **tol** (*float*) – Tolerance for floating point errors.

**Returns**
True if PSD, False otherwise.

**Return type**
bool

Irene.matrices.**is_psd_symbolic**(*matrix*)

Checks if a SymPy matrix is Positive Semidefinite.

Irene.matrices.**numpy_to_latex**(*matrix*, *precision=3*, *env='bmatrix'*)

Converts a numpy matrix or array into LaTeX code.

**Parameters**

- **matrix** (*np.ndarray*) – The input matrix.
- **precision** (*int*) – Decimal places for floating point numbers.
- **env** (*str*) – The LaTeX environment (bmatrix, pmatrix, vmatrix, etc.)

**Returns**
The generated LaTeX string.

**Return type**
str

class Irene.invariant.**InvariantPolynomial**(*Prg*)

A class for computations on polynomials invariant under a finite group of permutations.

**ConjugateClosure**(*H*)

Computes the conjugate closure of a subgroup.

**Parameters**
**H** (*Subgroup*) – The subgroup to compute the conjugate closure of.

**Returns**
The conjugate closure of the subgroup.

**Return type**
Subgroup

**GenMon**(*alpha*)

Returns the monomial corresponding to the input tuple.

**OmegaFtilde**()

Finds the equivalence classes of exponents of the polynomial.

**QOmega**()

Finds the equivalence classes of exponents with respect to the group action.

**RedPart**(*f*, *P*)

Computes the reduced part of a polynomial.

**Parameters**

- **f** (*Polynomial*) – The polynomial to compute the reduced part of.
- **P** (*list[list[int]]*) – The partition of the variables.

**Returns**
The reduced part of the polynomial.

**Return type**
Polynomial

**Reynolds**(*f*, *G*)

Computes the Reynolds operator associated o the group G on the polynomial f.

**SigmaAlpha**(*sigma*, *alpha*)

Takes a permutation and an n-tuple and returns the result of the permutation on the indices of the tuple.

**Stabilizer**(*G*, *alpha*)

Returns the stabilizer group of an exponent.

**StblTldMax**()

Finds the largest subgroup which fixes the associated ring of tilde{f}_{max}

**tildemax**()

Computes the tilde{f}_{max} corresponding to the polynomial and the group action.

CHAPTER

# TWENTYNINE

# DOCTEST INTEGRATION

To ensure that code snippets in docstrings remain synchronized with the Irene codebase, Sphinx can be configured to run `doctest` blocks during documentation builds.

Enable in `conf.py`:

```
extensions = [
    # ... other extensions ...
    'sphinx.ext.doctest',
]
```

Then run as part of the documentation build pipeline:

```
cd doc
make doctest
```

Alternatively, run via pytest against the installed package:

```
.venv/bin/python3 -m pytest --doctest-modules Irene/
```

The following modules are doctest-ready (their docstrings contain executable examples): `relaxation_api.py`, `program.py`, `border_basis.py`. See *Benchmarks and Performance Evaluation* for the gallery-based integration test suite that serves as the primary verification layer.

CHAPTER

# THIRTY

# REVISION HISTORY

**Version 1.3.1 (Sep 18, 2026)**

- Added SOS+SONC relaxations, correlative sparsity detection, and border-basis reduction.
- Updated documentation branding and hosted documentation links for Irene.
- Removed redundant page-level tables of contents for compatibility with the Furo theme.
- Updated the external optimizer reference from pyOpt to pyOptSparse and linked to its current documentation.

**Version 1.2.6 (Jun 12, 2026)**

- Improved documentation build portability by defaulting `SPHINXBUILD` to the repository virtual environment (`../.venv/bin/python -m sphinx`).
- Hardened `make latexpdf` to use `latexmk` when available and automatically fall back to two-pass `pdflatex` when `latexmk` is missing.
- Added `make latexpdf-clean` to remove stale LaTeX build artifacts and perform a clean PDF rebuild.
- Fixed a LaTeX compilation blocker in the SOS+SONC docs by replacing a non-ASCII real-number symbol in a code block with LaTeX-safe ASCII text.

**Version 1.2.5 (Mar 12, 2026)**

- Expanded documentation from SDP-only emphasis to a unified POP guide covering SDP, geometric programming, and SONC relaxations.
- Added new chapters for architecture, group-ring foundations, optimization problem representation, geometric relaxations, SONC relaxations, and runnable examples.
- Added method-selection guidance, dependency matrix, and solver troubleshooting notes.
- Added explicit theory-to-code mapping for constrained SONC families (Section 3 style equations) and geometric lower-bound construction (Section 4 equation mapping).
- Expanded API documentation coverage in Sphinx for `program`, `grouprings`, `geometric`, and `sonc` modules.
- Updated Sphinx configuration for modern toolchain compatibility and warning-free documentation builds.

**Version 1.2.2 (Mar 15, 2017)**

- Serialization and pickling: Saving the latest state of the program on break which can be retrieved later and resume.

**Version 1.2.1 (Mar 2, 2017)**

- Removed dependency on `joblib` for multiprocessing.

**Version 1.2.0 (Jan 5, 2017)**

- LaTeX representation of Irene's objects.
- `SDRelaxSol` can be called as an iterable.
- Default objective function is set to a `sympy` object for truncated moment problems.

**Version 1.1.0 (Dec 25, 2016 - Merry Christmas)**

- Extracting minimizers via `SDRelaxSol.ExtractSolution()` and help of `scipy`,
- Extracting minimizers implementing Lasserre-Henrion algorithm,
- Adding `SDPRelaxations.Probability` and `SDPRelaxations.PSDMoment` to give more flexibility over moments and enables rational minimization.
- SOS decomposition implemented.
- `__str__` method for `SDPRelaxations`.
- Using *pyOptSparse* as the external optimizer.
- More benchmark examples.

**Version 1.0.0 (Dec 07, 2016)**

- Initial release (Birth of Irene)

CHAPTER

# THIRTYONE

# APPENDIX

## 31.1 Global Notation Index

This table standardizes notation used across all chapters of the Irene manual. Where a symbol has different meanings in different contexts, the primary usage is listed first.

| Symbol | Meaning | Primary chapter(s) |
|---|---|---|
| $t$ | Relaxation order (hierarchy level) | *Optimization*, *Unified Relaxation API* |
| $d$ | Polynomial degree bound; $2t$ for SOS | *Border Basis Theory and Implementation*, *Optimization* |
| $M_t(y)$ | Moment matrix at order $t$ | *Optimization*, *Semidefinite Programming Relaxations* |
| $M_t(g_i y)$ | Localizing matrix for constraint $g_i \geq 0$ | *Optimization* |
| $L$, $L_\mu$ | Moment functional / integration w.r.t. measure $\mu$ | *Optimization* |
| $\rho$ | Global minimum value | *Optimization* |
| $\gamma$, $\gamma_t$ | Lower bound at relaxation order $t$ | *Optimization*, *Unified Relaxation API* |
| $K$ | Semialgebraic feasible set $\{x : g_i(x) \geq 0\}$ | *Optimization* |
| $Q_{\mathbf{g}}$ | Quadratic module generated by $g_1, \ldots, g_m$ | *Optimization* |
| $\Sigma$, $\sum A^2$ | Sums of squares in algebra $A$ | *Optimization* |
| $\Sigma + C$ | Schick cone (SOS + circuit polynomials) | *SOS+SONC Relaxations* |
| $\mathbb{R}[S]$ | Semigroup algebra over semigroup $S$ | *Group-Ring Foundations* |
| $\mathcal{I}$, $\mathcal{I}_{\text{ADE}}$ | Ideal (algebraic / differential) | *Group-Ring Foundations*, *Differential SDP and Mean Relaxations* |
| $B_t$, $B_{t,k}$ | Monomial basis (full / per-clique) | *Border Basis Theory and Implementation*, *Unified Relaxation API* |
| $\partial B$ | Border of monomial basis | *Border Basis Theory and Implementation* |
| $\text{New}(f)$ | Newton polytope of polynomial $f$ | *Newton Polytope Pruning* |
| $G = (V, E)$ | Correlative sparsity graph | *Correlative Sparsity Detection* |
| $C_1, \ldots, C_p$ | Maximal cliques after chordal completion | *Correlative Sparsity Detection*, *Unified Relaxation API* |
| $D, D_i$ | Derivation operator on $\mathbb{R}[S]$ | *Group-Ring Foundations* |
| $\Theta$ | Operator semigroup $\langle \delta_1, \ldots, \delta_m \rangle$ | *Group-Ring Foundations* |
| $M_{q,p}(X, w)$ | Weighted power mean form | *Differential SDP and Mean Relaxations* |
| $\mathcal{M}_{n,2d}$ | Cone of nonnegative mean polynomials | *Differential SDP and Mean Relaxations* |
| $\prec$ | Admissible term order (e.g., $\prec_{\text{degrevlex}}$) | *Border Basis Theory and Implementation* |

## 31.2 pyProximation

pyProximation is a companion library for function approximation in Hilbert spaces. Full documentation is included in this manual — see the *pyProximation* section.

## 31.3 pyOptSparse

pyOptSparse is a Python-based package for formulating and solving nonlinear constrained optimization problems in an efficient, reusable and portable manner. It is an open-source software distributed under the terms of the GNU Lesser General Public License.

***pyOptSparse* provides a unified interface to nonlinear optimizers.**

- SNOPT - Sparse NOlinear OPTimizer
- NLPQL - Non-Linear Programming by Quadratic Lagrangian
- NLPQLP - NonLinear Programming with Non-Monotone and Distributed Line Search
- FSQP - Feasible Sequential Quadratic Programming
- SLSQP - Sequential Least Squares Programming
- PSQP - Preconditioned Sequential Quadratic Programming
- ALGENCAN - Augmented Lagrangian with GENCAN
- FILTERSD
- MMA - Method of Moving Asymptotes
- GCMMA - Globally Convergent Method of Moving Asymptotes
- CONMIN - CONstrained function MINimization
- MMFD - Modified Method of Feasible Directions
- KSOPT - Kreisselmeier–Steinhauser Optimizer
- COBYLA - Constrained Optimization BY Linear Approximation
- SDPEN - Sequential Penalty Derivative-free method for Nonlinear constrained optimization
- SOLVOPT - SOLver for local OPTimization problems
- ALPSO - Augmented Lagrangian Particle Swarm Optimizer
- NSGA2 - Non Sorting Genetic Algorithm II
- ALHSO - Augmented Lagrangian Harmony Search Optimizer
- MIDACO - Mixed Integer Distributed Ant Colony Optimization

### 31.3.1 Basic usage:

*pyOptSparse* is designed to solve general constrained nonlinear optimization problems:

$$\min f(x) \quad \text{Subject to} \quad g_j(x) = 0 \quad j = 1, \ldots, m_e \; g_j(x) \leq 0 \quad j = m_e + 1, \ldots, m \; l_i \leq x_i \leq u_i \quad i = 1, \ldots, n,$$

**where:**

- $x$ is the vector of design variables
- $f(x)$ is a nonlinear function

- $g(x)$ is a linear or nonlinear function
- $n$ is the number of design variables
- $m_e$ is the number of equality constraints
- $m$ is the total number of constraints (number of equality constraints: $m_i = m - m_e$).

The following is illustrative pseudo-code for a `pyOptSparse` optimization workflow:

```
# General Objective Function Template:
def obj_fun(x, *args, **kwargs):
        """
        f: objective value
        g: array (or list) of constraint values
        fail: 0 for successful function evaluation, 1 for unsuccessful function␣
↪evaluation (test must be provided by user)
        If the Optimization problem is unconstrained, g must be returned as an empty␣
↪list or array: g = []
        Inequality constraints are handled as `<=`.
        """
        fail = 0
        f = function(x,*args,**kwargs)
        g = function(x,*args,**kwargs)

        return f,g,fail

# Instantiating an Optimization Problem:
opt_prob = Optimization('name', obj_fun)
# Assigning Objective:
opt_prob.addObj('name', value=0.0, optimum=0.0)
# Single Design variable:
opt_prob.addVar('name', type='c', value=0.0, lower=-inf, upper=inf, choices=listochoices)
# A Group of Design Variables:
opt_prob.addVarGroup('name', numerinGroup, type='c', value=value, lower=lb, upper=up,␣
↪choices=listochoices)
# where `value`, `lb`, `ub` (float or int or list or 1Darray).
# and supported Types are 'c': continuous design variable;
# `i`: integer design variable;
# `d`: discrete design variable (based on choices, e.g.: list/dict of materials).
# Assigning Constraints:
## Single Constraint:
opt_prob.addCon('name', type='i', lower=-inf, upper=inf, equal=0.0)
## A Group of Constraints:
opt_prob.addConGroup('name', numberinGroup, type='i', lower=lb, upper=up, equal=eq)
# where `lb`, `ub`, `eq` are (float or int or list or 1Darray).
# and supported types are
# `i` - inequality constraint;
# `e` - equality constraint.

# Instantiating an Optimizer (e.g.: Snopt):
opt = pySNOPT.SNOPT()
# Solving the Optimization Problem:
opt(opt_prob, sens_type='FD', disp_opts=False, sens_mode='', *args, **kwargs)
# Output:
print opt_prob
```

For installation and usage details, see the pyOptSparse documentation.

## 31.4 LaTeX support

There is a method implemented for every class of Irene module that generates a LaTeX code related to the object. This code can be retrieved by calling `LaTeX` function on the instance.

Calling `LaTeX` on a

- `SDPRelaxations` instance returns the code which demonstrates user entered optimization problem.
- `SDRelaxSol` instance returns the code for the moment matrix of the solution.
- `Mom` instance (moment object) returns the LaTeX code of the object.
- `sdp` instance returns the code for the SDP.

Moreover, if `LaTeX` function is called on a `SymPy` object it calls `sympy.latex` function and returns the output.

The `LaTeX` function is a member of `Irene.base` module which calls `__latex__` method of its classes.

CHAPTER

# THIRTYTWO

# REFERENCES

This page is the consolidated bibliography for the Irene documentation. It lists the works the method chapters rest on, grouped by the construction each one supports. Page-level reading lists in individual chapters remain for local context; when the two disagree, this page is authoritative.

Bibliographic data here was checked against Crossref or the arXiv API (2026-10); volume, issue, page range and author names are the published ones, not preprint-stage metadata.

## 32.1 The truncated moment problem on algebras

The results the generalization from polynomials to arbitrary unital commutative algebras rests on (see *Optimization* and *Theory Overview*).


- J. Stochel, *Solving the truncated moment problem solves the full moment problem*, Glasgow Mathematical Journal **43** (3) (2001), 335–341. DOI: 10.1017/S0017089501030130.


## 32.2 Moment/SOS hierarchy and polynomial optimization


- P. A. Parrilo, *Structured semidefinite programs and semialgebraic geometry methods in robustness and optimization*, PhD thesis, California Institute of Technology, 2000.
- M. Laurent, *Sums of squares, moment matrices and optimization over polynomials*, in Emerging Applications of Algebraic Geometry, Springer (2009), 157–270.
- M. Marshall, *Positive Polynomials and Sums of Squares*, Mathematical Surveys and Monographs **146**, American Mathematical Society, 2008.
- J.-B. Lasserre, *An Introduction to Polynomial and Semi-Algebraic Optimization*, Cambridge Texts in Applied Mathematics **52**, Cambridge University Press, 2015.


## 32.3 Geometric-programming bounds

The GP path in *Geometric Programming Relaxations* descends from the lower-bound construction of Ghasemi and Marshall, extended with Lasserre's moment relaxation and then to constrained geometries.


- M. Ghasemi and M. Marshall, *Lower bounds for polynomials using geometric programming*, SIAM Journal on Optimization **22** (2) (2012), 460–473. DOI: 10.1137/110836869.
- M. Ghasemi, J.-B. Lasserre, and M. Marshall, *Lower bounds on the global minimum of a polynomial*, Computational Optimization and Applications **57** (2) (2013), 387–402. DOI: 10.1007/s10589-013-9596-x.
- M. Ghasemi and M. Marshall, *Lower bound for a polynomial on a product of hyperellipsoids using geometric programming*, preprint arXiv:2510.03105 (2025).

- R. J. Duffin, E. L. Peterson, and C. Zener, *Geometric Programming: Theory and Application*, Wiley, 1967.
- S. Iliman and T. de Wolff, *Lower bounds for polynomials with simplex Newton polytopes based on geometric programming*, SIAM Journal on Optimization **26** (2) (2016), 1128–1146. DOI: 10.1137/140962425.
- S. Iliman and T. de Wolff, *Amoebas, nonnegative polynomials and sums of squares supported on circuits*, Research in the Mathematical Sciences **3** (2016), article 9. DOI: 10.1186/s40687-016-0052-2.


## 32.4 SONC relaxations

For the circuit-based family in *SONC Relaxations*: the characterization of nonnegative circuit polynomials, the Positivstellensatz and its constrained extension that turn the problem into a geometric program, and the duality theory behind optimal SONC bounds.


- M. Dressler, S. Iliman, and T. de Wolff, *A Positivstellensatz for sums of nonnegative circuit polynomials*, SIAM Journal on Applied Algebra and Geometry **1** (1) (2017), 536–555. DOI: 10.1137/16M1086303.
- M. Dressler, S. Iliman, and T. de Wolff, *An approach to constrained polynomial optimization via nonnegative circuit polynomials and geometric programming*, Journal of Symbolic Computation **91** (2019), 149–172. DOI: 10.1016/j.jsc.2018.06.018.
- D. Papp, *Duality of sum of nonnegative circuit polynomials and optimal SONC bounds*, Journal of Symbolic Computation **114** (2023), 246–266. DOI: 10.1016/j.jsc.2022.04.015.
- L. Katthän, H. Naumann, and T. Theobald, *A unified framework of SAGE and SONC polynomials and its duality theory*, Mathematics of Computation **90** (329) (2021), 1297–1322. DOI: 10.1090/mcom/3607.
- J. Heuer and T. de Wolff, *The duality of SONC: advances in circuit-based certificates*, Journal of Symbolic Computation **132** (2026), 102479. DOI: 10.1016/j.jsc.2025.102479.
- M. Dressler, D. Heuer, D. Naumann, and T. de Wolff, *Global optimization via the dual SONC cone and linear programming*, preprint arXiv:2002.09368 (2020).


## 32.5 SOS+SONC hybrid certificates

For the two-step framework and the cone geometry in *SOS+SONC Relaxations*:


- M. Schick, *Sums of squares plus sums of nonnegative circuit polynomials*, PhD dissertation, Universität Konstanz, 2025. Toolbox: SOS_plus_SONC_toolbox.
- M. Dressler, S. Kuhlmann, and M. Schick, *Geometrical study of the cone of sums of squares plus sums of nonnegative circuits*, preprint arXiv:2305.14848 (2023).
- M. Ghasemi and S. Kuhlmann, *The cone generated by positive semidefinite mean polynomials*, preprint arXiv:2608.22739 (2026).


## 32.6 Mean forms and differential SDP

For the mean-polynomial certificate family and the differential hierarchy in *Differential SDP and Mean Relaxations*:


- M. Ghasemi, *Mean polynomials: generalizing SOS and SONC via power mean forms*, companion manuscript, 2026 (Chapters 1–7).
- J. F. Ritt, *Differential Algebra*, American Mathematical Society Colloquium Publications **33**, 1950.
- E. R. Kolchin, *Differential Algebra and Algebraic Groups*, Academic Press, 1973.

## 32.7 Structure exploitation and numerics

For the reductions in *Correlative Sparsity Detection*, *Newton Polytope Pruning*, *Border Basis Theory and Implementation* and *Sparse Moment Method (SparseMomentSDP)*:

- J.-B. Lasserre, *Convergent SDP-relaxations in polynomial optimization with sparsity*, SIAM Journal on Optimization **17** (3) (2006), 822–843. DOI: 10.1137/05064504x.
- J. Wang, V. Magron, and J.-B. Lasserre, *TSSOS: a moment-SOS hierarchy that exploits term sparsity*, SIAM Journal on Optimization **31** (1) (2021), 30–58. DOI: 10.1137/19M1307871.
- A. Kehrein and M. Kreuzer, *Computing border bases*, Journal of Pure and Applied Algebra **205** (2) (2006), 279–295. DOI: 10.1016/j.jpaa.2005.07.006.
- A. Kehrein, M. Kreuzer, and L. Robbiano, *An algebraist's view on border bases*, in Algorithms and Computation in Mathematics **14**, Springer (2005), 169–202. DOI: 10.1007/3-540-27357-3_4.
- T. Baumbach and M. Bender, *The moment method for computing real points of sparse polynomial systems*, preprint arXiv:2609.33313 (2026) – the source of the toric construction in *Sparse Moment Method (SparseMomentSDP)*.

## 32.8 Archival versions

Released versions of Irene are archived on Zenodo. Cite the software itself through `CITATION.cff` in the repository root, which carries the DOI of the current release.

CHAPTER

# THIRTYTHREE

# INDICES AND TABLES

- genindex
- modindex
- search

# BIBLIOGRAPHY


[BB2026] T. Baumbach, M. Bender, *The Moment Method for Computing Real Points of Sparse Polynomial Systems*, arXiv:2609.33313 [cs.SC], 2026.

[Stochel2001] J. Stochel, *Solving the truncated moment problem solves the full moment problem*, Glasgow Math. J. 43(3), 335–341 (2001).

[CGIK] R. E. Curto, M. Ghasemi, M. Infusino, and S. Kuhlmann, *The truncated moment problem for unital commutative $\mathbb{R}$-algebras*, Journal of Operator Theory **90** (1) (2023), 223–261. DOI: 10.7900/jot.2021nov26.2392.

[JNie] J. Nie, *The A-truncated K-moment problem*, Foundations of Computational Mathematics **14** (6) (2014), 1243–1276.

[JBL] J.-B. Lasserre, *Global optimization with polynomials and the problem of moments*, SIAM Journal on Optimization **11** (3) (2001), 796–817. DOI: 10.1137/S1052623400366802.

[HL] D. Henrion and J.-B. Lasserre, *Detecting global optimality and extracting solutions in GloptiPoly*, in Positive Polynomials in Control, Lecture Notes in Control and Information Sciences **312**, Springer (2005), 293–310.

[JH16] C. Josz and D. Henrion, *Strong duality in Lasserre's hierarchy for polynomial optimization*, Optimization Letters **10** (1) (2016), 3–10.

# PYTHON MODULE INDEX

## i

Irene.base, 117
Irene.border_basis, 142
Irene.cvxpy_solver, 166
Irene.dsdp, 168
Irene.geometric, 139
Irene.grouprings, 133
Irene.invariant, 174
Irene.matrices, 173
Irene.newton_polytope, 146
Irene.program, 125
Irene.relaxation_api, 149
Irene.relaxations, 117
Irene.sdp, 123
Irene.sonc, 140
Irene.sosonc, 140
Irene.sparse_moment, 152
Irene.sparsity, 144
Irene.symbolic_engine, 164
Irene.telemetry, 171

## p

pyProximation.base, 113
pyProximation.interpolation, 115
pyProximation.measure, 113
pyProximation.orthsys, 113
pyProximation.rational, 114

# INDEX


## A

A (*Irene.sparse_moment.ToricSetup attribute*), 159
Abs() (*Irene.symbolic_engine.SymbolicEngine method*), 165
add_ade_moment_constraint() (*Irene.dsdp.DSDPRelaxations method*), 169
add_constraints() (*Irene.program.OptimizationProblem method*), 127
add_derivative() (*Irene.grouprings.SemigroupAlgebra method*), 137
add_poly_terms() (*Irene.sparsity.CorrelativeSparsity method*), 144
add_relations() (*Irene.grouprings.CommutativeSemigroup method*), 136
add_term() (*Irene.sparsity.CorrelativeSparsity method*), 144
AddConstantBlock() (*Irene.cvxpy_solver.CvxpySDPSolver method*), 167
AddConstantBlock() (*Irene.sdp.sdp method*), 123
AddConstraint() (*Irene.relaxations.SDPRelaxations method*), 119
AddConstraintBlock() (*Irene.cvxpy_solver.CvxpySDPSolver method*), 167
AddConstraintBlock() (*Irene.sdp.sdp method*), 123
AddEquality() (*Irene.cvxpy_solver.CvxpySDPSolver method*), 167
adjacency (*Irene.sparsity.CorrelativeSparsity attribute*), 144
Algorithm1Result (*class in Irene.sparse_moment*), 152
analyse_program() (*Irene.program.OptimizationProblem method*), 127
AtomicSGElement (*class in Irene.grouprings*), 133
auto_transform_matrix() (*Irene.geometric.GPRelaxations method*), 139
aux_rels (*Irene.grouprings.CommutativeSemigroup attribute*), 135
available_backends() (*Irene.symbolic_engine.SymbolicEngine static method*), 165
available_solvers() (*in module Irene.cvxpy_solver*), 168
AvailableSDPSolvers() (*Irene.base.base method*), 117

## B

base (*class in Irene.base*), 117
basis (*Irene.border_basis.BorderBasis attribute*), 143
Basis() (*pyProximation.orthsys.OrthSystem method*), 114
border (*Irene.border_basis.BorderBasis attribute*), 143
border_basis_degree (*Irene.relaxations.RelaxationConfig attribute*), 118
BorderBasis (*class in Irene.border_basis*), 142
boxCheck() (*pyProximation.measure.Measure method*), 113
build_ade_relations() (*Irene.dsdp.DSDPRelaxations method*), 169
bwe (*Irene.sparse_moment.SparseRootsResult attribute*), 158
bwe_components (*Irene.sparse_moment.SparseRootsResult attribute*), 158

## C

Calpha() (*Irene.relaxations.SDPRelaxations method*), 119
Calpha_() (*in module Irene.relaxations*), 117
Calpha__() (*in module Irene.relaxations*), 117
certificate (*Irene.relaxation_api.RelaxResult attribute*), 150
check() (*pyProximation.measure.Measure method*), 113
check_relations() (*Irene.sparse_moment.SparseBorderBasis method*), 156
clear_fallback_log() (*Irene.symbolic_engine.SymbolicEngine method*), 165

clear_telemetry() (*in module Irene.telemetry*), 172
coefficients (*Irene.sparse_moment.SparseRootsResult attribute*), 159
combined_newton_polytope() (*in module Irene.newton_polytope*), 147
Commit() (*Irene.relaxations.SDPRelaxations method*), 119
CommutativeSemigroup (*class in Irene.grouprings*), 135
compare() (*Irene.relaxation_api.RelaxationEngine method*), 151
compare_all() (*in module Irene.relaxation_api*), 151
compare_diags() (*Irene.geometric.GPRelaxations static method*), 139
components (*Irene.sparsity.CorrelativeSparsity attribute*), 144
compute_pruned_basis() (*Irene.newton_polytope.NewtonPruner method*), 146
condition (*Irene.sparse_moment.RankTestResult attribute*), 154
conditioning_diagnostic() (*Irene.border_basis.BorderBasis method*), 143
ConjugateClosure() (*Irene.invariant.InvariantPolynomial method*), 174
connected_to_one (*Irene.sparse_moment.SparseBorderBasis attribute*), 156
connected_to_one (*Irene.sparse_moment.SparseRootsResult attribute*), 159
constraint_terms_with_even_exponent (*Irene.program.OptimizationProblem attribute*), 127
constraint_terms_with_odd_exponent (*Irene.program.OptimizationProblem attribute*), 127
constraint_trms_with_negative_coefficient (*Irene.program.OptimizationProblem attribute*), 126
constraint_trms_with_positive_coefficient (*Irene.program.OptimizationProblem attribute*), 126
constraints (*Irene.program.OptimizationProblem attribute*), 125
constraints_degree (*Irene.program.OptimizationProblem attribute*), 126
constraints_half_degree (*Irene.program.OptimizationProblem attribute*), 126
construct_mean_moment_matrix() (*Irene.dsdp.DSDPMeanRelaxation method*), 169
content (*Irene.grouprings.AtomicSGElement attribute*), 134
content (*Irene.grouprings.SemigroupAlgebraElement attribute*), 138
converged (*Irene.sparse_moment.Algorithm1Result attribute*), 152
converged (*Irene.sparse_moment.RankTestResult attribute*), 154
convex_combination() (*Irene.program.OptimizationProblem method*), 128
CorrelativeSparsity (*class in Irene.sparsity*), 144
csdp() (*Irene.sdp.sdp method*), 124
CvxOpt() (*Irene.cvxpy_solver.CvxpySDPSolver method*), 167
CvxOpt() (*Irene.sdp.sdp method*), 123
CvxpySDPSolver (*class in Irene.cvxpy_solver*), 166

## D

D (*Irene.sparse_moment.RankTestResult attribute*), 154
Decompose() (*Irene.relaxations.SDPRelaxations method*), 119
deg_A() (*in module Irene.sparse_moment*), 160
degree (*Irene.border_basis.BorderBasis attribute*), 143
degree() (*Irene.grouprings.CommutativeSemigroup method*), 136
delta() (*Irene.program.OptimizationProblem method*), 128
Delta() (*pyProximation.interpolation.Interpolation method*), 115
delta_vertex() (*Irene.program.OptimizationProblem method*), 128
derivative() (*Irene.grouprings.SemigroupAlgebra method*), 137
derivatives (*Irene.grouprings.SemigroupAlgebra attribute*), 137
detect_sparsity_from_polys() (*in module Irene.sparsity*), 145
detect_sparsity_from_problem() (*in module Irene.sparsity*), 145
DetSymEnv() (*pyProximation.base.Foundation method*), 113
diff() (*Irene.grouprings.SemigroupAlgebra method*), 138
differentiate() (*Irene.dsdp.DSDPRelaxations method*), 170
divide() (*Irene.grouprings.AtomicSGElement method*), 134
divide() (*Irene.grouprings.SemigroupAlgebraElement method*), 139
DomainMatrix() (*Irene.symbolic_engine.SymbolicEngine method*), 165

DSDPKKTRelaxation (*class in Irene.dsdp*), 168
DSDPMeanRelaxation (*class in Irene.dsdp*), 168
DSDPRelaxations (*class in Irene.dsdp*), 169

## E

edges (*Irene.grouprings.CommutativeSemigroup attribute*), 135
eigenvalues (*Irene.sparse_moment.SparseRootsResult attribute*), 158
element_sub_lattice() (*Irene.grouprings.CommutativeSemigroup method*), 136
error_code (*Irene.relaxation_api.RelaxResult attribute*), 150
error_code (*Irene.sosonc.SOSONCRelaxSol attribute*), 141
expand() (*Irene.symbolic_engine.SymbolicEngine method*), 165
ExponentsVec() (*Irene.relaxations.SDPRelaxations method*), 119
export_json() (*in module Irene.telemetry*), 172
ExtractSolution() (*Irene.relaxations.SDRelaxSol method*), 122
ExtractSolutionLH() (*Irene.relaxations.SDRelaxSol method*), 122
ExtractSolutionScipy() (*Irene.relaxations.SDRelaxSol method*), 122

## F

f_sonc (*Irene.sosonc.SOSONCRelaxSol attribute*), 141
f_sos (*Irene.sosonc.SOSONCRelaxSol attribute*), 140
fallback_log (*Irene.symbolic_engine.SymbolicEngine attribute*), 164
fallback_stats() (*Irene.symbolic_engine.SymbolicEngine method*), 165
fallback_to_sympy() (*in module Irene.symbolic_engine*), 166
finalize() (*Irene.sparsity.CorrelativeSparsity method*), 144
find_psd_gram_matrix() (*in module Irene.matrices*), 173
FormBasis() (*pyProximation.orthsys.OrthSystem method*), 114
Foundation (*class in pyProximation.base*), 113
FourierBasis() (*pyProximation.orthsys.OrthSystem method*), 114
from_exponents() (*Irene.sparse_moment.ToricSetup class method*), 160
from_problem() (*Irene.relaxations.SDPRelaxations class method*), 121
full_basis_size (*Irene.newton_polytope.NewtonPruner attribute*), 146
Function() (*Irene.symbolic_engine.SymbolicEngine method*), 165

## G

G_A (*Irene.sparse_moment.ToricSetup attribute*), 159
GenMon() (*Irene.invariant.InvariantPolynomial method*), 174
gens (*Irene.grouprings.CommutativeSemigroup attribute*), 135
gens (*Irene.grouprings.SemigroupAlgebra attribute*), 137
get_active_record() (*in module Irene.telemetry*), 172
get_backend() (*Irene.symbolic_engine.SymbolicEngine method*), 165
get_components() (*Irene.sparsity.UnionFind method*), 145
get_gram_matrix() (*in module Irene.matrices*), 173
get_symbolic_backend() (*in module Irene.symbolic_engine*), 166
get_telemetry() (*in module Irene.telemetry*), 172
getConstraint() (*Irene.relaxations.SDPRelaxations method*), 121
getMomentConstraint() (*Irene.relaxations.SDPRelaxations method*), 121
getObjective() (*Irene.relaxations.SDPRelaxations method*), 121
globalMinSONC() (*Irene.sosonc.SOSONCRelaxations method*), 141
globalMinSOS() (*Irene.sosonc.SOSONCRelaxations method*), 142
globalMinSOSPSONC() (*Irene.sosonc.SOSONCRelaxations method*), 142
GPRelaxations (*class in Irene.geometric*), 139
groebner() (*Irene.symbolic_engine.SymbolicEngine method*), 165

## H

h_count (*Irene.sparse_moment.SparseMomentResult attribute*), 157
h_plus() (*Irene.geometric.GPRelaxations method*), 139
has_symbol() (*Irene.program.OptimizationProblem static method*), 129

## I

in_newton() (*Irene.program.OptimizationProblem method*), 129

infeasible_at_t (*Irene.sparse_moment.Algorithm1Result attribute*), 153
init_time (*Irene.relaxation_api.RelaxResult attribute*), 150
InitSDP() (*Irene.relaxations.SDPRelaxations method*), 119
inner() (*pyProximation.orthsys.OrthSystem method*), 114
integral() (*pyProximation.measure.Measure method*), 113
Interpolate() (*pyProximation.interpolation.Interpolation method*), 115
Interpolation (*class in pyProximation.interpolation*), 115
InvariantPolynomial (*class in Irene.invariant*), 174
inverses (*Irene.grouprings.CommutativeSemigroup attribute*), 135
Irene.base
    module, 117
Irene.border_basis
    module, 142
Irene.cvxpy_solver
    module, 166
Irene.dsdp
    module, 168
Irene.geometric
    module, 139
Irene.grouprings
    module, 133
Irene.invariant
    module, 174
Irene.matrices
    module, 173
Irene.newton_polytope
    module, 146
Irene.program
    module, 125
Irene.relaxation_api
    module, 149
Irene.relaxations
    module, 117
Irene.sdp
    module, 123
Irene.sonc
    module, 140
Irene.sosonc
    module, 140
Irene.sparse_moment
    module, 152
Irene.sparsity
    module, 144
Irene.symbolic_engine
    module, 164
Irene.telemetry
    module, 171
is_abelian (*Irene.grouprings.CommutativeSemigroup attribute*), 135
is_full_ring (*Irene.sparse_moment.ToricSetup property*), 160
is_psd_numeric() (*in module Irene.matrices*), 173
is_psd_symbolic() (*in module Irene.matrices*), 173
is_semigroup (*Irene.grouprings.CommutativeSemigroup attribute*), 135
is_sparse (*Irene.sparsity.CorrelativeSparsity attribute*), 144
iterations (*Irene.sparse_moment.Algorithm1Result attribute*), 153

## K

kernel_polys (*Irene.sparse_moment.Algorithm1Result attribute*), 153
kernel_polys (*Irene.sparse_moment.SparseRootsResult attribute*), 159

## L

lambda_vec (*Irene.sparse_moment.SparseMomentResult attribute*), 157
lambdify() (*Irene.symbolic_engine.SymbolicEngine method*), 165
LaTeX() (*in module Irene.base*), 117
latex() (*Irene.symbolic_engine.SymbolicEngine method*), 165
lattice_edges() (*Irene.grouprings.CommutativeSemigroup method*), 136
lattice_vertices() (*Irene.grouprings.CommutativeSemigroup method*), 136
LC() (*Irene.grouprings.AtomicSGElement method*), 134
LC() (*Irene.grouprings.SemigroupAlgebraElement method*), 138
linear_combination() (*Irene.program.OptimizationProblem method*), 129
LM() (*Irene.grouprings.AtomicSGElement method*), 134
LM() (*Irene.grouprings.SemigroupAlgebraElement method*), 138
LocalizedMoment() (*Irene.relaxations.SDPRelaxations method*), 119
LocalizedMoment_() (*Irene.relaxations.SDPRelaxations method*), 119
LT() (*Irene.grouprings.AtomicSGElement method*), 134
LT() (*Irene.grouprings.SemigroupAlgebraElement method*), 138

lt_divisible_by() (*Irene.grouprings.AtomicSGElement method*), 134
lt_divisible_by() (*Irene.grouprings.SemigroupAlgebraElement method*), 139

## M

M_s (*Irene.sparse_moment.SparseMomentResult attribute*), 157
Matrix() (*Irene.symbolic_engine.SymbolicEngine method*), 165
max_deg (*Irene.grouprings.CommutativeSemigroup attribute*), 135
Measure (*class in pyProximation.measure*), 113
measure() (*pyProximation.measure.Measure method*), 113
message (*Irene.relaxation_api.RelaxResult attribute*), 150
message (*Irene.sosonc.SOSONCRelaxSol attribute*), 141
metadata (*Irene.telemetry.TelemetryRecord attribute*), 172
method (*Irene.relaxation_api.RelaxResult attribute*), 150
method (*Irene.sosonc.SOSONCRelaxSol attribute*), 140
Minimize() (*Irene.relaxations.SDPRelaxations method*), 119
minkowski_sum() (*in module Irene.newton_polytope*), 147
MinNumPoints() (*pyProximation.interpolation.Interpolation method*), 115
module
    Irene.base, 117
    Irene.border_basis, 142
    Irene.cvxpy_solver, 166
    Irene.dsdp, 168
    Irene.geometric, 139
    Irene.grouprings, 133
    Irene.invariant, 174
    Irene.matrices, 173
    Irene.newton_polytope, 146
    Irene.program, 125
    Irene.relaxation_api, 149
    Irene.relaxations, 117
    Irene.sdp, 123
    Irene.sonc, 140
    Irene.sosonc, 140
    Irene.sparse_moment, 152
    Irene.sparsity, 144
    Irene.symbolic_engine, 164
    Irene.telemetry, 171
    pyProximation.base, 113
    pyProximation.interpolation, 115
    pyProximation.measure, 113
    pyProximation.orthsys, 113
    pyProximation.rational, 114
Mom (*class in Irene.relaxations*), 117
moment_indices() (*in module Irene.sparse_moment*), 160
moment_matrix_dimension_reduction() (*Irene.newton_polytope.NewtonPruner method*), 147
moment_matrix_partition() (*Irene.sparsity.CorrelativeSparsity method*), 144
moment_matrix_structure() (*Irene.border_basis.BorderBasis method*), 143
MomentConstraint() (*Irene.relaxations.SDPRelaxations method*), 120
MomentMat() (*Irene.relaxations.SDPRelaxations method*), 120
MomentsOrd() (*Irene.relaxations.SDPRelaxations method*), 120
mono2ord_tuple() (*Irene.program.OptimizationProblem method*), 130
monomial_pruning (*Irene.relaxations.RelaxationConfig attribute*), 118
Monomials() (*pyProximation.interpolation.Interpolation method*), 115
mult_tables (*Irene.border_basis.BorderBasis attribute*), 143

## N

n_monomials_s (*Irene.sparse_moment.SparseMomentResult attribute*), 157
n_monomials_t (*Irene.sparse_moment.SparseMomentResult attribute*), 157
n_real (*Irene.sparse_moment.RankTestResult attribute*), 154
newton() (*Irene.program.OptimizationProblem method*), 130
newton_polytope() (*in module Irene.newton_polytope*), 147
NewtonPruner (*class in Irene.newton_polytope*), 146
nf_residuals (*Irene.sparse_moment.SparseBorderBasis attribute*), 156
no_real_roots (*Irene.sparse_moment.Algorithm1Result attribute*), 153
norm() (*pyProximation.measure.Measure method*), 113
normal_form() (*Irene.sparse_moment.SparseBorderBasis method*), 156
normal_forms (*Irene.sparse_moment.SparseBorderBasis attribute*), 156

NumericalRank() (*Irene.relaxations.SDRelaxSol method*), 123
numpy_to_latex() (*in module Irene.matrices*), 173
nvars (*Irene.border_basis.BorderBasis attribute*), 143

## O

objective (*Irene.program.OptimizationProblem attribute*), 125
objective_degree (*Irene.program.OptimizationProblem attribute*), 126
objective_half_degree (*Irene.program.OptimizationProblem attribute*), 126
objective_terms_with_even_exponent (*Irene.program.OptimizationProblem attribute*), 126
objective_terms_with_odd_exponent (*Irene.program.OptimizationProblem attribute*), 126
objective_trms_with_negative_coefficient (*Irene.program.OptimizationProblem attribute*), 126
objective_trms_with_positive_coefficient (*Irene.program.OptimizationProblem attribute*), 126
objective_value (*Irene.sparse_moment.SparseMomentResult attribute*), 157
omega() (*Irene.program.OptimizationProblem static method*), 131
OmegaFtilde() (*Irene.invariant.InvariantPolynomial method*), 174
OptimizationProblem (*class in Irene.program*), 125
Option() (*Irene.cvxpy_solver.CvxpySDPSolver method*), 167
Option() (*Irene.sdp.sdp method*), 123
OrthSystem (*class in pyProximation.orthsys*), 113

## P

parse_solution_matrix() (*Irene.sdp.sdp static method*), 124
phase (*Irene.telemetry.TelemetryRecord attribute*), 172
PhaseTiming (*class in Irene.telemetry*), 171
pInitSDP() (*Irene.relaxations.SDPRelaxations method*), 121
Pivot() (*Irene.relaxations.SDRelaxSol method*), 123
Poly() (*Irene.symbolic_engine.SymbolicEngine method*), 165
PolyBasis() (*pyProximation.orthsys.OrthSystem method*), 114
PolyCoefFullVec() (*Irene.relaxations.SDPRelaxations method*), 120
PolyMatrix() (*Irene.symbolic_engine.SymbolicEngine method*), 165
positive_exp() (*Irene.grouprings.CommutativeSemigroup method*), 137
program_degree() (*Irene.program.OptimizationProblem method*), 131
project() (*pyProximation.orthsys.OrthSystem method*), 114
prolongations() (*in module Irene.sparse_moment*), 161
prune_basis_from_polys() (*in module Irene.newton_polytope*), 148
prune_basis_from_problem() (*in module Irene.newton_polytope*), 148
pruned_basis_size (*Irene.newton_polytope.NewtonPruner attribute*), 146
pyProximation.base module, 113
pyProximation.interpolation module, 115
pyProximation.measure module, 113
pyProximation.orthsys module, 113
pyProximation.rational module, 114

## Q

QOmega() (*Irene.invariant.InvariantPolynomial method*), 174
QQ (*Irene.symbolic_engine.SymbolicEngine property*), 165
quotient_basis (*Irene.sparse_moment.Algorithm1Result attribute*), 153

## R

r (*Irene.sparse_moment.ToricSetup attribute*), 159
rank_test (*Irene.sparse_moment.Algorithm1Result attribute*), 153
ranks (*Irene.sparse_moment.RankTestResult attribute*), 154
RankTestResult (*class in Irene.sparse_moment*), 154
RationalApprox (*in module pyProximation.rational*), 114
RationalAprox (*class in pyProximation.rational*), 114
RatLSQ() (*pyProximation.rational.RationalAprox method*), 114
read_csdp_out() (*Irene.sdp.sdp method*), 124
read_sdpa_out() (*Irene.sdp.sdp method*), 124
recover_all_sparse_roots() (*in module Irene.sparse_moment*), 161

RedPart() (*Irene.invariant.InvariantPolynomial method*), 174
reduce() (*Irene.border_basis.BorderBasis method*), 143
reduced() (*Irene.symbolic_engine.SymbolicEngine method*), 165
ReducedMonomialBase() (*Irene.relaxations.SDPRelaxations method*), 120
ReduceExp() (*Irene.relaxations.SDPRelaxations method*), 120
reduction_factor() (*Irene.sparsity.CorrelativeSparsity method*), 145
reduction_method (*Irene.relaxations.RelaxationConfig attribute*), 118
reduction_ratio (*Irene.newton_polytope.NewtonPruner attribute*), 146
relations (*Irene.program.OptimizationProblem attribute*), 125
relax() (*in module Irene.relaxation_api*), 151
RelaxationConfig (*class in Irene.relaxations*), 118
RelaxationDeg() (*Irene.relaxations.SDPRelaxations method*), 120
RelaxationEngine (*class in Irene.relaxation_api*), 150
RelaxMethod (*class in Irene.relaxation_api*), 149
RelaxResult (*class in Irene.relaxation_api*), 149
rels (*Irene.grouprings.CommutativeSemigroup attribute*), 135
Resume() (*Irene.relaxations.SDPRelaxations method*), 120
Reynolds() (*Irene.invariant.InvariantPolynomial method*), 174
rho_A (*Irene.sparse_moment.ToricSetup attribute*), 159
roots (*Irene.sparse_moment.SparseRootsResult attribute*), 158
runtime (*Irene.relaxation_api.RelaxResult attribute*), 150
runtime (*Irene.sosonc.SOSONCRelaxSol attribute*), 141

## S

s_star (*Irene.sparse_moment.RankTestResult attribute*), 154
sample() (*pyProximation.measure.Measure method*), 113
SaveState() (*Irene.relaxations.SDPRelaxations method*), 120
scale_polytope() (*in module Irene.newton_polytope*), 149
sdp (*class in Irene.sdp*), 123
sdpa() (*Irene.sdp.sdp method*), 125
sdpa_param() (*Irene.sdp.sdp method*), 125
SDPRelaxations (*class in Irene.relaxations*), 119
SDPResult (*class in Irene.cvxpy_solver*), 168
SDRelaxSol (*class in Irene.relaxations*), 122
semigroup (*Irene.grouprings.AtomicSGElement attribute*), 133
semigroup (*Irene.grouprings.SemigroupAlgebra attribute*), 137
semigroup (*Irene.grouprings.SemigroupAlgebraElement attribute*), 138
semigroup (*Irene.program.OptimizationProblem attribute*), 125
semigroup_level_map() (*in module Irene.sparse_moment*), 162
SemigroupAlgebra (*class in Irene.grouprings*), 137
SemigroupAlgebraElement (*class in Irene.grouprings*), 138
Series() (*pyProximation.orthsys.OrthSystem method*), 114
set() (*Irene.telemetry.TelemetryContext method*), 172
set_backend() (*Irene.symbolic_engine.SymbolicEngine method*), 165
set_derivation() (*Irene.dsdp.DSDPRelaxations method*), 170
set_objective() (*Irene.program.OptimizationProblem method*), 131
set_symbolic_backend() (*in module Irene.symbolic_engine*), 166
SetMeasure() (*pyProximation.orthsys.OrthSystem method*), 114
SetMonoOrd() (*Irene.relaxations.SDPRelaxations method*), 120
SetNumCores() (*Irene.relaxations.SDPRelaxations method*), 120
SetObjective() (*Irene.cvxpy_solver.CvxpySDPSolver method*), 167
SetObjective() (*Irene.relaxations.SDPRelaxations method*), 121
SetObjective() (*Irene.sdp.sdp method*), 124
SetOrthBase() (*pyProximation.orthsys.OrthSystem method*), 114
SetScipySolver() (*Irene.relaxations.SDRelaxSol method*), 123
SetSDPSolver() (*Irene.relaxations.SDPRelaxations method*), 121
sga (*Irene.program.OptimizationProblem attribute*), 125
SigmaAlpha() (*Irene.invariant.InvariantPolynomial method*), 174
sInitSDP() (*Irene.relaxations.SDPRelaxations method*), 122
solve() (*Irene.cvxpy_solver.CvxpySDPSolver method*), 167
solve() (*Irene.dsdp.DSDPRelaxations method*), 170

solve() (*Irene.geometric.GPRelaxations method*), 140
solve() (*Irene.relaxation_api.RelaxationEngine method*), 151
solve() (*Irene.sdp.sdp method*), 125
solve() (*Irene.sonc.SONCRelaxations method*), 140
solve() (*Irene.sparse_moment.SparseMomentSDP method*), 158
solve_algorithm1() (*in module Irene.sparse_moment*), 162
solve_kkt() (*Irene.dsdp.DSDPKKTRelaxation method*), 168
solve_mean() (*Irene.dsdp.DSDPMeanRelaxation method*), 169
solve_sparse_real_roots() (*in module Irene.sparse_moment*), 163
solver_info (*Irene.relaxation_api.RelaxResult attribute*), 150
solver_info (*Irene.sparse_moment.SparseMomentResult attribute*), 157
SONCRelaxations (*class in Irene.sonc*), 140
sosonc_bounds() (*in module Irene.sosonc*), 142
SOSONCRelaxations (*class in Irene.sosonc*), 141
SOSONCRelaxSol (*class in Irene.sosonc*), 140
sp_auxsyms (*Irene.dsdp.DSDPRelaxations property*), 171
SparseBorderBasis (*class in Irene.sparse_moment*), 155
SparseMomentResult (*class in Irene.sparse_moment*), 156
SparseMomentSDP (*class in Irene.sparse_moment*), 157
SparseRootsResult (*class in Irene.sparse_moment*), 158
sparsity_block_sdp (*Irene.relaxations.RelaxationConfig attribute*), 118
sparsity_detection (*Irene.relaxations.RelaxationConfig attribute*), 118
sqrt() (*Irene.symbolic_engine.SymbolicEngine method*), 166
square_exponent() (*Irene.program.OptimizationProblem static method*), 132
Stabilizer() (*Irene.invariant.InvariantPolynomial method*), 174
State() (*Irene.relaxations.SDPRelaxations method*), 121
status (*Irene.relaxation_api.RelaxResult attribute*), 150
status (*Irene.sosonc.SOSONCRelaxSol attribute*), 141
status (*Irene.sparse_moment.SparseMomentResult attribute*), 157
StblRedEch() (*Irene.relaxations.SDRelaxSol method*), 123
StblTldMax() (*Irene.invariant.InvariantPolynomial method*), 174
success (*Irene.relaxation_api.RelaxResult property*), 150
summary() (*Irene.newton_polytope.NewtonPruner method*), 147
summary() (*Irene.sparsity.CorrelativeSparsity method*), 145
symbol (*Irene.grouprings.AtomicSGElement attribute*), 134
Symbol() (*Irene.symbolic_engine.SymbolicEngine method*), 165
SymbolicEngine (*class in Irene.symbolic_engine*), 164
symbols (*Irene.grouprings.CommutativeSemigroup attribute*), 136
symbols() (*Irene.symbolic_engine.SymbolicEngine method*), 166
sympify() (*Irene.symbolic_engine.SymbolicEngine method*), 166

## T

t (*Irene.sparse_moment.Algorithm1Result attribute*), 153
tables (*Irene.sparse_moment.SparseBorderBasis attribute*), 155
TelemetryContext (*class in Irene.telemetry*), 171
TelemetryRecord (*class in Irene.telemetry*), 172
TensorPrd() (*pyProximation.orthsys.OrthSystem method*), 114
Term2Mmnt() (*Irene.relaxations.SDRelaxSol method*), 123
test_theorem_327() (*in module Irene.sparse_moment*), 163
tldemax() (*Irene.invariant.InvariantPolynomial method*), 174
timed() (*in module Irene.telemetry*), 172
timings (*Irene.telemetry.TelemetryRecord attribute*), 172
to_info_dict() (*Irene.cvxpy_solver.SDPResult method*), 168
to_symengine() (*in module Irene.symbolic_engine*), 166
to_sympy() (*in module Irene.symbolic_engine*), 166
to_sympy() (*Irene.program.OptimizationProblem method*), 132
toric_ideal() (*in module Irene.sparse_moment*), 164
ToricSetup (*class in Irene.sparse_moment*), 159
total_degree (*Irene.program.OptimizationProblem attribute*), 127
transform_program() (*Irene.geometric.GPRelaxations method*), 140

trials (*Irene.sparse_moment.SparseRootsResult attribute*), 159
tuple2mono() (*Irene.program.OptimizationProblem method*), 133

## U

UnionFind (*class in Irene.sparsity*), 145
use_symengine (*Irene.symbolic_engine.SymbolicEngine attribute*), 164

## V

val (*Irene.sosonc.SOSONCRelaxSol attribute*), 140
value (*Irene.relaxation_api.RelaxResult attribute*), 149
VEC() (*Irene.sdp.sdp static method*), 124
verbose_reduction (*Irene.relaxations.RelaxationConfig attribute*), 118
vertices (*Irene.grouprings.CommutativeSemigroup attribute*), 136

## W

wall_time (*Irene.sparse_moment.SparseMomentResult attribute*), 157
which() (*Irene.base.base static method*), 117
write_sdpa_dat() (*Irene.sdp.sdp method*), 125
write_sdpa_dat_sparse() (*Irene.sdp.sdp method*), 125

## Z

z (*Irene.sparse_moment.ToricSetup attribute*), 159
zeros() (*Irene.symbolic_engine.SymbolicEngine method*), 166